# Multi-Line InsurancePolicy and Claims ManagementSystem

## User Story:

Multi-Line Insurance Policy and Claims Management System is designed to provide a centralized Salesforce-based solution for managing Vehicle, Property, and Life Insurance policies and claims. The system enables insurance agents to create and manage policy information using a flexible data model with policy-specific Record Types and Field Sets. Guided Screen Flows simplify policy quoting, while Apex automation calculates premiums based on policy and vehicle details. Claims are automatically routed to the appropriate queue based on policy type, and high-value claims are processed through an approval workflow involving Senior Adjusters and Claims Managers. An LWC-based Claims Dashboard provides adjusters with a focused view of their assigned claims, including claim amount, policy type, policy holder, status, and days open. This initiative improves policy processing efficiency, claim handling, data accuracy, operational visibility, and customer service.

## 1. INTRODUCTION

### 1.1 Project Overview:

The Multi-Line Insurance Policy and Claims Management System is a Salesforce-based solution designed to centralize and streamline the management of Vehicle, Property, and Life Insurance policies and claims. The system replaces fragmented and manual processes with structured Salesforce objects, Record Types, Field Sets, Flows, Apex automation, Approval Processes, and Lightning Web Components.

The solution enables insurance agents to create and manage policies through a flexible data model, while guided Screen Flows simplify the policy quoting process. The system validates policy information and uses Apex automation to calculate premiums based on factors such as state and model year. Once a claim is created, automation identifies the related policy type and routes the claim to the appropriate claims queue.

For claims adjusters, an LWC-based Claims Dashboard provides visibility into assigned claims, including claim number, policy type, policy holder, claim amount, approval status, and days open. High-value claims exceeding the defined threshold are processed through a structured approval workflow involving Senior Adjusters and Claims Managers. Security and access are managed using sharing rules and role-specific Permission Sets.

Overall, the project provides a centralized platform for improving policy processing, premium calculation, claim routing, claim approval, data accuracy, operational visibility, and customer service across multiple insurance lines.

## 1.2 Purpose:

The purpose of the Multi-Line Insurance Policy and Claims Management System is to provide a centralized Salesforce-based platform for managing Vehicle, Property, and Life Insurance policies and claims. The system is designed to overcome the limitations of legacy processes that result in manual, slow, and inconsistent quoting, policy issuance, and claims handling.

The solution aims to standardize and accelerate policy quoting and issuance, automate claim initiation and routing based on policy type and claim severity, and provide a 360-degree view of Customer, Policy, and Claim information. It also supports guided quoting through Screen Flows, automated premium and claim processing, data validation, and an LWC-based dashboard to improve claims adjuster efficiency.

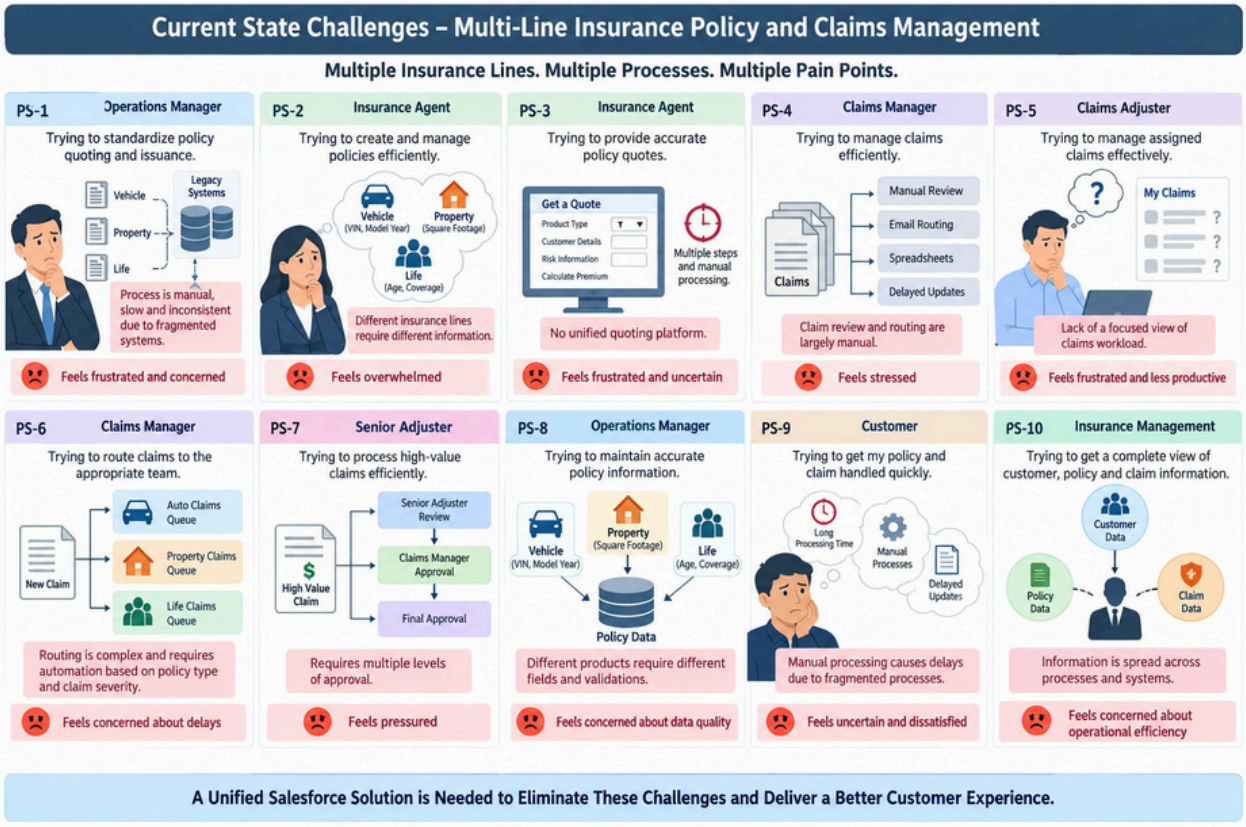
Overall, the project focuses on improving operational efficiency, data accuracy, claims processing, and customer experience by bringing multiple insurance lines into a unified Salesforce solution.

# 2. IDEATION PHASE

## 2.1 Problem Statement (Customer Problem Statement Template)

| PS | I am (Customer) | I'm trying to | But | Because | Which makes me feel |
|---|---|---|---|---|---|
| PS-1 | Insurance Operations Manager | standardize policy quoting and issuance | the process is manual, slow, and inconsistent | legacy systems and processes are fragmented | frustrated and concerned |
| PS-2 | Insurance Agent | create and manage policies efficiently | different insurance lines require different information | Vehicle, Property, and Life policies have diverse data requirements | overwhelmed |
| PS-3 | Insurance Agent | provide accurate policy quotes | quoting requires multiple steps and manual processing | there is no unified quoting platform | frustrated and uncertain |
| PS-4 | Claims Manager | manage claims efficiently | claim review and routing are largely manual | claims processes are not centralized or automated | stressed |

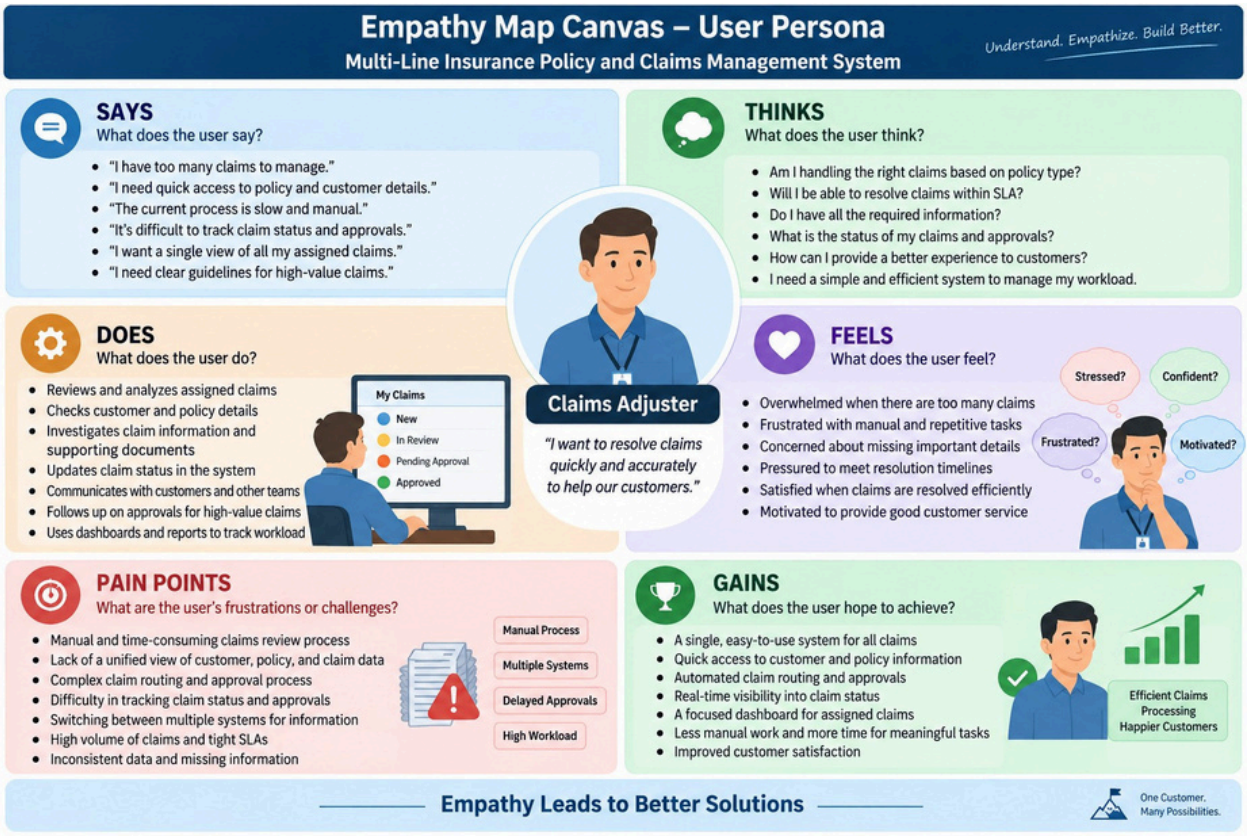
| | | | | | |
|---|---|---|---|---|---|
| PS-5 | Claims Adjuster | manage assigned claims effectively | I lack a focused view of my claims workload | claim information is not easily available in one place | frustrated and less productive |
| PS-6 | Claims Manager | route claims to the appropriate team | claims need to be handled according to policy type and claim severity | routing processes are complex and require automation | concerned about delays |
| PS-7 | Senior Adjuster | process high-value claims efficiently | approval requires multiple levels of review | high-value claims require structured approval and escalation | pressured |
| PS-8 | Insurance Operations Manager | maintain accurate policy information | different products require different fields and validations | Vehicle, Property, and Life policies have varying data requirements | concerned about data quality |
| PS-9 | Customer | get my policy and claim handled quickly | manual processing causes delays | policy and claims processes are fragmented | uncertain and dissatisfied |
| PS-10 | Insurance Management | get a complete view of customer, policy, and claim information | information is spread across processes and systems | there is no unified platform | concerned about operational efficiency |




## 2.2 Empathy Map Canvas

**User Persona:** Insurance Claims Adjuster

- **Think & Feel:** Concerned about claim delays and workload
- **Hear:** Customer and team concerns about claim resolution
- **See:** Multiple claims, policy details, and approval statuses
- **Say & Do:** Reviews claims and follows up on approvals
- **Pain:** Manual processing, complex routing & limited visibility
- **Gain:** Claims dashboard, automation & faster approvals




## 2.3 Brainstorming & Idea Prioritization

### Ideas Considered

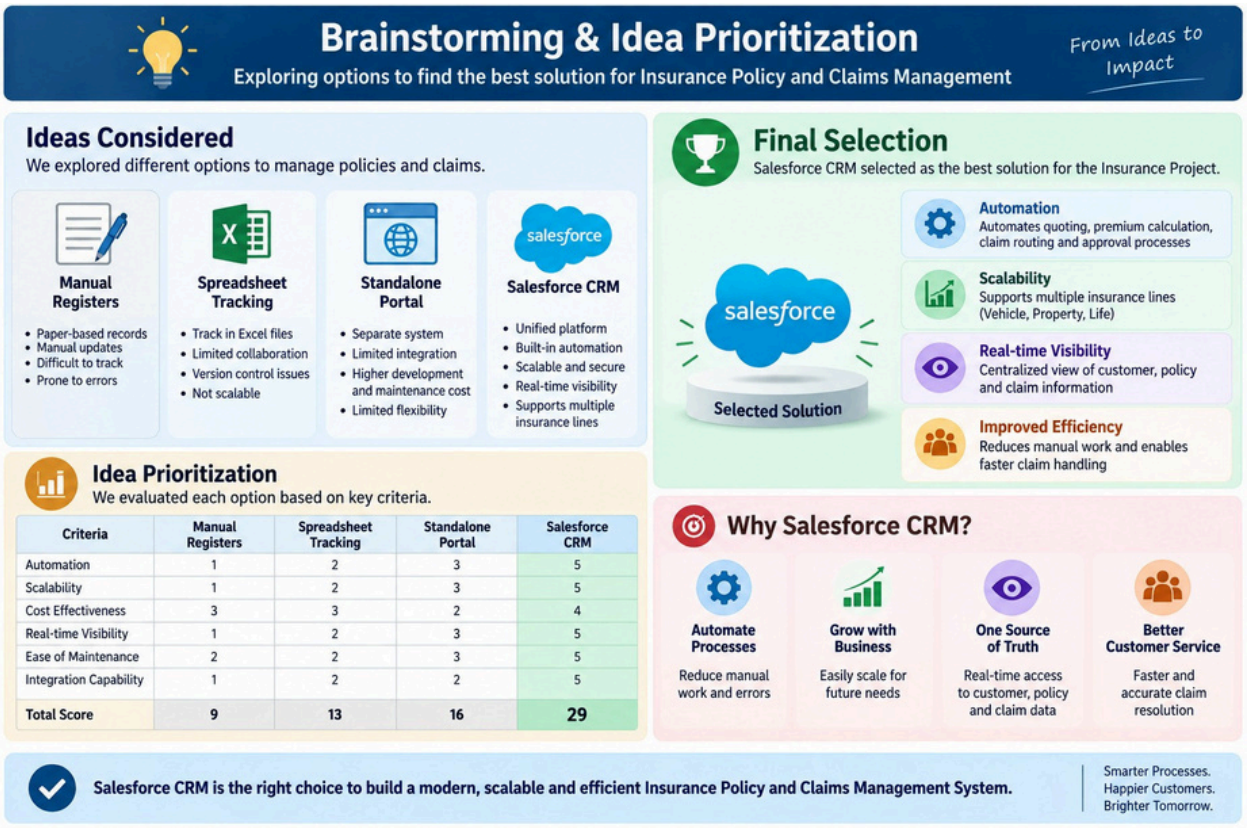
- Manual Registers – Policy and claim information maintained manually
- Spreadsheet Tracking – Policies, claims, and approvals tracked using spreadsheets
- Standalone Portal – Separate portal for policy and claim management
- Salesforce CRM – Unified platform for policies, claims, automation, and reporting

### Final Selection

Salesforce CRM selected due to:

- Automation – Automates quoting, premium calculation, claim routing, and approval processes
- Scalability – Supports multiple insurance lines including Vehicle, Property, and Life Insurance

- Real-Time Visibility – Provides centralized visibility into customer, policy, and claim information
- Unified Platform – Brings policy and claims processes together in one system
- Improved Efficiency – Reduces manual processing and supports faster claim handling



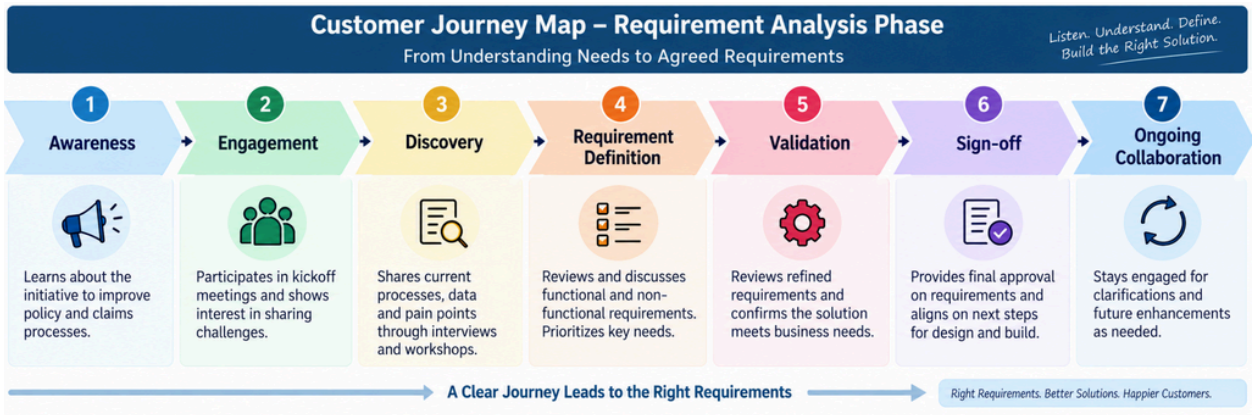

# 3.REQUIREMENT ANALYSIS PHASE

## 3.1 Customer Journey Map

Stages:

- Insurance Inquiry
- Policy Selection – Vehicle / Property / Life
- Quote Request

- Policy Information Capture
- Premium Calculation
- Policy Issuance
- Claim Initiation
- Claim Routing
- Claim Review & Approval
- Claim Resolution
- Customer Confirmation
- Reporting & Visibility




## 3.2 Solution Requirements

### Functional Requirements

| FR No | Functional Requirement |
|---|---|
| FR-01 | The system shall allow users to create and manage **Insurance Policy** records for Vehicle, Property, and Life Insurance. |

| | |
|---|---|
| **FR-02** | The system shall support separate **Record Types** for Vehicle, Property, and Life policies. |
| **FR-03** | The system shall capture **product-specific policy information** such as VIN for Vehicle Insurance and Square Footage/Year Built for Property Insurance. |
| **FR-04** | The system shall provide a guided **Auto Quoting Screen Flow** to capture policy holder, policy start date, state, VIN, and model year. |
| **FR-05** | The system shall create a **draft policy** during the quoting process. |
| **FR-06** | The system shall calculate policy premiums using **Apex-based premium calculation logic**. |
| **FR-07** | The system shall validate Vehicle Insurance VINs and ensure that the VIN contains exactly **17 characters**. |
| **FR-08** | The system shall allow users to create and manage **Claim** records associated with policies. |
| **FR-09** | The system shall support different Claim Record Types for **Accident, Property, and Life** claims. |
| **FR-10** | The system shall automatically **route claims to the appropriate queue** based on the related policy type. |
| **FR-11** | The system shall identify **high-value claims** based on the configured claim amount threshold. |
| **FR-12** | The system shall submit qualifying high-value claims through a **multi-level approval process** involving Senior Adjusters and Claims Managers. |
| **FR-13** | The system shall provide an **Approve/Reject Claim** capability with approver comments. |
| **FR-14** | The system shall provide Claims Adjusters with an **LWC Claims Dashboard** showing assigned claims. |

| | |
|---|---|
| **FR-15** | The dashboard shall display information such as **Claim Number, Policy Type, Policy Holder, Claim Amount, Status, and Days Open**. |
| **FR-16** | The dashboard shall allow users to filter claims by **All, Auto, Property, and Life**. |
| **FR-17** | The system shall provide automated updates to related Policy and Claim records through **Record-Triggered Flows**. |
| **FR-18** | The system shall support **state-based access/sharing** for applicable insurance users and records. |
| **FR-19** | The system shall provide role-specific access through **Permission Sets** for Insurance Agents, Claims Adjusters, and Claims Managers. |
| **FR-20** | The system shall support reporting and visibility of **Customer, Policy, and Claim information** from the centralized Salesforce platform. |

**Non-Functional Requirements**

The following requirements describe the expected quality, usability, security, scalability, and maintainability of the solution. Some are derived requirements based on the project's stated objectives and challenges rather than explicitly documented as formal NFRs.

| ID | Non-Functional Requirement | Description |
|---|---|---|
| **NFR-01** | Usability | The system should provide simple, guided Screen Flows that minimize unnecessary clicks and make policy quoting easy for agents. |
| **NFR-02** | Performance | Policy quoting, premium calculation, claim routing, and dashboard operations should execute efficiently to support day-to-day insurance operations. |

| ID | Non-Functional Requirement | Description |
|---|---|---|
| **NFR-03** | Scalability | The solution should support the three insurance lines—Vehicle, Property, and Life—and allow additional insurance products to be accommodated in the future. |
| **NFR-04** | Data Integrity | Policy and claim information should be validated to maintain accurate and consistent data across insurance products. |
| **NFR-05** | Security | Access to insurance information should be controlled according to the user's responsibility, including state-based sharing and role-specific Permission Sets. |
| **NFR-06** | Maintainability | The solution should use modular Salesforce components such as Flows, Apex classes, LWC components, Record Types, and Field Sets so that individual components can be maintained independently. |
| **NFR-07** | Reliability | Automated claim routing, premium calculation, and approval processes should consistently execute according to the configured business rules. |
| **NFR-08** | Extensibility | The data model should accommodate different product-specific fields without requiring completely separate structures for each insurance line. |
| **NFR-09** | Visibility | Users should have timely access to relevant Customer, Policy, and Claim information through centralized Salesforce records and the Claims Dashboard. |
| **NFR-10** | User Adoption | The application should provide an intuitive experience that reduces manual effort and supports efficient adoption by agents and claims users. |

## 3.3 Data Flow Diagram

Flow:

Customer / Agent→Screen Flow → Policy Validation → Policy__c → PremiumCalculator→Claim__c → Claim Routing Flow → Claims Queue → Claims Adjuster → ApprovalProcess → Claim Update → LWC Dashboard / Reports

**Main Data Flow**

Step 1 – Customer/AgentInput

→ Customer andpolicyinformation is provided by the Insurance Agent.

Step 2 – PolicyDataCapture

→ AutoQuotingScreenFlow captures policy holder, policy start date, state, VIN, and model year.

Step 3 – Policy Validation

→ Salesforce validates the entered information, including the 17-character VIN requirement for Auto policies.

Step 4 – Policy Creation

→ Valid information is stored in the Policy__c object with the appropriate Vehicle, Property, or Life Record Type.

Step 5 – Premium Calculation

→ Policy information is passed to PremiumCalculator Apex to calculate the applicable premium.

Step 6 – Claim Creation

→ When a claim occurs, the user creates a Claim__c record linked to the relevant Policy.

Step 7 – Claim Routing

→ A Record-Triggered Flow identifies the policy type and routes the claim to the appropriate Auto, Property, or Life Claims Queue.

Step 8 – Claim Review

→ The Claims Adjuster accesses assigned claims through the Claims Dashboard and reviews policy and claim information.

Step 9 – High-Value Claim Approval

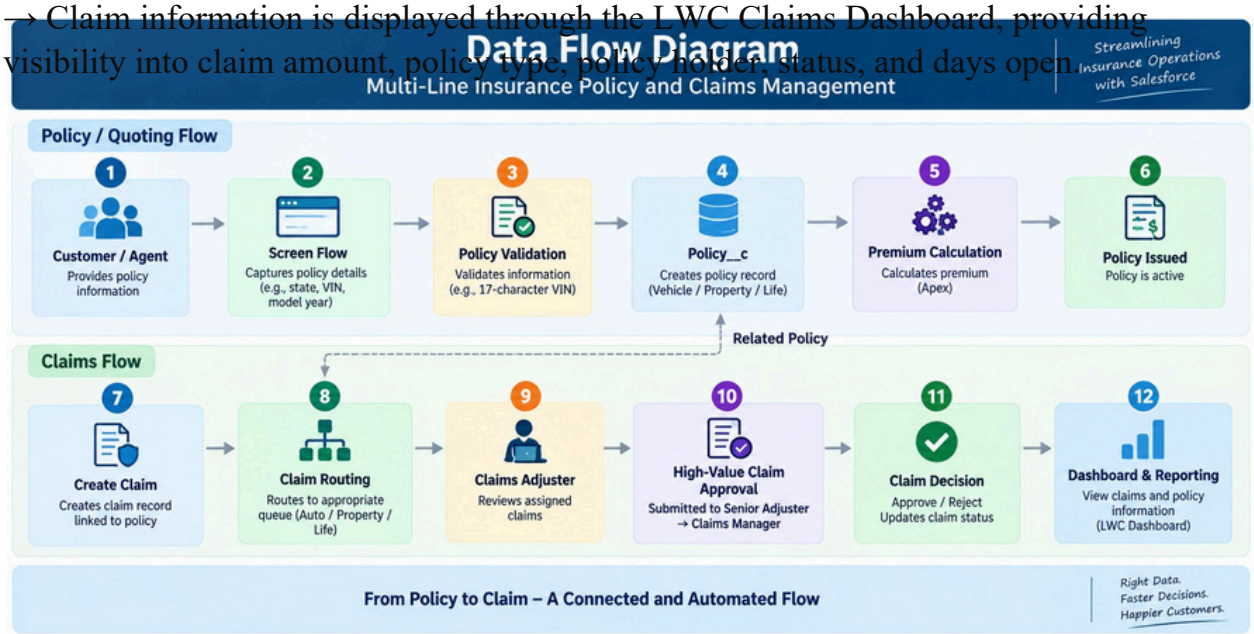
→ Claims exceeding the configured threshold are submitted for approval to the Senior Adjuster → Claims Manager.

Step 10 – Claim Decision & Update

→ The approver selects Approve/Reject and provides comments; the claim status is updated.

Step 11 – Dashboard & Reporting

→ Claim information is displayed through the LWC Claims Dashboard, providing visibility into claim amount, policy type, policy holder, status, and days open.



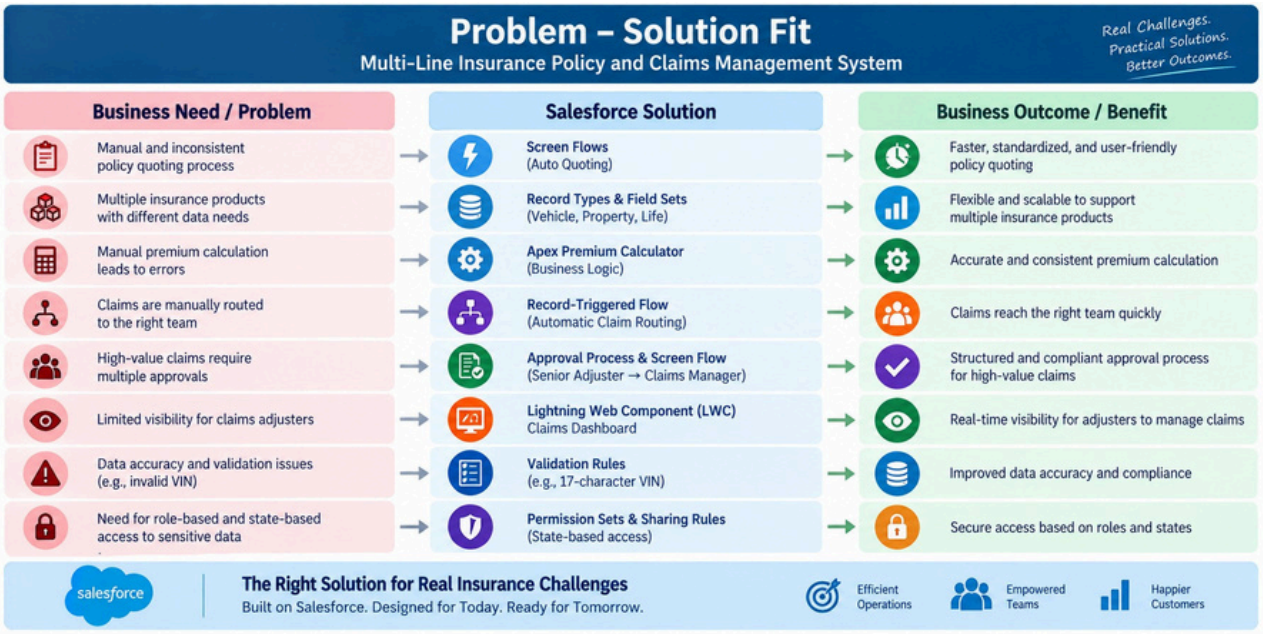
### 3.4 Technology Stack

| Layer | Technology |
|---|---|
| UI | Salesforce Lightning, Lightning Web Components (LWC) |
| Logic | Salesforce Flows, Apex |
| Database | Salesforce Custom Objects (Policy__c, Claim__c), Contact, User |
| Automation | Record-Triggered Flows, Validation Rules, Approval Processes |
| Processing | Apex Premium Calculator, Apex Controllers |
| Data Configuration | Record Types, Field Sets, Custom Fields |
| Security | Permission Sets, State-Based Sharing Rules |
| Reporting | Salesforce Reports & Dashboards |
| Testing | Apex Test Classes |

# 4. PROJECT DESIGN PHASE

### 4.1 Problem–Solution Fit

Salesforce is a strong fit for the Multi-Line Insurance Policy and Claims Management System because it directly addresses the project's key challenges through configurable automation, flexible data management, secure access, and real-time visibility. The solution combines Screen Flows, Record Types, Field Sets, Apex, Record-Triggered Flows, Approval Processes, and LWC to support policy quoting, premium calculation, claim routing, claim approval, and adjuster operations.




## 4.2 Proposed Solution

### Functional Requirements

| Parameter | Description |
|---|---|
| Problem | Manual and inconsistent policy quoting, issuance, and claims handling |
| Solution | Salesforce CRM-based Insurance Policy & Claims Management System |
| Innovation | Automated quoting, premium calculation, claim routing, and high-value claim approval |
| Impact | Faster policy processing, efficient claim handling, and improved customer service |
| Business Model | Centralized management of Vehicle, Property, and Life Insurance |
| Scalability | Flexible data model supporting multiple insurance products and future expansion |

| | |
|---|---|
| Visibility | Centralized Customer, Policy, and Claim information with Claims Dashboard |
| Security | Role-based and state-based access using Permission Sets and Sharing Rules |

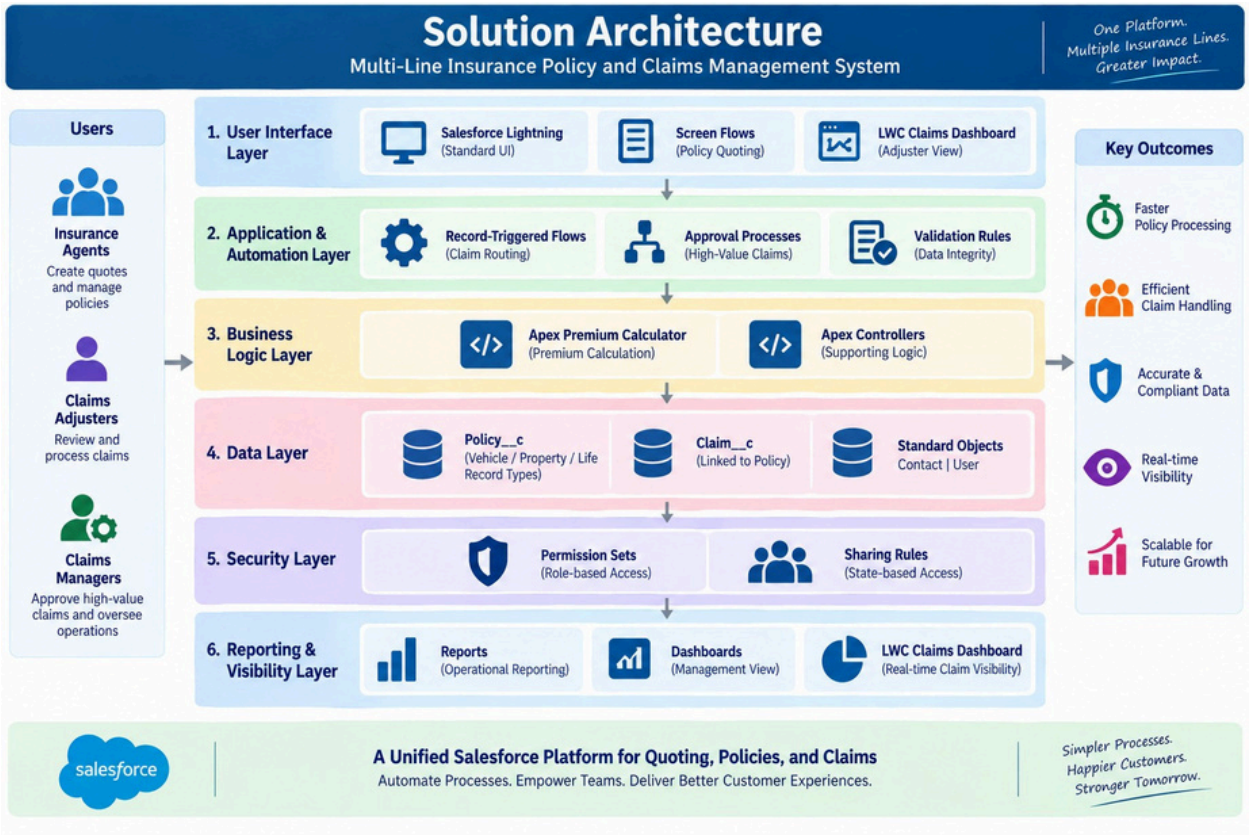
The proposed solution is aligned with the project's documented objective of creating a single Salesforce platform to standardize policy processes, automate claims, and provide a 360-degree view of Customer, Policy, and Claim data.

## 4.3 Solution Architecture

The Multi-Line Insurance Policy and Claims Management System uses a layered Salesforce architecture to manage policy quoting, premium calculation, claims processing, approvals, security, and reporting.

**Key Architecture Components**

| **Layer** | **Project Implementation** |
|---|---|
| User Layer | Insurance Agents, Claims Adjusters, Claims Managers |
| UI Layer | Salesforce Lightning, Screen Flows, LWC |
| Automation Layer | Record-Triggered Flows, Approval Processes, Validation Rules |
| Business Logic | Apex Premium Calculator and Claims Controller |
| Data Layer | Policy and Claim custom objects with Vehicle, Property, and Life Record Types |
| Security Layer | Permission Sets and state-based Sharing Rules |
| Reporting Layer | Claims Dashboard, Reports and Salesforce Dashboards |




# 5. PROJECT PLANNING & SCHEDULING

## 5.1 Project Planning

- Agile methodology
- Epics → Stories → Story Points
- Sprint-based execution
- Velocity calculation used for estimation

**Product Backlog, Sprint Schedule, and Estimation (4 Marks)**

Use the below template to create product backlog and sprint schedule

| Sprint | Functional Requirement (Epic) | User Story Number | User Story / Task | Story Points | Priority | Team Members |
|---|---|---|---|---|---|---|
| Sprint-1 | Data Modeling | US-01 | As an admin, I want to create Policy and Claim objects so that insurance policy and claim information can be stored and managed systematically. | 5 | High | Member1 |

| Sprint | Functional Requirement (Epic) | User Story Number | User Story / Task | Story Points | Priority | Team Members |
|---|---|---|---|---|---|---|
| Sprint-1 | Data Modeling | US-02 | As an admin, I want to configure Vehicle, Property, and Life Record Types so that different insurance products can be managed using a unified Policy structure. | 3 | High | Member2 |
| Sprint-1 | Data Modeling | US-03 | As an admin, I want to configure policy Field Sets and product-specific fields so that diverse insurance information can be captured efficiently. | 5 | High | Member3 |
| Sprint-1 | Quoting & Automation | US-04 | As an insurance agent, I want a guided Auto Quoting Screen Flow so that I can capture policy-specific information through a standardized quoting process. | 8 | High | Member4 |
| Sprint-2 | Validation | US-05 | As an insurance agent, I want VIN validation during vehicle quoting so that incorrect vehicle information can be prevented. | 3 | High | Member1 |
| Sprint-2 | Apex | US-06 | As an insurance system, I want PremiumCalculator Apex to calculate premiums so that premium values are generated accurately. | 8 | High | Member2 |
| Sprint-2 | Automation | US-07 | As an insurance agent, I want the premium calculation integrated with the quoting Flow so that the calculated premium is available during the quote process. | 5 | High | Member3 |
| Sprint-2 | Claims Automation | US-08 | As a claims manager, I want claims to be routed based on policy type so that claims are assigned to the appropriate support queue. | 8 | High | Member4 |
| Sprint-3 | Apex | US-09 | As a claims adjuster, I want a ClaimsAdjusterController to retrieve my assigned claims so that I can efficiently manage my claim workload. | 8 | High | Member1 |
| Sprint-3 | LWC Dashboard | US-10 | As a claims adjuster, I want a claims dashboard and reusable claim tile so that I can view and manage assigned claims efficiently. | 8 | High | Member2 |
| Sprint-4 | Approval | US-11 | As a claims manager, I want high-value claims to go through an approval process and approver Flow so that high-value claims receive appropriate review. | 8 | High | Member3 |
| Sprint-4 | Security & Testing | US-12 | As a system administrator, I want to implement tests, sharing rules, and Permission Sets so that the solution is validated and users receive appropriate access. | 8 | High | Member4 |

**Project Tracker, Velocity & Burndown Chart: (4 Marks)**

| Sprint | Total Story Points | Duration | Sprint Start Date | Sprint End Date (Planned) | Story Points Completed (as on Planned End Date) | Sprint Release Date (Actual) |
|---|---|---|---|---|---|---|
| Sprint-1 | 20 | 6Days | 24 Oct 2022 | 29 Oct 2022 | 20 | 29 Oct 2022 |
| Sprint-2 | 20 | 6Days | 31 Oct 2022 | 05 Nov 2022 | | |
| Sprint-3 | 20 | 6Days | 07 Nov 2022 | 12 Nov 2022 | | |
| Sprint-4 | 20 | 6Days | 14 Nov 2022 | 19 Nov 2022 | | |
| | | | | | | |
| | | | | | | |
| | | | | | | |
| | | | | | | |

# 6. Project Development Phase:

### What you'll learn

1. Data Modelling
2. Fields and Relationships
3. Lightning App Builder
4. Record Triggered Flows
5. Apex and Apex Triggers
6. Batch Apex
7. Scheduled Apex

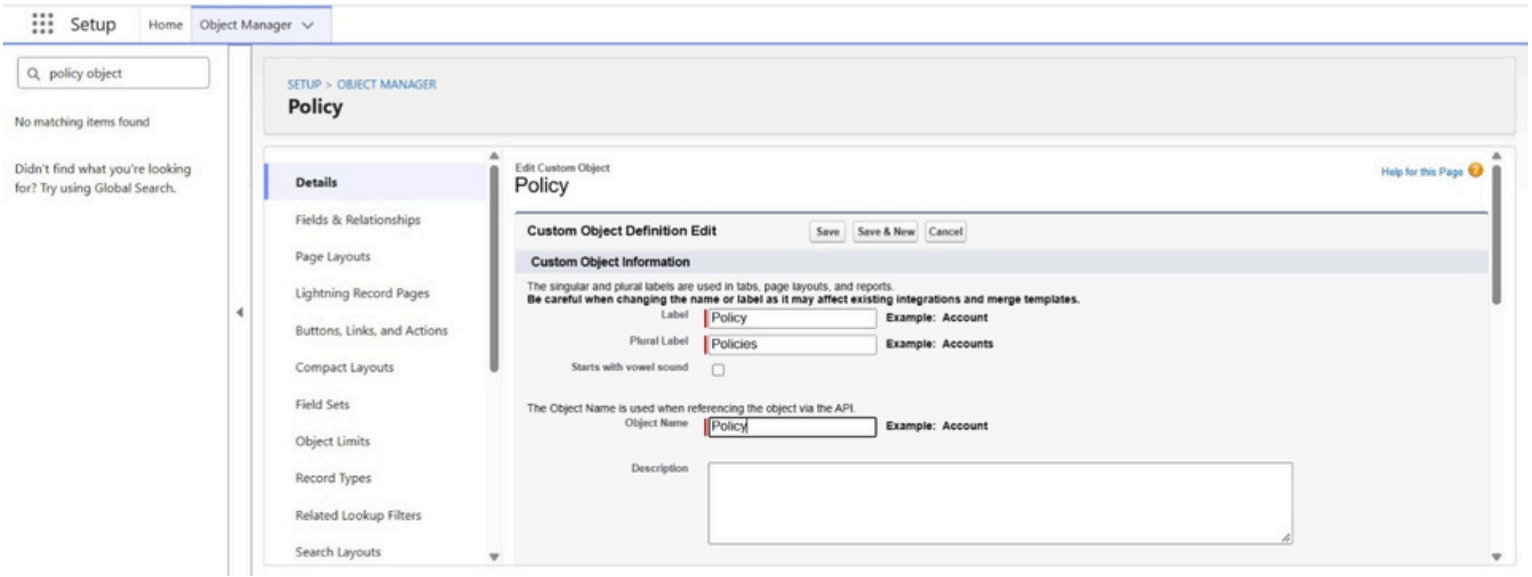
### Milestones

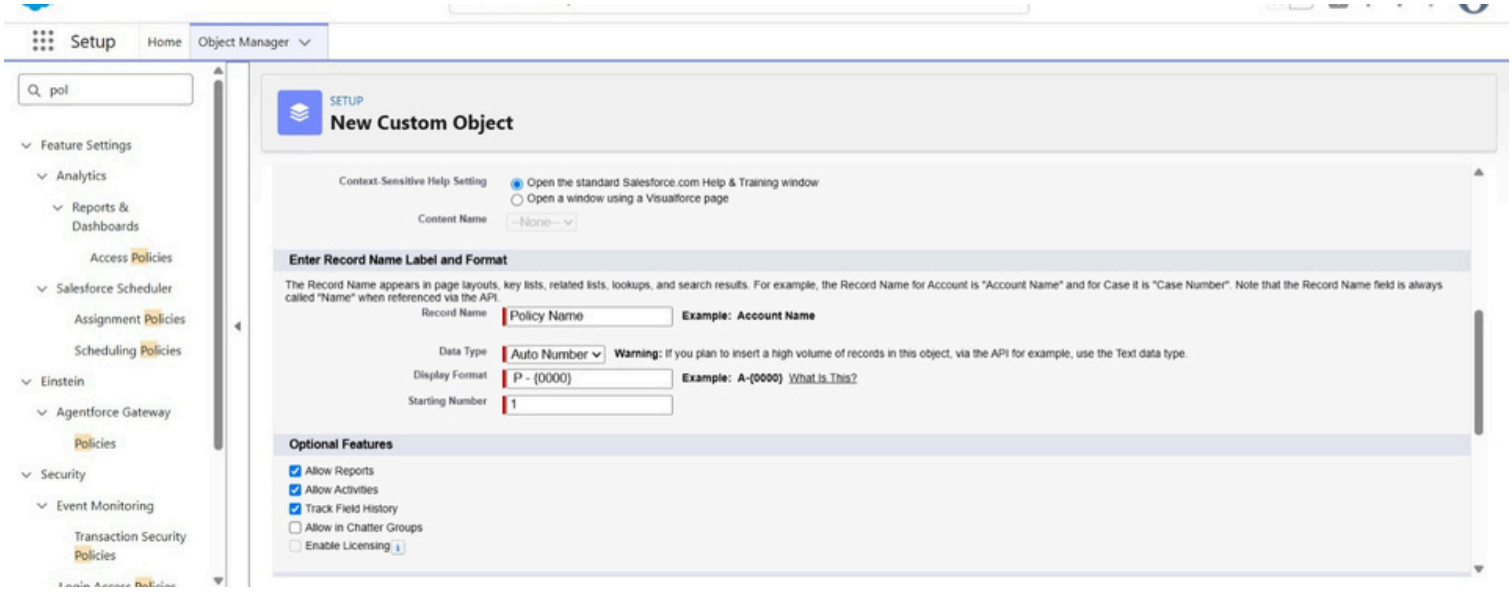
### Milestone 1: Core Data Model & Policy Configuration

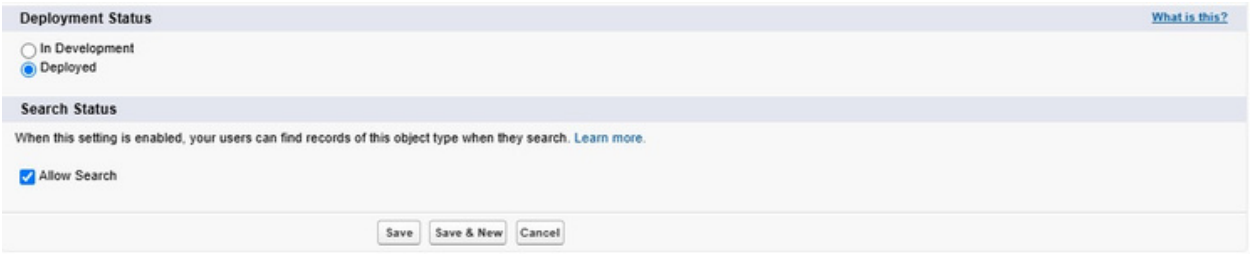
## User Story1: Creation of custom objects

### 1. Create Policy custom object:

- Go to setup, Click on object Manager.
- Click on create and select custom object.
- Fill the following details:
- Label: Policy
- Plural Label: Policies
- Object Name: Auto Populated
- Record Name: Policy Number
- Data type: Auto Number
- Format: P - {0000}
- Starting Number: 1
- Under optional features:
- Allow Reports, Activities and Track field History
- Under Deployment Status:
- Select the deployed radio button.
- Under Search Status:
- Check the Allow Search checkbox.
- Click on save.



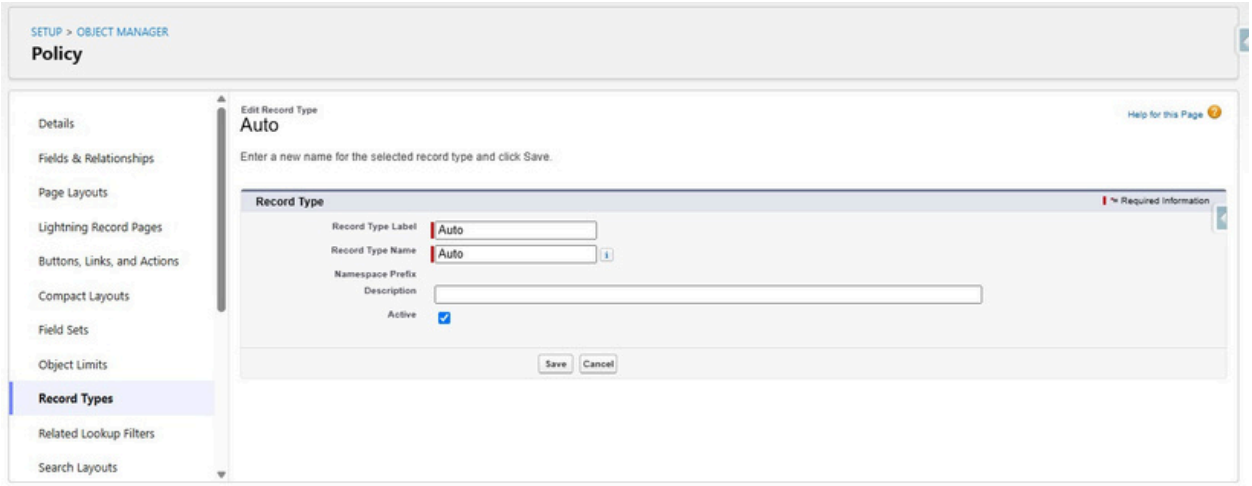



## 2. Define Record Types on Policy Object:

- Go to setup, Click on Object Manager.
- Search for Policy objects and click on Policy__c.
- In the left side panel, click on the record types.
- Click on New.
- Fill out the following details:
- Select the existing record type as Master.
- Give record type label as “Auto”
- Record Type Name will be auto populated.
- Give a meaningful description.
- Check the active checkbox.
- Check the Make Available checkbox for all the profiles.
- Click on Next.

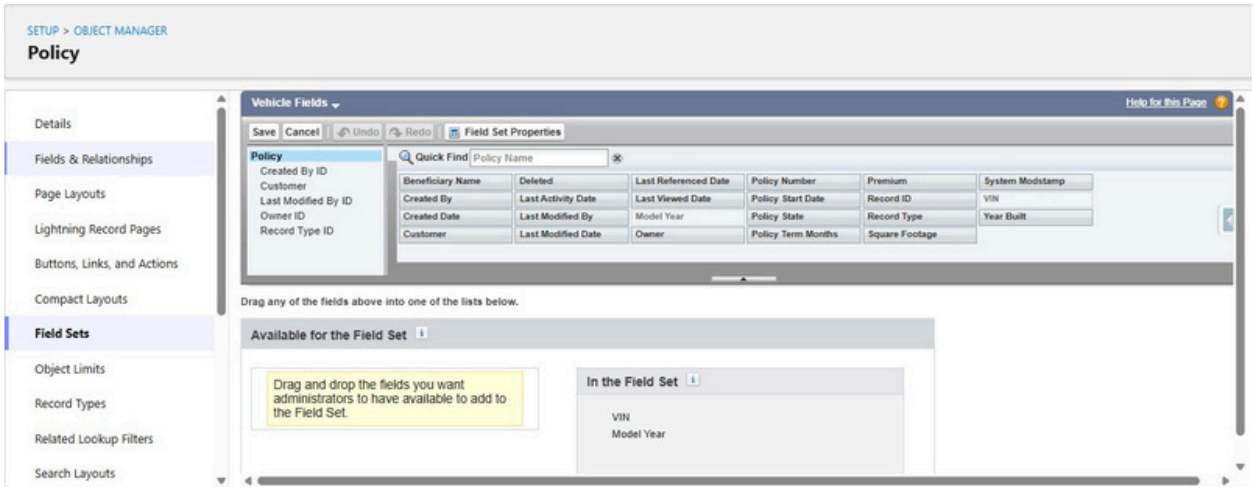
- It will redirect to the Assign page layouts page, Leave everything as is, and click on save.




**Repeatthe same steps for creating the property and life record types as well.**

### 3.Create Field Sets for policy specific data:

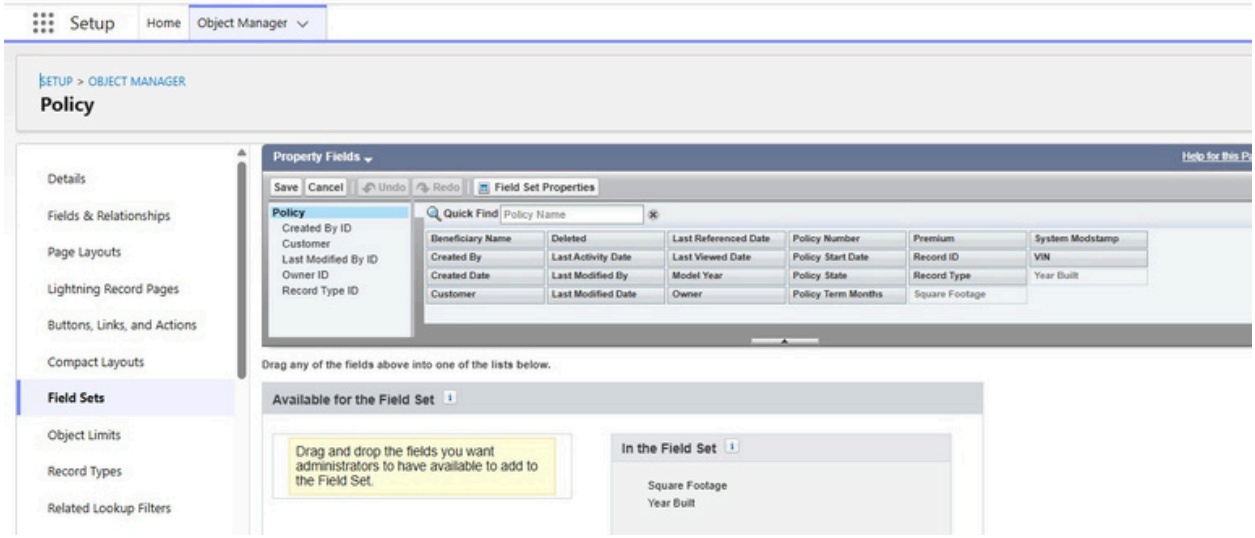
- Go to setup, Click on Object Manager.
- Search for Policy object and click on Policy__c.
- In the left side panel, click on the Field Sets.
- Click on New.
- Fill out the following details:
- Field Set Label as Vehicle
- Field Set Name will be Auto populated.
- Where is this used: In the policy object.
- Click on save.
- Click on the field set you have just created and it will redirect to the page layout setup.
- Drag VIN, and model year fields to the field set and click on save.




**Repeat the same steps for creating the property and life Field sets as well.**

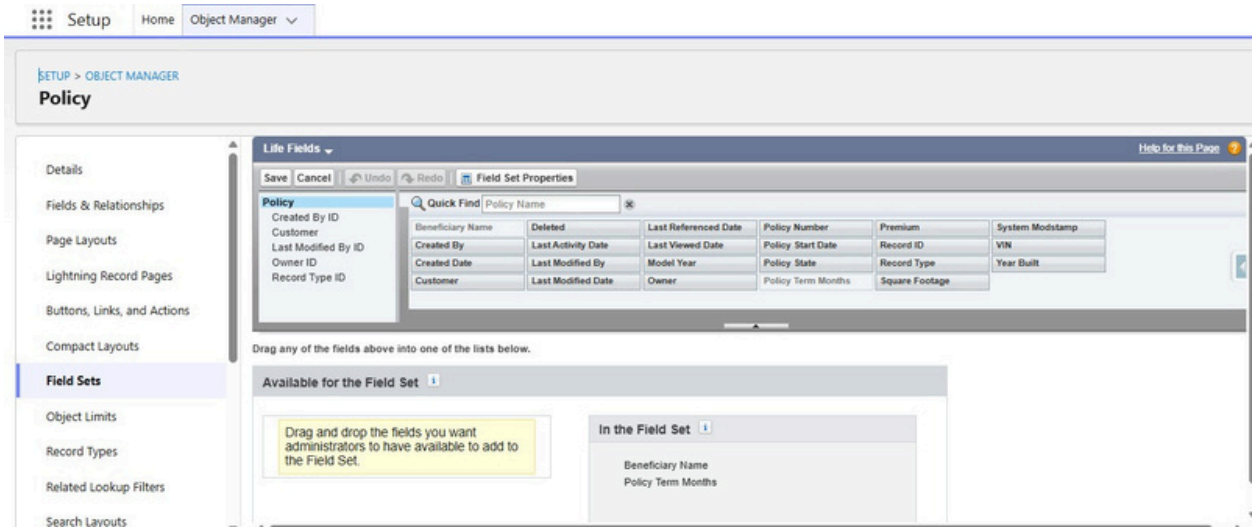
### 3.1 Property FieldSet Requires:

- Squarefootage
- Year Built




### 3.2 Life FieldSet Requires:

- Beneficiary Name
- Policy Term Months



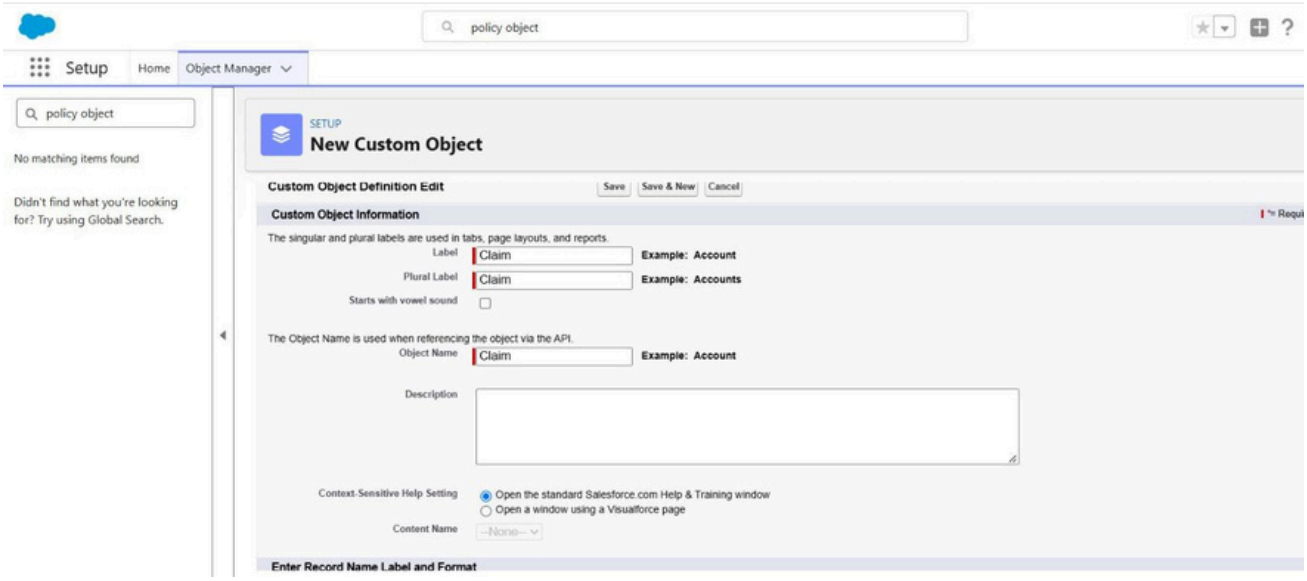

**4. Create Fields in the Policy Object:**

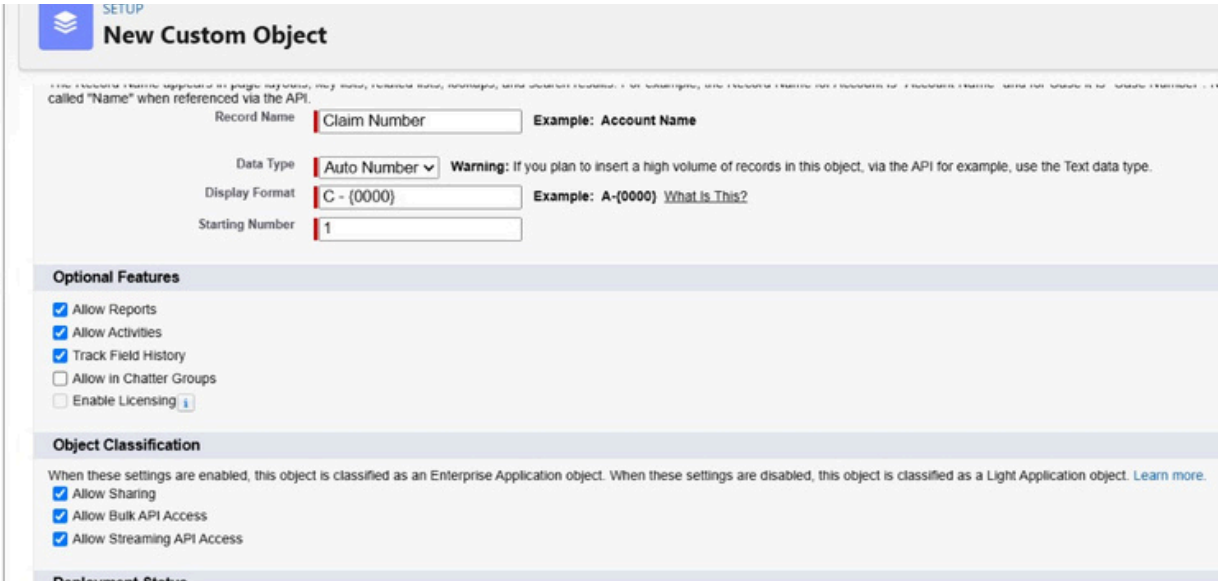
| S.No | Name of the Field | Data Type | Required |
|---|---|---|---|
| 1 | Beneficiary Name | Text(50) | |
| 2 | Customer | Lookup(Contact) | |
| 3 | Model Year | Text(5) | |
| 4 | Policy Start Date | Date | |
| 5 | Policy Term Months | Number(18, 0) | |
| 6 | Premium | Currency(16, 2) | |
| 7 | Square Footage | Number(18, 0) | |
| 8 | VIN | Text(20) | |
| 9 | Year Built | Text(5) | |

**5. Create Claim custom object:**

- Go to setup, Click on object Manager.
- Click on create and select custom object.
- Fill the following details:
- Label: Claim
- Plural Label: Claim
- Object Name: Auto Populated
- Record Name: Claim Number
- Data type: Auto Number

- Format: C - {0000}
- Starting Number: 1
- Under optional features:
- Allow Reports, Activities and Track field History
- Under Deployment Status:
- Select the deployed radio button.
- Under Search Status:
- Check the Allow Search checkbox.
- Click on save.



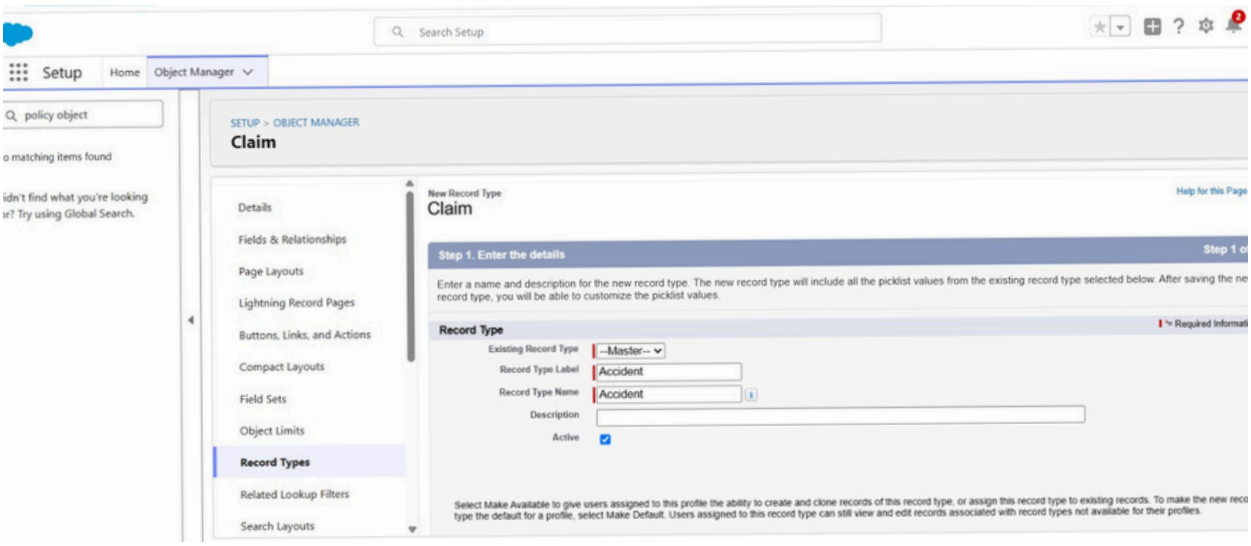




## 6. Define Record Types on Claim Object:

- Go to setup, Click on Object Manager.
- Search for Claim object and click on Claim__c.
- In the left side panel, click on the record types.

- Click on New.
- Fill out the following details:
- Select the existing record type as Master.
- Give record type label as “Accident”
- Record Type Name will be auto populated.
- Give a meaningful description.
- Check the active checkbox.
- Check the Make Available checkbox for all the profiles.
- Click on Next.

- It will redirect to the Assign page layouts page, Leave everything as is, and click on save.




**//Repeat the same steps for creating the property and life record types as well.**

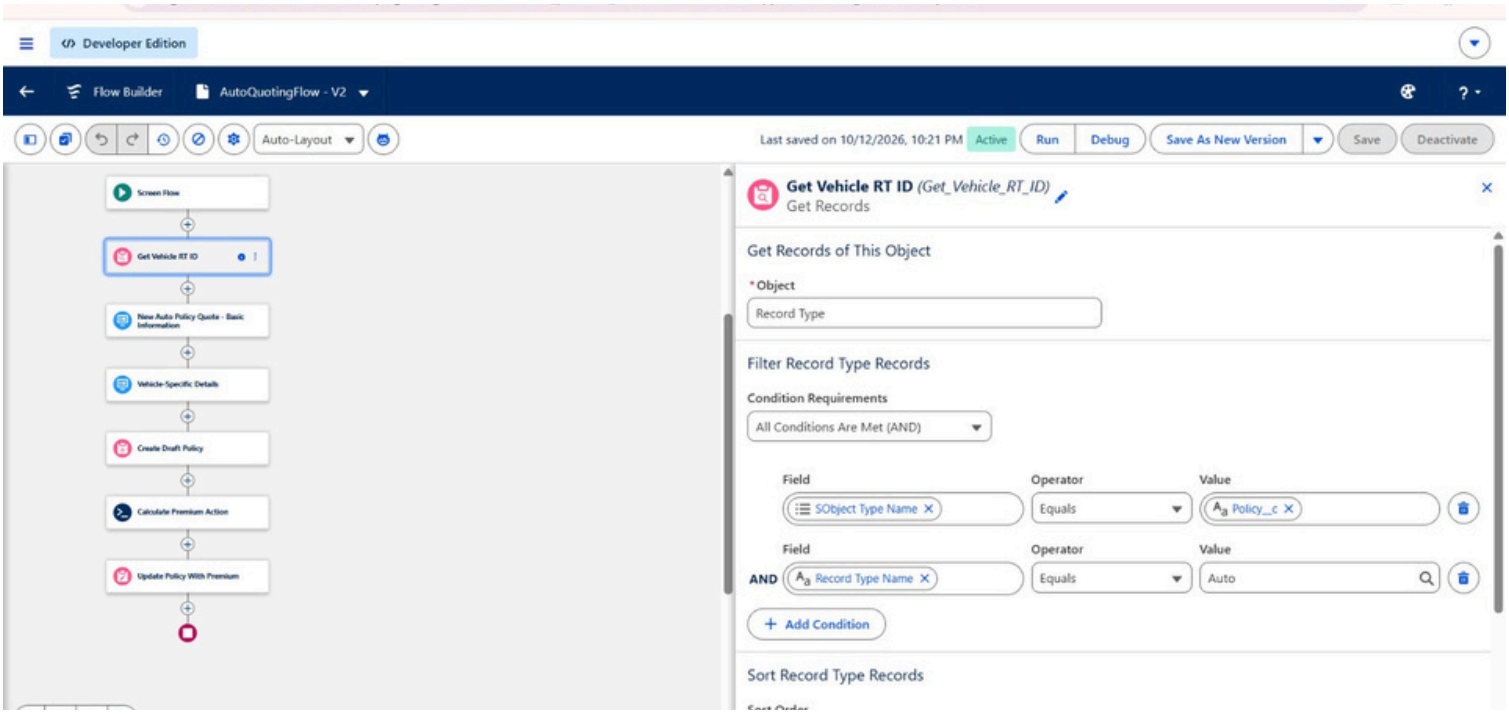
## 7. Create Fields in the Claim Object:

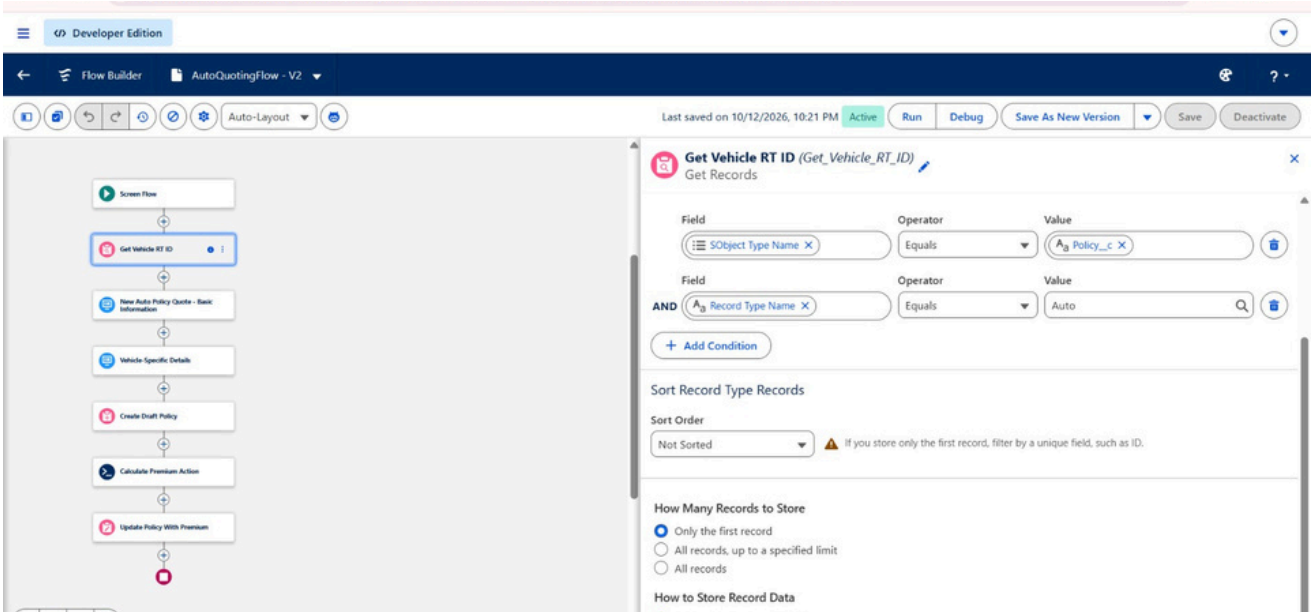
| S.No | Field Name | Data Type | Required |
|---|---|---|---|
| 1 | Adjuster | Lookup(User) | |
| 2 | Approval Status | Picklist(New, Submitted for Approval, Approved, Rejected) | |
| 3 | Claim Amount | Currency(16, 2) Date/Time | |
| 4 | Date of Loss | | |

| | | | |
|---|---|---|---|
| 5 | Description | Long Text Area | |
| 6 | Policy | Lookup(Policy) | |

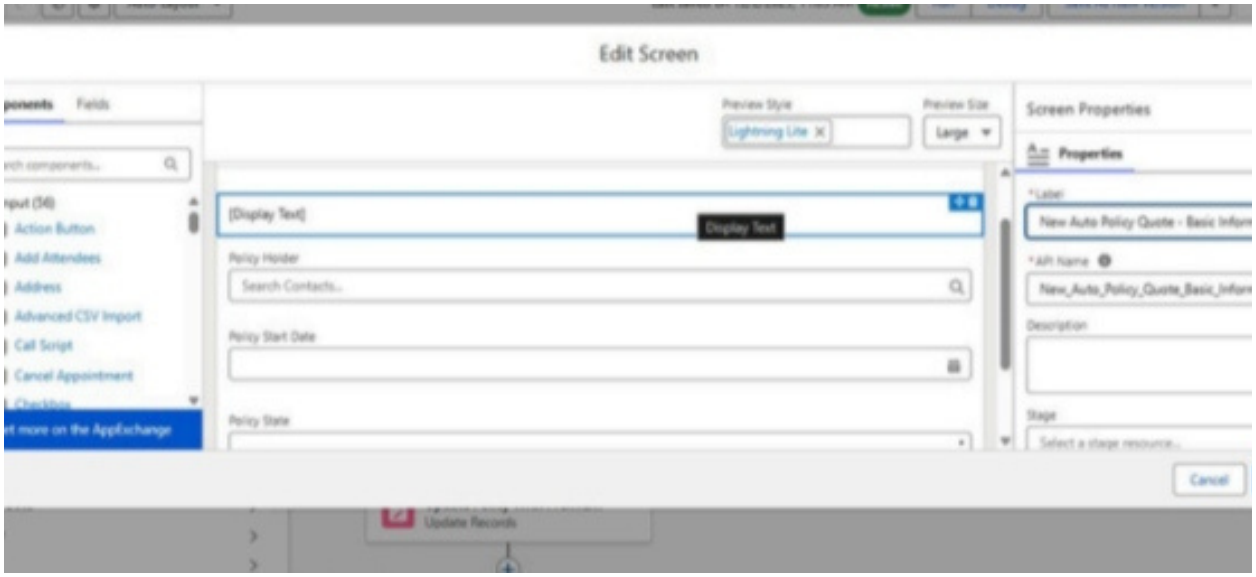
**8. Create an AutoQuoting Screen Flow**

- Goto setup, Search for flows in the quick find box.
- Click on flows and click on new flow.
- Select Screen Flow and Click on Create.
- Add Get Records Element immediately after the start element.
- Give Label as **Get Vehicle RT ID**
- Api Name will be auto populated
- Select Record Type for Object
- For Conditional Requirements, Select All Conditions are met
- The Conditions include:
- sObject Type Name Equals Policy__c
- And
- Record Type Name Equals Auto

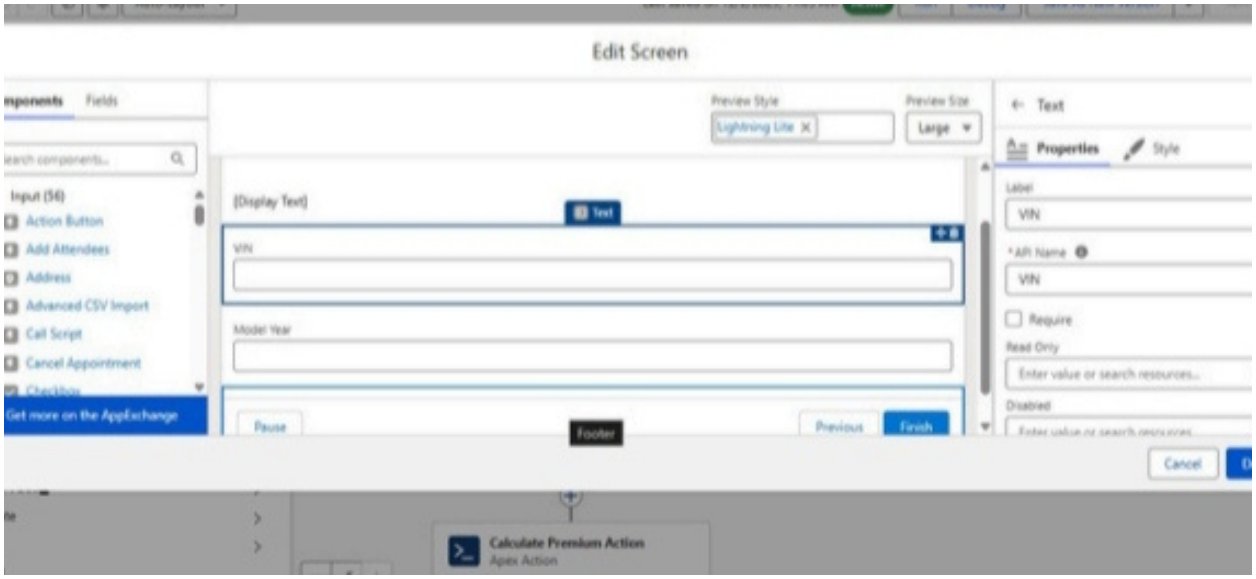




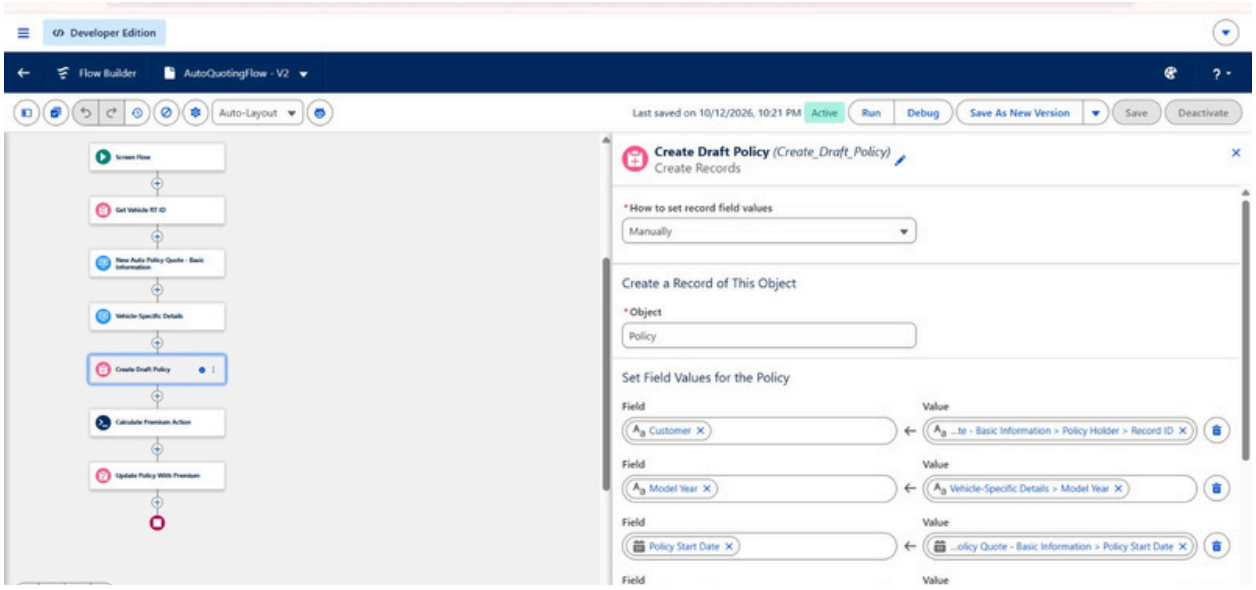

- Add Screen Element immediately after the Get Records Element
- Give the label as New Auto Policy Quote - Basic Information
- API Name will be auto populated.
- Drag the display text component into the flow canvas
- Give the API Name as Screen_1_Policy_Basics
- Drag the Lookup Component into the Canvas, Give API Name as Customer Lookup, Field API Name as Customer__c, Label as Policy Holder, ObjectApiName as Policy__c
- Drag Date component into the canvas, Give Label as Policy Start Date, API Name will be auto populated.
- Drag the picklist component into the canvas, Give the label as Policy State, API Name will be auto populated, For Choice, Create a new Resource, Resource Type should be Picklist Choice Set, Give the API Name, Select the Policy for Object, Select Picklist as Datatype, Select Policy State as Field and Click on save
- Click on Done

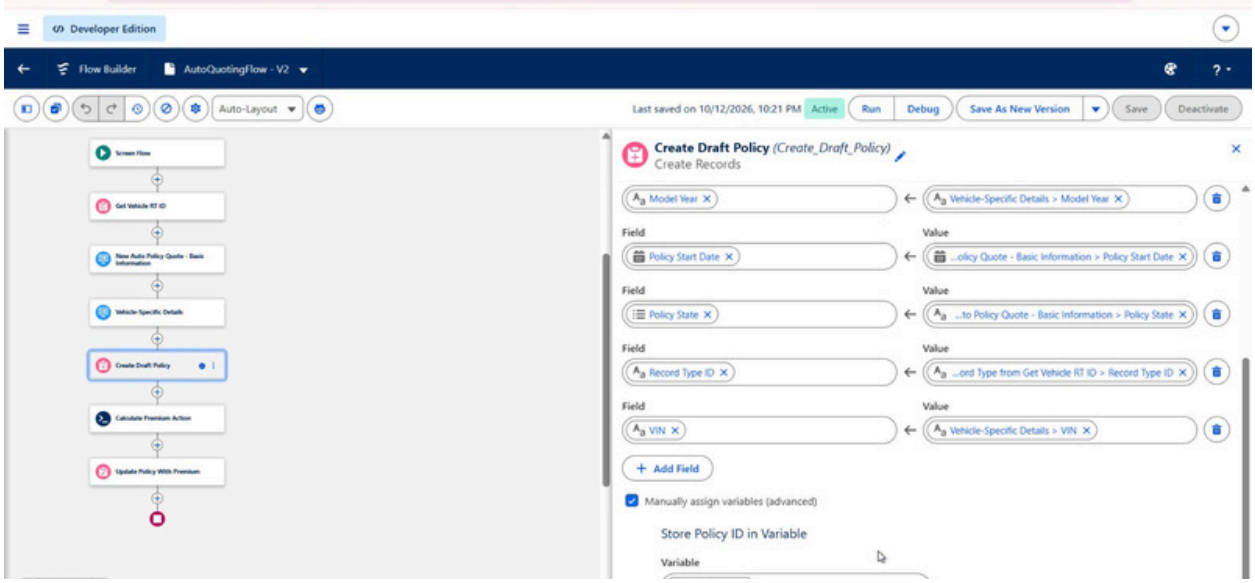



- Add another screen element after the Auto Policy Quote - Basic Information screen
- Give the label as Vehicle-Specific Details
- API Name will be auto populated.
- Drag the display text component into the flow canvas
- Give the API Name as Screen_2_Vehicle_Details
- Drag the text component into the canvas, Give the label as VIN, API name will be auto populated
- Again, Drag the text component into the canvas, Give the label as Model year, API name will be auto populated.
- Click on done.



- Add Create Records Element after the screen element.
- Give the label as **Create Draft Policy,** API name will be auto populated
- Store field values manually
- Select the policy object
- Set fields for the policy object as shown below:
- Customer —> From Basic information screen-> Policy Holder -> Record ID
- Model Year—--> Vehicle information screen -> Model Year
- Policy Start Date —> From Basic information screen→Policy Start Date
- Policy State —> From Basic information screen —> Policy State
- RecordType ID —> From Get Vehicle RT Id —> RecordType ID
- VIN —> From Vehicle Information Screen —> VIN
- Check manually assign variables checkbox
- In the place of variable, create a new resource, Whose resource type is variable, API Name would be varPolicyId, Datatype would be Text, Click on Done.

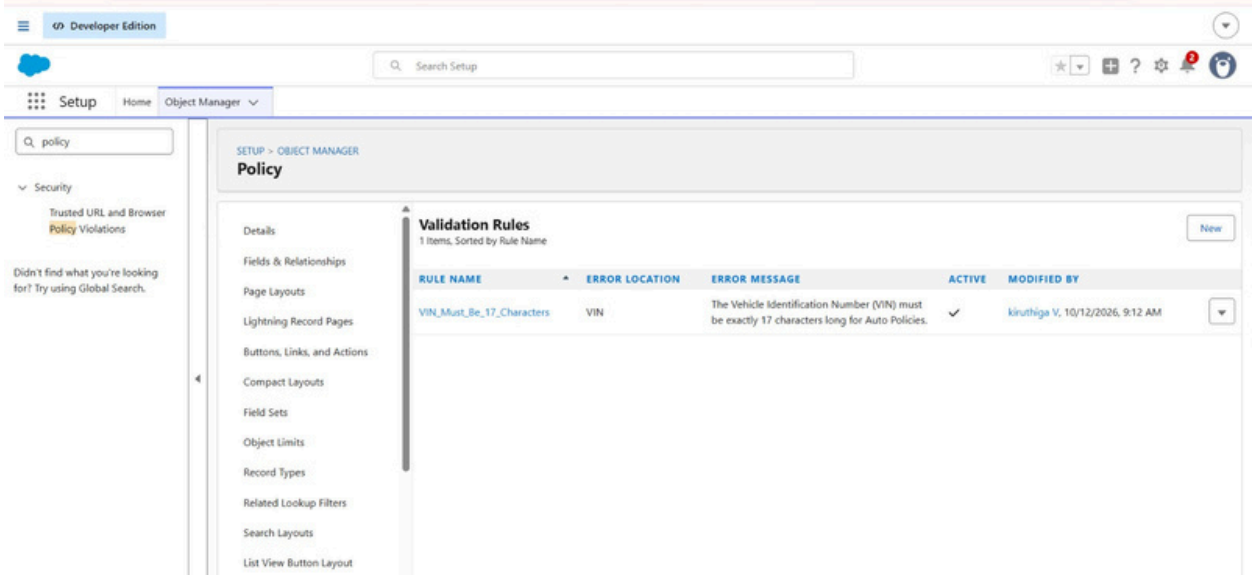





- Save the flow with AutoQuotingFlow
- Activate the flow

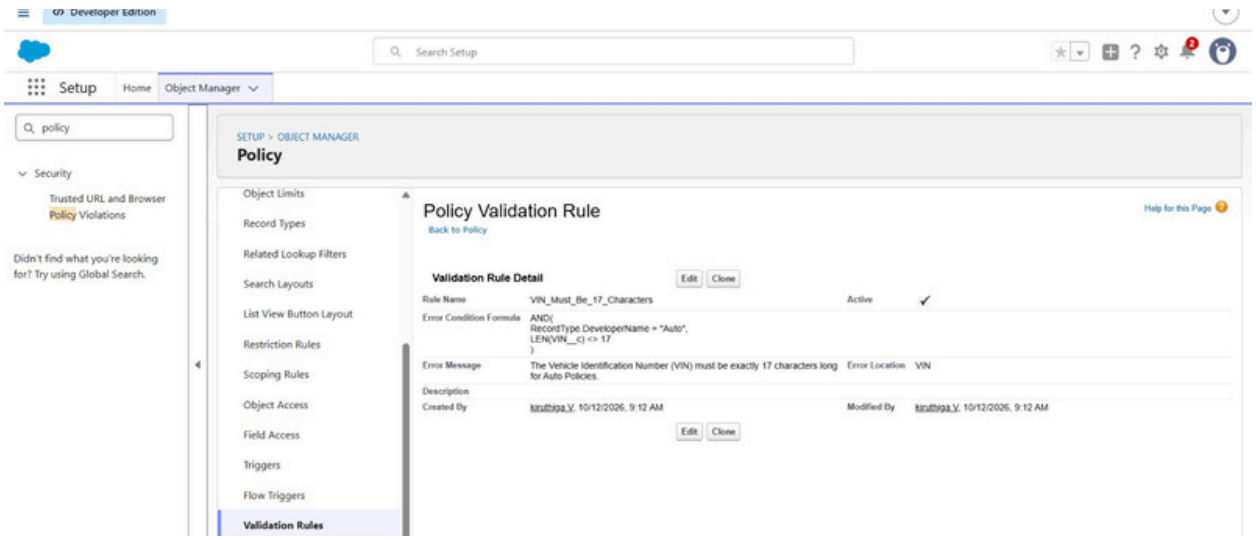
## 9. CreateValidationRuleonPolicyObject:

- Go to object manager, Search for the policy object and click on policy object
- In the left side panel, scroll down and find validation rules and click on it
- Click on New




- Give the rule name as VIN_Must_Be_17_Characters

- Check the active checkbox
- Write the following formula in the formula editor
    - AND( RecordType.DeveloperName = "Auto", LEN( VIN__c )<> 17)
- For error message, The Vehicle Identification Number (VIN) must be exactly 17 characters long for Auto Policies.
- Select error location as field and select VIN field
- Click on Save




## Milestone 2: Complex Policy Issuance & Claim Routing Automation

1. **Create an Apex Class (PremiumCalculator) with a method that accepts policy data**
    - Click on setup icon, Right click on developer console, Open the link in new tab
    - At the top left corner, click on the file, then click on new, and click on Apex class, Give the name as PremiumCalculator and click on create.
    - Paste the following code in the apex class:

```
public class PremiumCalculator {

    public class PremiumInputWrapper {

        @InvocableVariable(required=true)
```

```
        public Id policyId;

    }



 @InvocableMethod(label='Calculate Policy Premium' description='Calculates
premium based on policy fields (VIN, Model Year, State).')

    public static List<Decimal> calculatePremium(List<PremiumInputWrapper> inputs) {



        List<Decimal> premiumResults = new List<Decimal>();

        Set<Id> policyIds = new Set<Id>();



        for (PremiumInputWrapper input : inputs) {

            if (input.policyId != null) {

                policyIds.add(input.policyId);

            }

        }



        List<Policy__c> policiesToRate = [
```

```
        SELECT Id, VIN__c, Model_Year__c, Policy_State__c
        FROM Policy__c
        WHERE Id IN :policyIds
    ];

    for (Policy__c policy : policiesToRate) {
        Decimal basePremium = 1000.00;

        if (policy.Policy_State__c == 'CA') {
            basePremium *= 1.15;
        }else if (policy.Policy_State__c == 'TX') {
            basePremium *= 1.05;
        }

        if (policy.Model_Year__c != null && Integer.valueOf(policy.Model_Year__c) <
2018) {
            basePremium += 100;
```

```
            }


            premiumResults.add(basePremium);

        }


        return premiumResults;

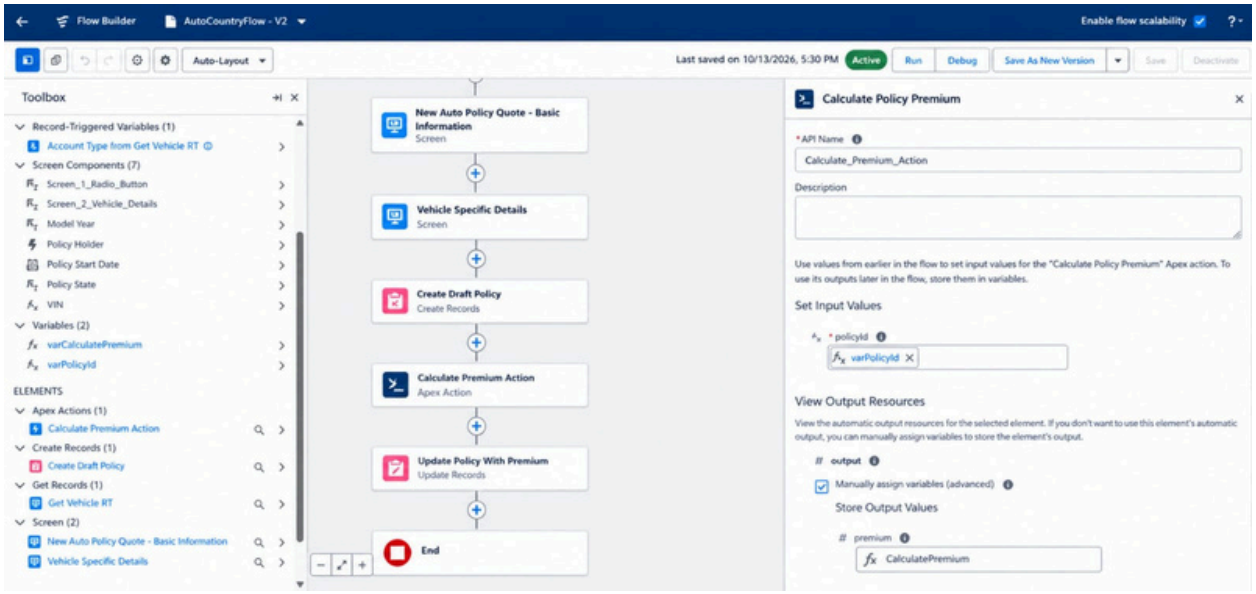
    }

}
```

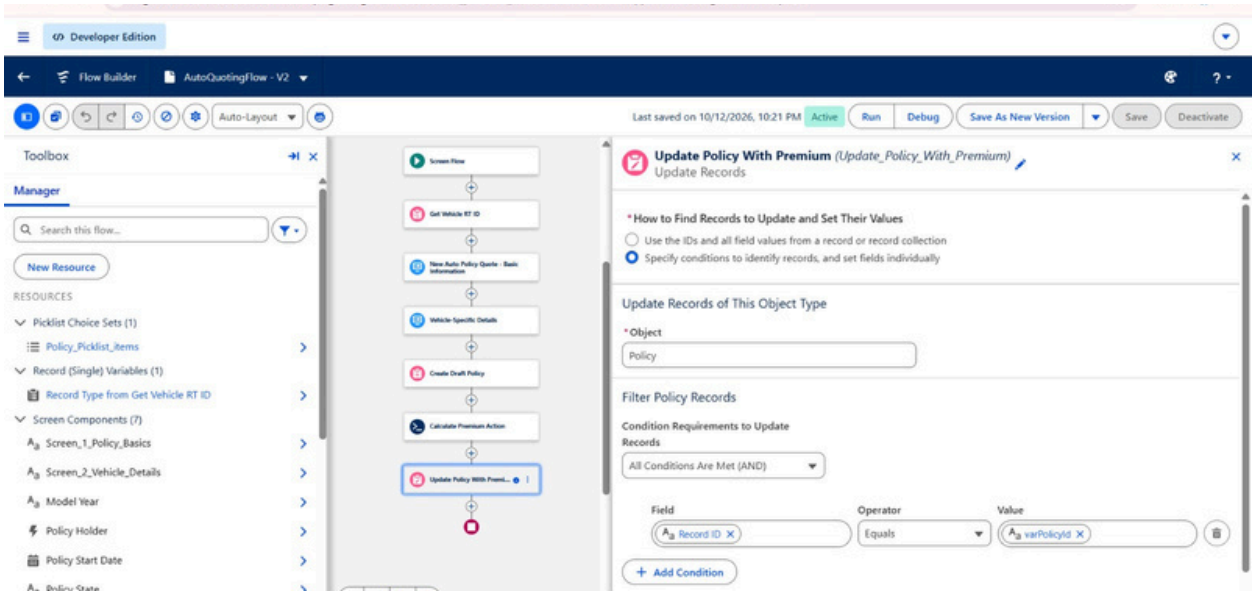
- Click on CTRL + S to save the apex class
- And this apex class need to be invoked in the AutoQuotingFlow

2. **Integrate this Apex action into the final step of the *AutoQuotingFlow* toupdate the draft Policy record**
   - Go to setup, in the quick find box search for flows and click on flows
   - Find for AutoQuotingFlow and open in the flow builder
   - Under the Create Records element,
   - Add an action element after the create records element.
   - Give the label as Calculate Premium Action
   - API Name will be auto populated
   - Under set Input values, For policy Id, Set the varPolicyId variable created in the previous step
   - Under view output resources, check the manually assign variables checkbox and for the output placeholder, create a resource

- Resource type would be variable, API Name would be varCalculatedPremium, Data type would be currency, Decimal places are ‘2’ and click Done.



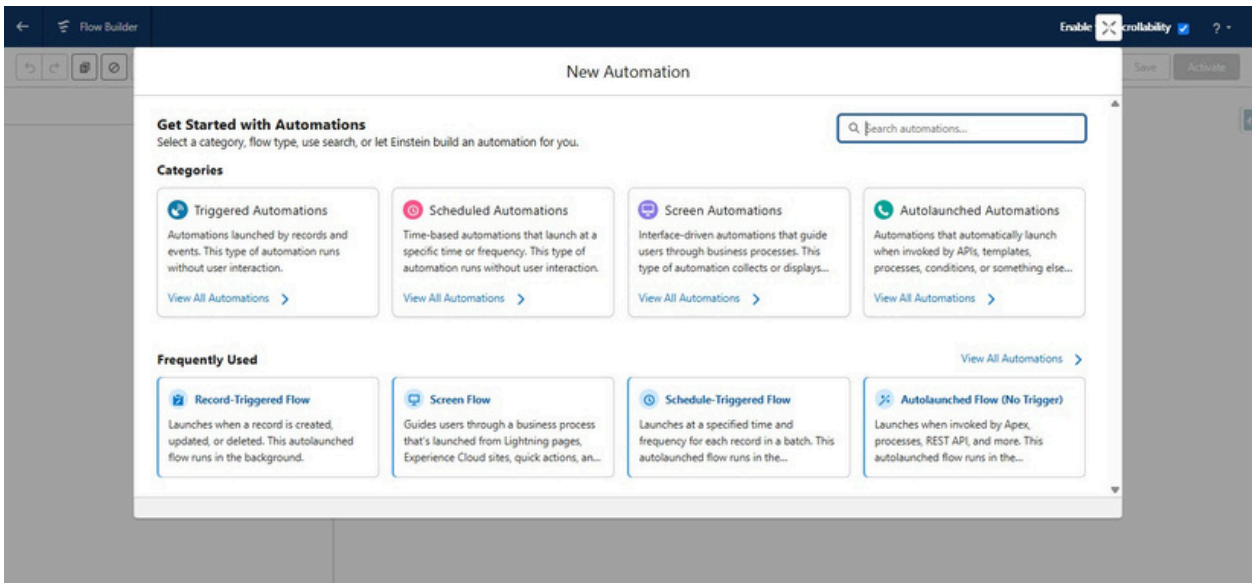

- After the action element, add the update records element, Give the label as Update Policy With Premium
- For how to find records to update and set their values, Select Specify conditions to identify records, and set fields individually
- Select object as policy
- Select all conditions are met for filter criteria
- The conditions include:
- Record Id Equals varPolicyId variable
- Under set field values for policy record
- Assign varCalculatedPremium variable to the Premium field



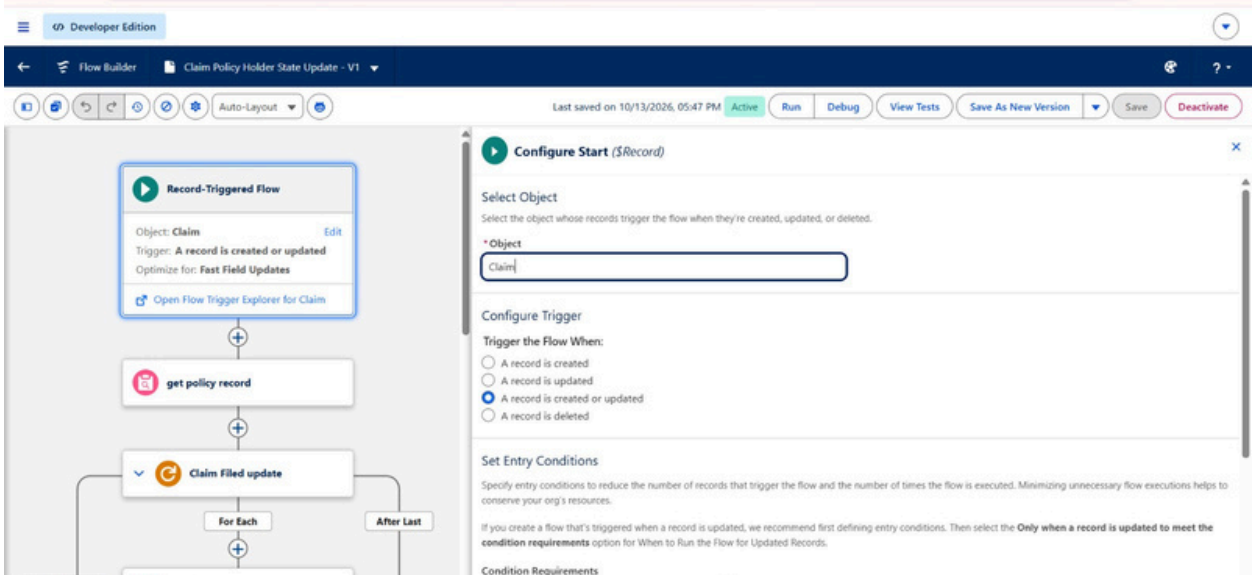

- Save the flow as a new version
- Activate the flow.

**3. Create a Record-Triggered Flow (After Save) on the Claim object.**

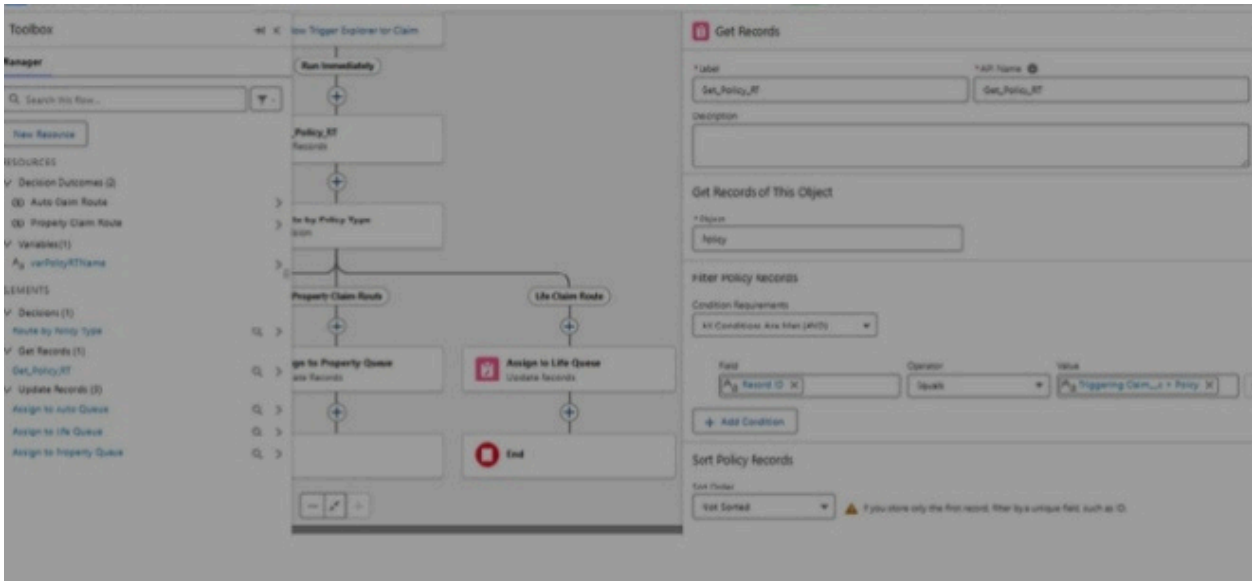
- Goto setup, Inthe quick findbox, Searchfor flows and click on flows
- Click on New Flow, select Record Triggered Flow and Click on Create

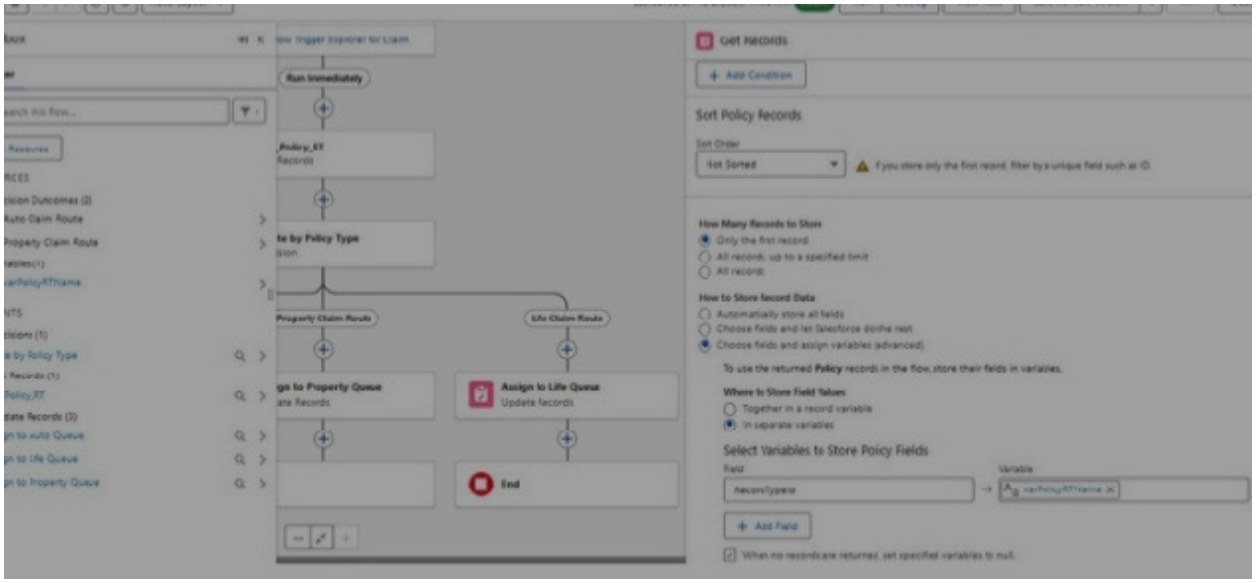


- Select Claim for an object
- Configure trigger when the record is created
- Optimize flow for action and Related Records



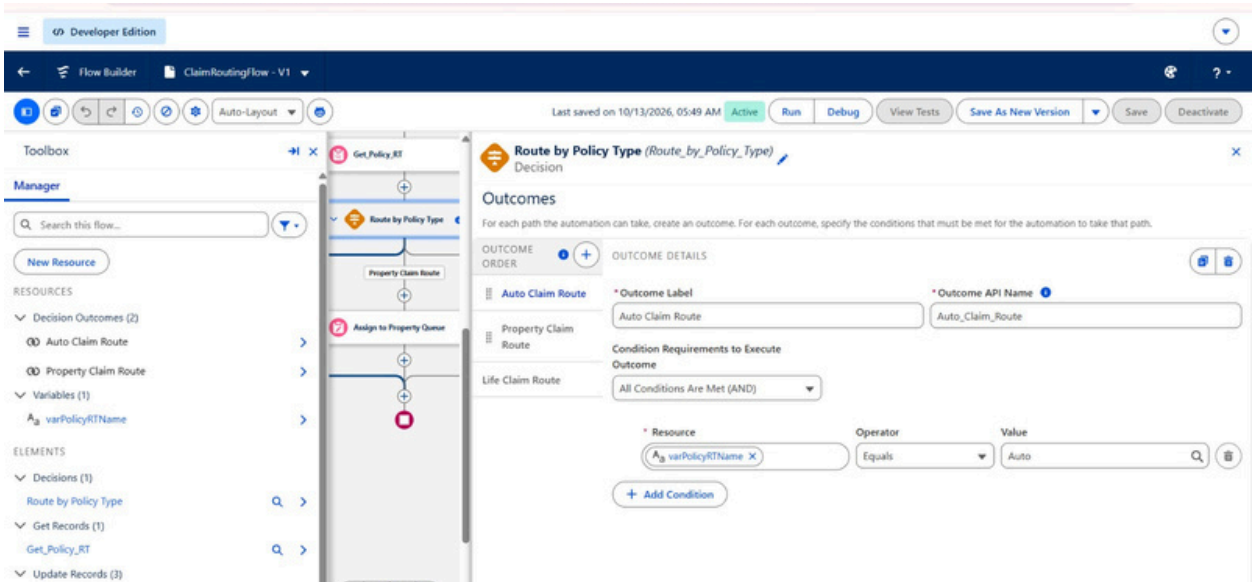

- Add Get Records element after the start and give the label as Get_Policy_RT
- Select policy for object
- Specify all conditions are met for conditional requirements
- The conditions include:
- Record Id Equals Triggering Claim → Policy
- Store only the first record
- Choose fields and assign variables for storing record data
- Select in Separate variables for where to store field values section
- Under select variables to store policy fields
- For recordTypeID field Create a resource, Resource type would be variable, API Name would be varPolicyRTName, Data type would be Text and click on Done

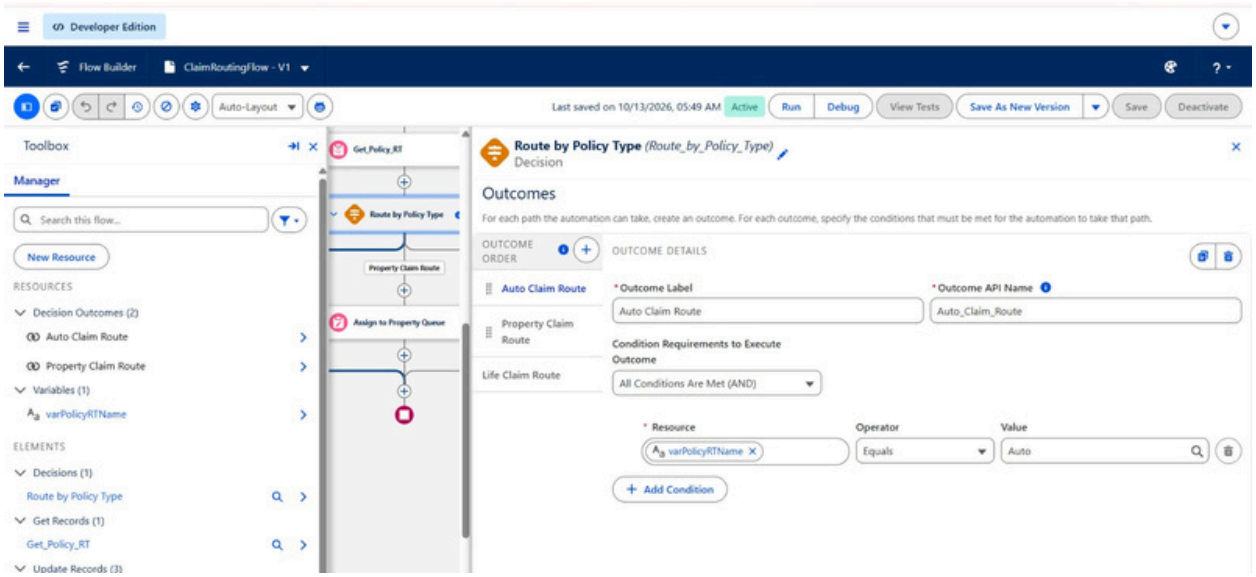


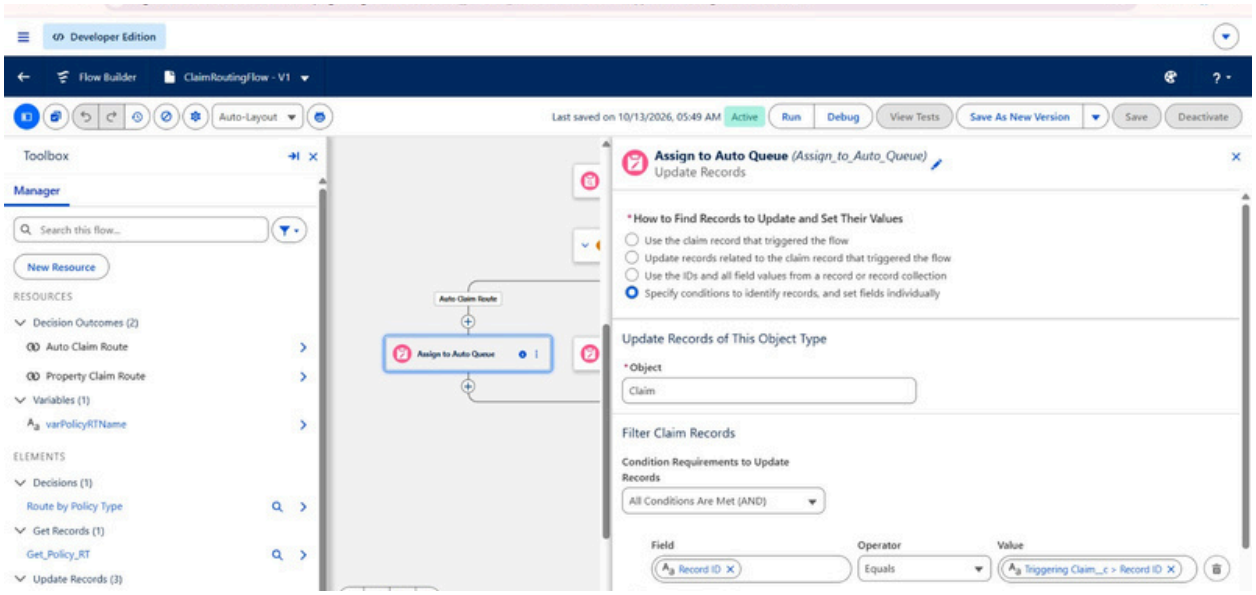

- Under Get Records Element, Add decision element
- Give the label as Route by Policy Type
- API name will be auto populated
- Select Decision Logic as Manually set conditions
- Specify the Outcomes:
- Outcome1:
    - Label as Auto Claim Route
    - API name will be auto populated
    - Select All conditions for the conditional Requirements

    - The condition includes:
    - varPolicyRTName variable equals Auto
- Outcome2:
    - Label as Property Claim Route
    - API name will be auto populated
    - Select All conditions for the conditional Requirements
    - The condition includes:
    - varPolicyRTName variable equals Property
- Default Outcome:
    - Label as Life Claim Route

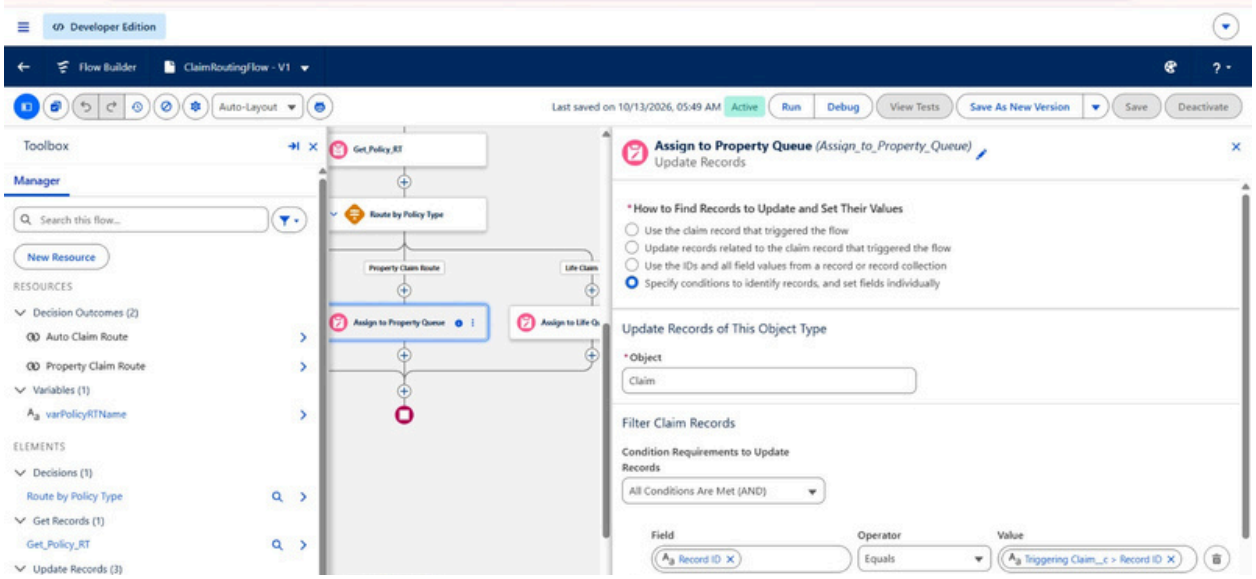


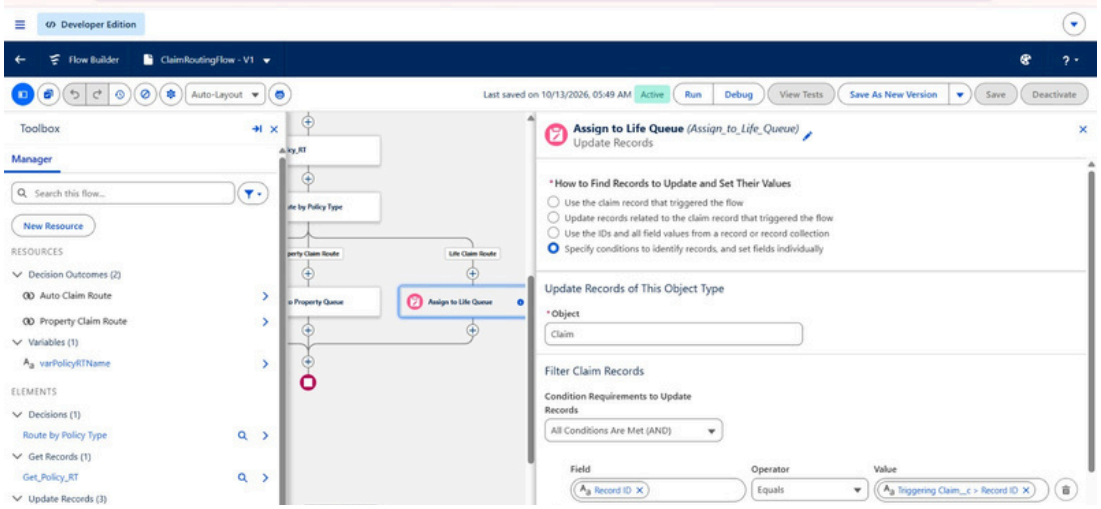
- Under Auto Claim Route Decision path, add the update records element
- Label it as Assign to Auto Queue
- API name will be auto populated
- Under How to find records to update and set their fields, select specify conditions to identify records
- Select Claim as an object
- Select All conditions are true for Conditional requirements
- The conditions include:
- Record Id equals Triggering Claim → Record ID
- Under Set fields for the Claim Records
- Set the hardcoded auto queue 18 digit Id for Owner ID field




- Under Property Claim Route Decision path, add the update records element
- Label it as Assign to Property Queue
- API name will be auto populated
- Under How to find records to update and set their fields, select specify conditions to identify records
- Select Claim as an object
- Select All conditions are true for Conditional requirements
- The conditions include:
- Record Id equals Triggering Claim → Record ID
- Under Set fields for the Claim Records
- Set the hardcoded Property queue 18 digit Id for Owner ID field




- Under Life Claim Route Decision path, add the update records element
- Label it as Assign to Life Queue
- API name will be auto populated
- Under How to find records to update and set their fields, select specify conditions to identify records
- Select Claim as an object
- Select All conditions are true for Conditional requirements
- The conditions include:
- Record Id equals Triggering Claim → Record ID
- Under Set fields for the Claim Records
- Set the hardcoded Life queue 18 digit Id for Owner ID field



## Milestone 3: Claims Adjuster LWC Dashboard Development

1. **Create a robust Apex class (ClaimsAdjusterController) with a method using SOQL to query all Claims owned by the logged-in user, joining related Policy and Customer data. The method must return a custom Wrapper Class listing key metrics.**
    - Click onsetup icon, Right click on developer console,Open the link innew tab
    - At the top left corner, click on the file, then click on new, and click on Apex class, Give the name as ClaimsAdjusterController and click on create.
    - Paste the following code in the apex class:

```
public class ClaimsAdjusterController {



    public class ClaimWrapper {

        @AuraEnabled public Id claimId;

        @AuraEnabled public String claimNumber;

        @AuraEnabled public Decimal claimAmount;

        @AuraEnabled public String status;

        @AuraEnabled public String policyType;

        @AuraEnabled public String policyHolderName;

        @AuraEnabled public Integer daysOpen;
```

```
    public ClaimWrapper(Claim__c c, Policy__c p, Contact customerRec) {
        this.claimId = c.Id;
        this.claimNumber = c.Name;
        this.claimAmount = c.Claim_Amount__c;
        this.status = c.Approval_Status__c;

        Policy__c policyRecord = (Policy__c)p;

        if (policyRecord != null && policyRecord.RecordType != null) {
            this.policyType = policyRecord.RecordType.Name;
        }else {
            this.policyType = 'Unknown';
        }

        this.policyHolderName = (customerRec != null) ? customerRec.Name : '';
```

```
            this.daysOpen = (c.Date_of_Loss__c != null)
                                ? (Date.today().daysBetween(c.Date_of_Loss__c.date()))
                                :0;
        }
    }

    @AuraEnabled(cacheable=true)
    public static List<ClaimWrapper> getAssignedClaims() {

        List<ClaimWrapper> results = new List<ClaimWrapper>();

        List<Claim__c> claims = [
        SELECT
            Id, Name, Claim_Amount__c, Approval_Status__c, Date_of_Loss__c,
            Policy__r.Name,
            Policy__r.RecordType.DeveloperName,
```

```
            Policy__r.Customer__r.FirstName,
            Policy__r.Customer__r.LastName
        FROM Claim__c
        WHERE OwnerId = :UserInfo.getUserId()
        ORDER BY Date_of_Loss__c DESC
    ];

    for (Claim__c claim : claims) {

        Policy__c policy = claim.Policy__r;

        Contact customer = (policy != null && policy.Customer__r != null)
                           ? (Contact)policy.Customer__r
                           :new Contact();

        results.add(new ClaimWrapper(claim, policy, customer));
```

```
        }


        return results;

    }

}
```

- Save the file using CTRL + S
- This apex class will be handled in the LWC component

**2. Create a component (claimsDashboardLwc) that uses the Apex method via the @wire service and implements client-side data filtering**

- Open the new tab in chrome and search for lightning studio extension
- Open the first link and click on add to chrome, and confirm by clicking add Extension button
- Now open the Lightning studio extension from your salesforce org
- On the Top left corner, you will find Create New
- Click on Create New and Click on Lightning Web Component
- Enter the name of the Component as claimsDashboardLwc
- Check the isExposed checkbox
- Under target configurations, Select Lightning app page, Home page and Record page
- Click on Deploy
- Open the HTML file and paste the following code:

```
<template>


    <lightning-card title="My Assigned Claims Dashboard"
icon-name="standard:claim">
```

```
        <div class="slds-m-around_medium">

            <lightning-combobox

                name="policyFilter"

                label="Filter by Policy Type"

                value="All"

                options={policyTypeOptions}

                onchange={handleFilterChange}

                class="slds-m-bottom_medium"

            ></lightning-combobox>



            <template if:true={visibleClaims}>

                <div class="slds-grid slds-wrap slds-grid_pull-padded">

                    <template for:each={visibleClaims} for:item="claim">

                        <div key={claim.claimId} class="slds-col
slds-size_1-of-1 slds-medium-size_1-of-2 slds-large-size_1-of-3
slds-p-horizontal_small">

                            <c-claim-tile-lwc
claim-data={claim}></c-claim-tile-lwc>
```

```
                        </div>

                    </template>

                </div>

                <p class="slds-m-top_medium slds-text-heading_small">Total
Claims Assigned: {visibleClaims.length}</p>

            </template>

            <template if:false={visibleClaims}>

                <p>No claims currently assigned to you.</p>

            </template>

        </div>

    </lightning-card>

</template>
```

- Open the JS file and paste the following code:

```
import { LightningElement, wire, track } from 'lwc';

// Import the Apex method

import getAssignedClaims from
'@salesforce/apex/ClaimsAdjusterController.getAssignedClaims';
```

```
export default class ClaimsDashboardLwc extends LightningElement{

    // List to hold all data retrieved from Apex

    @track allClaims = [];

    // List to display after filtering

    @track visibleClaims = [];

    error;



    // Wire service to call the Apex method

    @wire(getAssignedClaims)

    wiredClaims({ error, data }){

        if (data){

            this.allClaims = data;

            this.visibleClaims = data; // Initially display all claims

            this.error = undefined;

        }else if (error){
```

```
            this.error = error;

            this.allClaims = undefined;

            this.visibleClaims = undefined;

            console.error('Error retrieving claims:',
JSON.stringify(error));

        }

    }



    // Client-side filtering logic (Required by M3)

    handleFilterChange(event) {

        const selectedPolicyType = event.target.value;



        if (selectedPolicyType === 'All'){

            this.visibleClaims = this.allClaims;

        } else {

            // Filter claims where policyType matches the selected value

            this.visibleClaims = this.allClaims.filter(claim =>
```

```
                claim.policyType === selectedPolicyType

            );

        }

    }



    get policyTypeOptions(){

        // Options for the filter dropdown

        return [

            {label: 'All Policy Types', value: 'All' },

            {label: 'Auto', value: 'Auto' },

            {label: 'Property', value: 'Property' },

            {label: 'Life', value: 'Life'}

        ];

    }

}
```

- Open the meta-xml file and paste the following code:

```
<?xml version="1.0"?>

<LightningComponentBundle xmlns="http://soap.sforce.com/2006/04/metadata">

    <apiVersion>63.0</apiVersion>

    <isExposed>true</isExposed>

    <targets>

        <target>lightning__AppPage</target>

        <target>lightning__HomePage</target>

    </targets>

</LightningComponentBundle>
```

### 3. Create a reusable component (claimTileLwc) to display essential information for each claim

- Open the Lightning studio extension from your salesforce org
- On the Top left corner, you will find Create New
- Click on Create New and Click on Lightning Web Component
- Enter the name of the Component as **claimTileLwc**
- Check the isExposed checkbox
- Under target configurations, Select Lightning app page, Home page and Record page
- Click on Deploy
- Open the HTML file and paste the following code:

```
<template>

    <lightning-layout-item size="12" padding="around-small">

        <div class="tile slds-box slds-p-around_medium
slds-theme_default">

            <header class="slds-media slds-media_center
slds-m-bottom_small">

                <div class="slds-media__figure">

                    <lightning-icon icon-name={policyIcon}
alternative-text={claimData.policyType} size="small"></lightning-icon>

                </div>

                <div class="slds-media__body slds-m-left_small">

                    <h2 class="slds-text-heading_medium
slds-m-bottom_none">

                        Claim: {claimData.claimNumber}

                    </h2>

                    <p class="slds-text-body_small">Policy Type:
<strong>{claimData.policyType}</strong></p>

                </div>

            </header>
```

```
            <div class="slds-border_top slds-p-top_small">

                <p>Policy Holder:
<strong>{claimData.policyHolderName}</strong></p>



                <lightning-formatted-number

                    value={claimData.claimAmount}

                    format-style="currency"

                    currency-code="USD"

                    label="Claim Amount">

                </lightning-formatted-number>



                <p>Status: {claimData.status}</p>



                <p class="slds-text-body_regular">

                    Days Open: <strong
class="slds-text-color_error">{claimData.daysOpen}</strong>

                </p>
```

```
            </div>

        </div>

    </lightning-layout-item>

</template>
```

- Open the JS File and paste the following code

```
import { LightningElement, api } from 'lwc';



export default class ClaimTileLwc extends LightningElement{

    // @api decorator exposes this property to the parent component

    @api claimData;



    // Helper property to determine the correct icon based on Policy Type

    get policyIcon(){

        if (this.claimData && this.claimData.policyType){

            const type = this.claimData.policyType.toLowerCase();
```

```
            if (type === 'auto') return 'utility:truck';

            if (type === 'property') return 'utility:home';

            if (type === 'life') return 'utility:connected_apps';

        }

        return 'utility:help';

    }

}
```

- Open the meta-xml file and paste the following code

```
<?xml version="1.0"?>

<LightningComponentBundle xmlns="http://soap.sforce.com/2006/04/metadata">

    <apiVersion>63.0</apiVersion>

    <isExposed>true</isExposed>

</LightningComponentBundle>
```

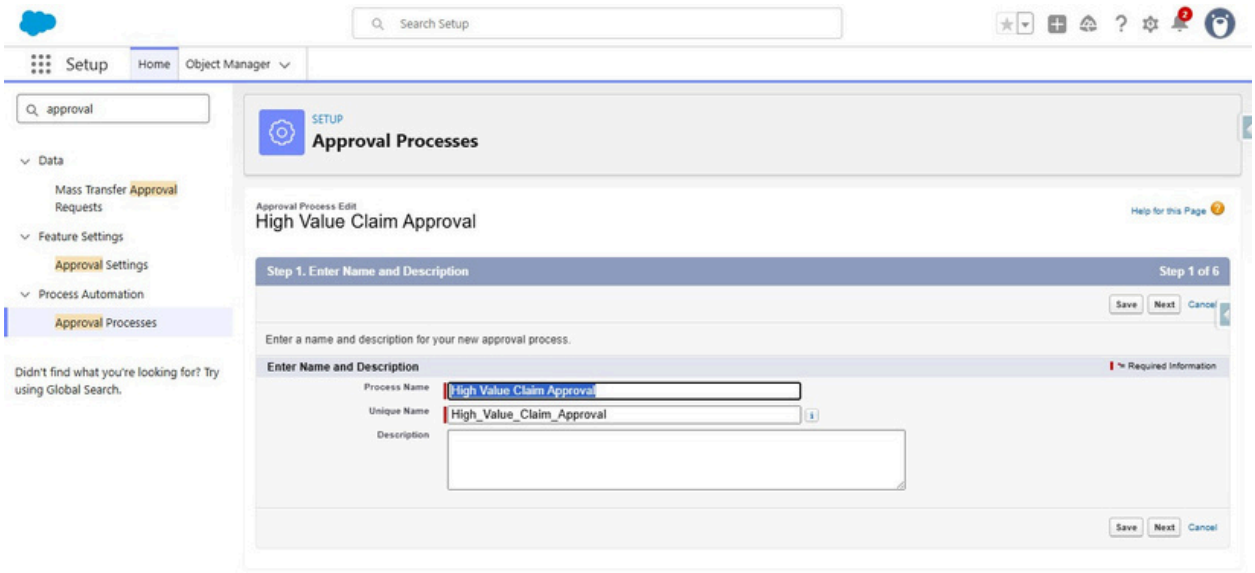
## Milestone 4: Advanced Claim Processing, Security & Code Setup

### 1. Modify the Claim object to include an Approval Status field.

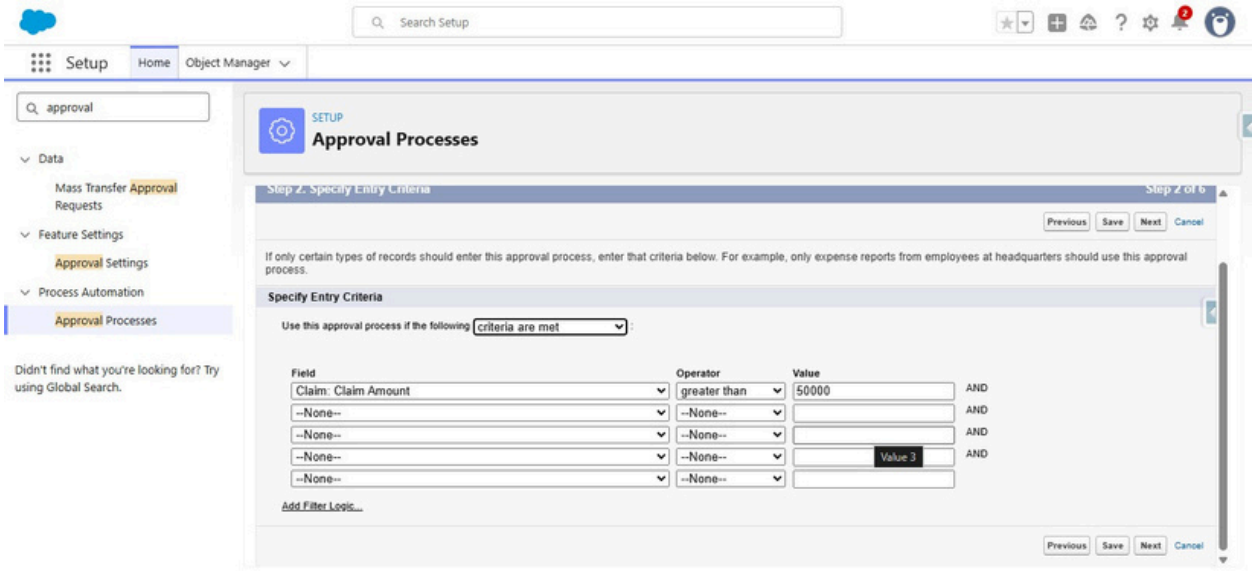
- Go to set up, Clickon object manager, searchforclaim objectand click on claim object.
- In the left side panel, click on fields and relationships.
- Click on new
- Select Datatype as picklist
- Label it as Approval Status
- Select enter values radio button
- Enter New, Submitted for Approval, Approved and Rejected line by line
- Field Name will be auto populated
- Click on Next, Next and Save.

### 2. Configure the Approval Process to route first to the Senior Adjuster, then to the Department Manager, using two sequential steps.

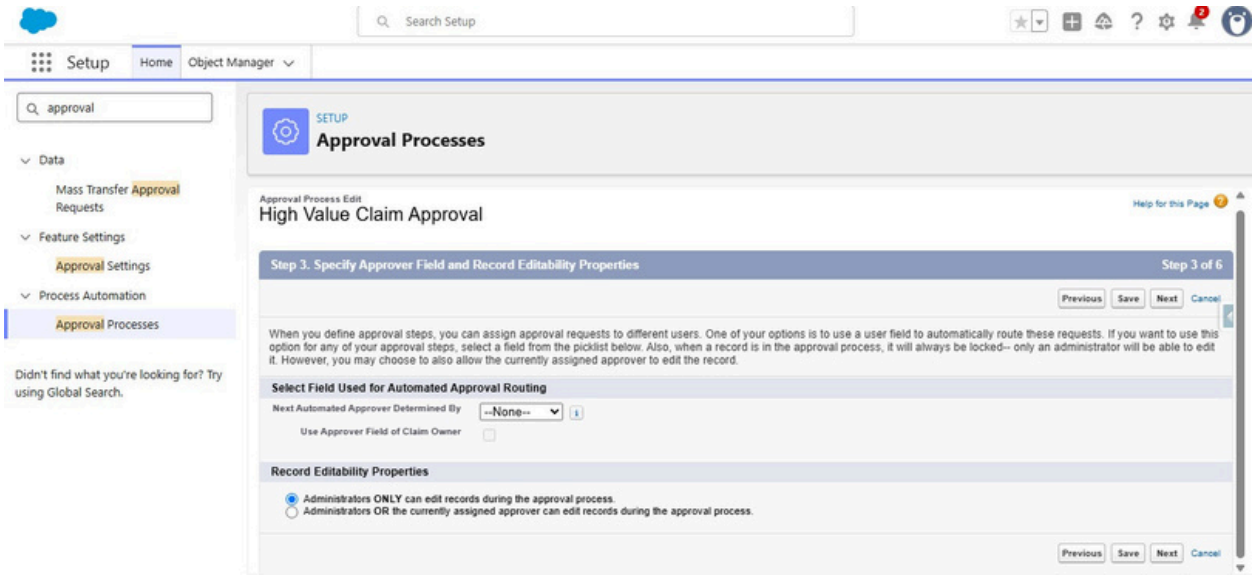
- Goto setup, In the quickfind box,Search forApproval process
- Select claim object for Manage approval process for
- Click on create new Approval Process and select use standard setup wizard
- Give the process name as High Value Claim Approval
- Unique Name will be auto populated
- Click on Next



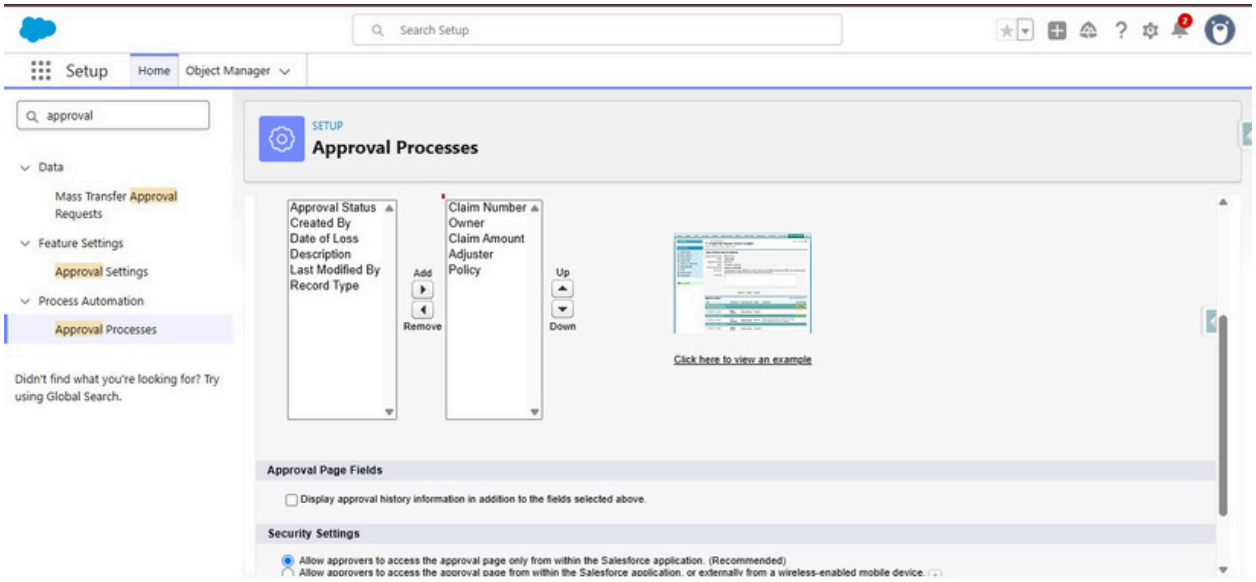

- Under specify entry criteria, Select criteria are met for Use this approval process if
- Criteria includes:
- Claim amount greater than 50000
- Click on Next




- Under Record Editability properties, Select Administrators ONLY can edit the records during the approval process
- Click on Next

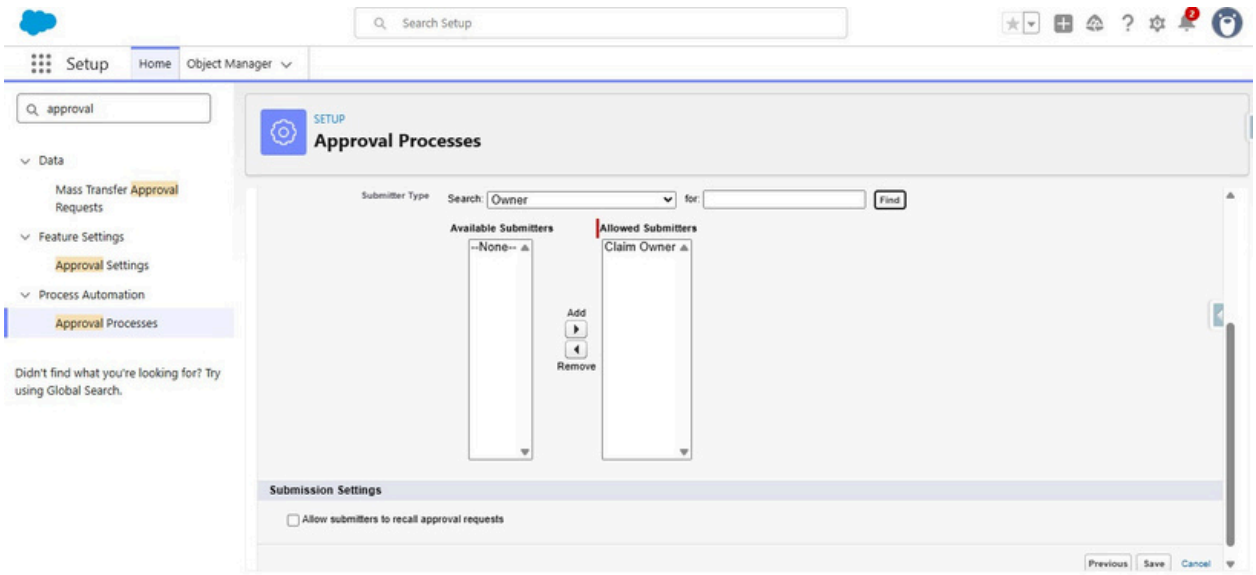



- Click on next again
- Under Approval Page Layout, Select Claim Number, Owner, Claim Amount, Adjuster, Policy fields to the selected fields from available fields
- Under security settings, Select Allow Approvers to access the page only from within the salesforce application.
- Click on Next




- Under Initial Submitters, Search for owner under submitter type, and select the claim owner as allowed submitters from available submitters

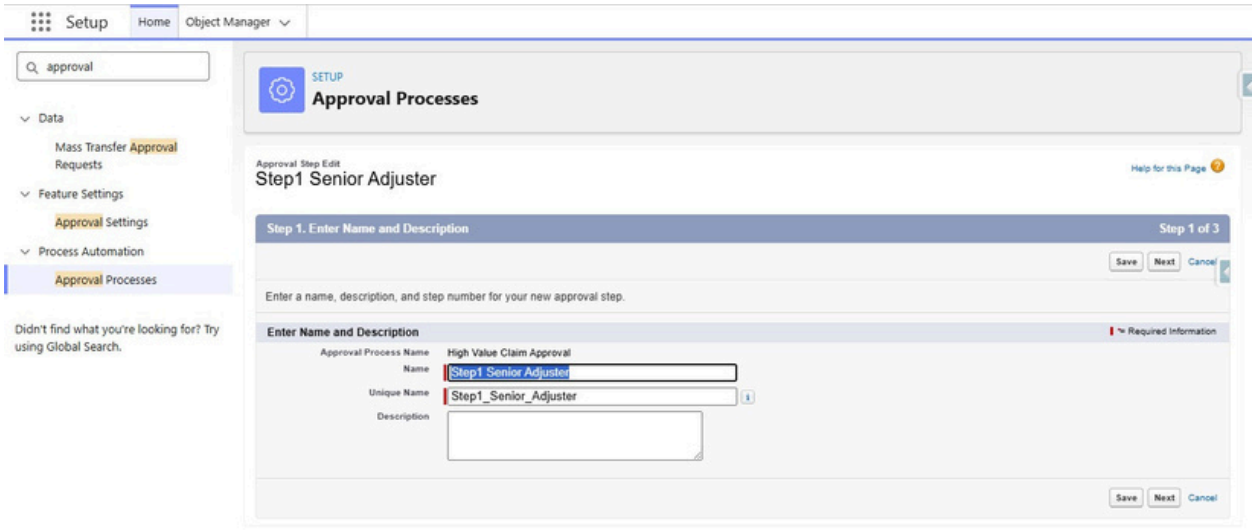
- Click on Save



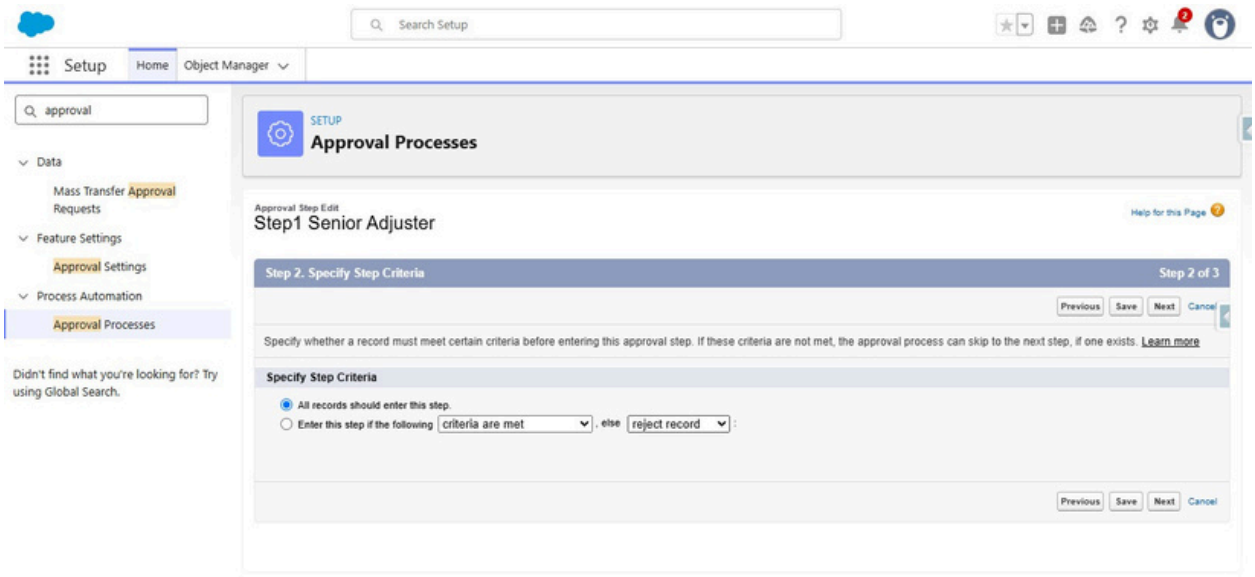

**2.1 Create Approval Steps:**

**Step1:**

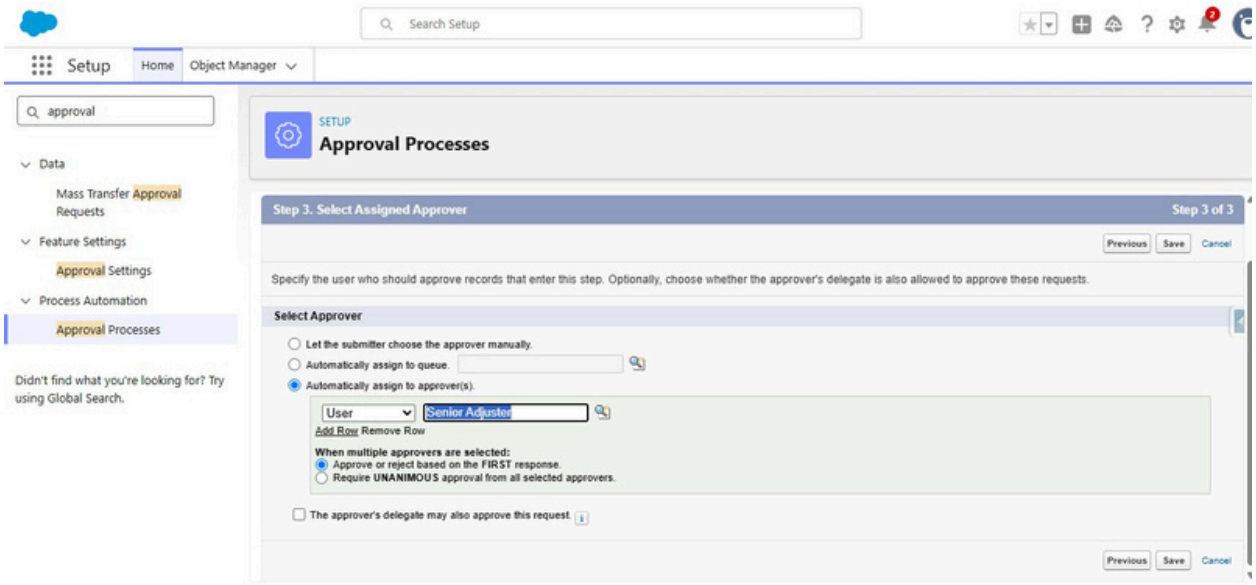
- Under initial submission actions, You can find Approval Steps
- Under approval steps, Click on New
- Give the Name as Step1 Senior Adjuster
- Unique name will be auto populated
- Click on Next



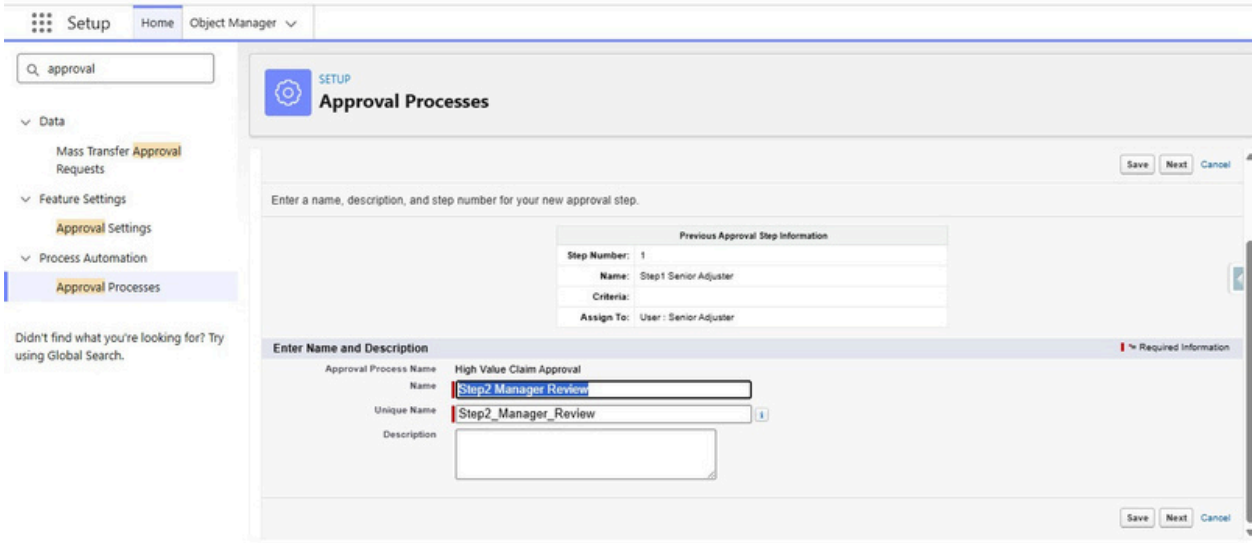

- Under specify step criteria, Select all records should enter the step
- Click on Next



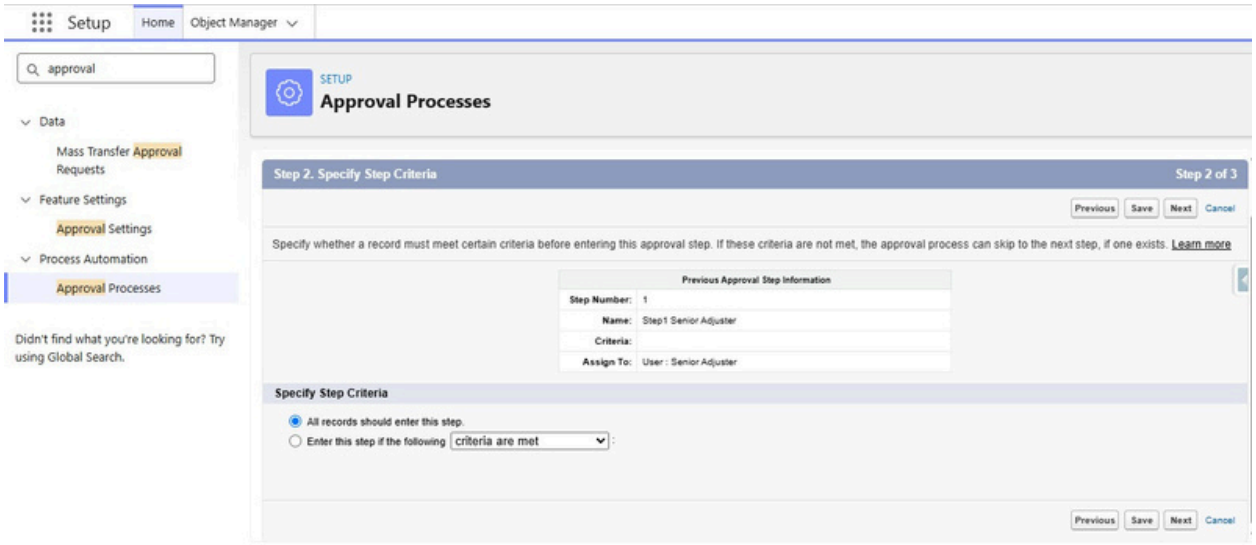

- Under select Approver, Select Automatically assign to approver
- Under that select the Senior Adjuster User
- Under when multiple approvers are selected, select Approve or Reject based on the FIRST Response
- Click on Save



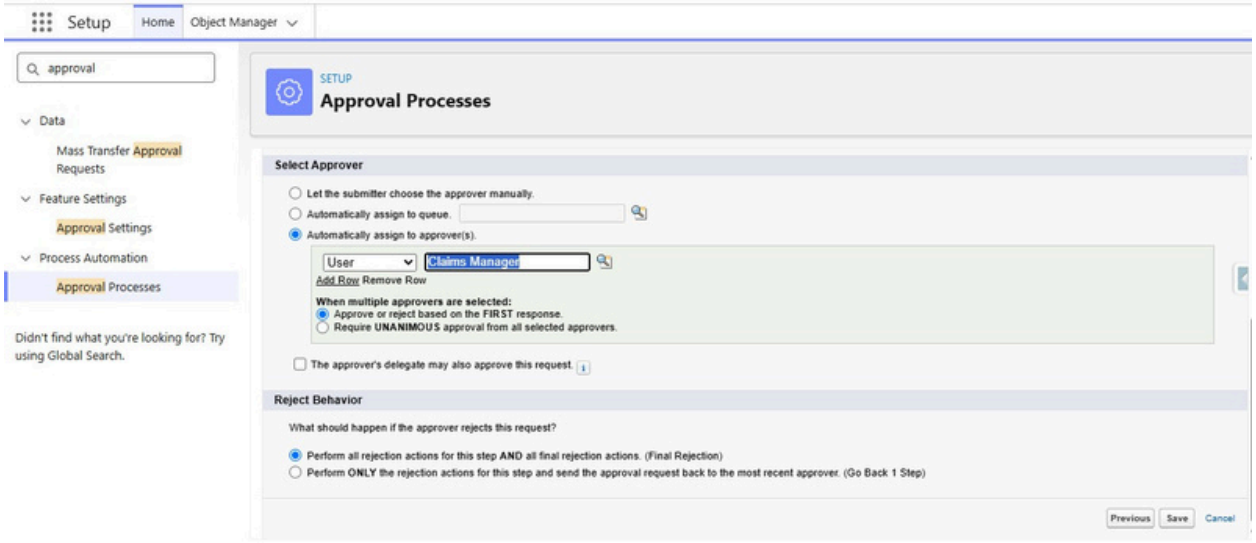

- Click on New again to create one more step
- Give the name as Step2 Manager Review
- Unique name will be auto populated
- Click on Next




- Under specify step criteria, Select all records should enter the step
- Click on Next

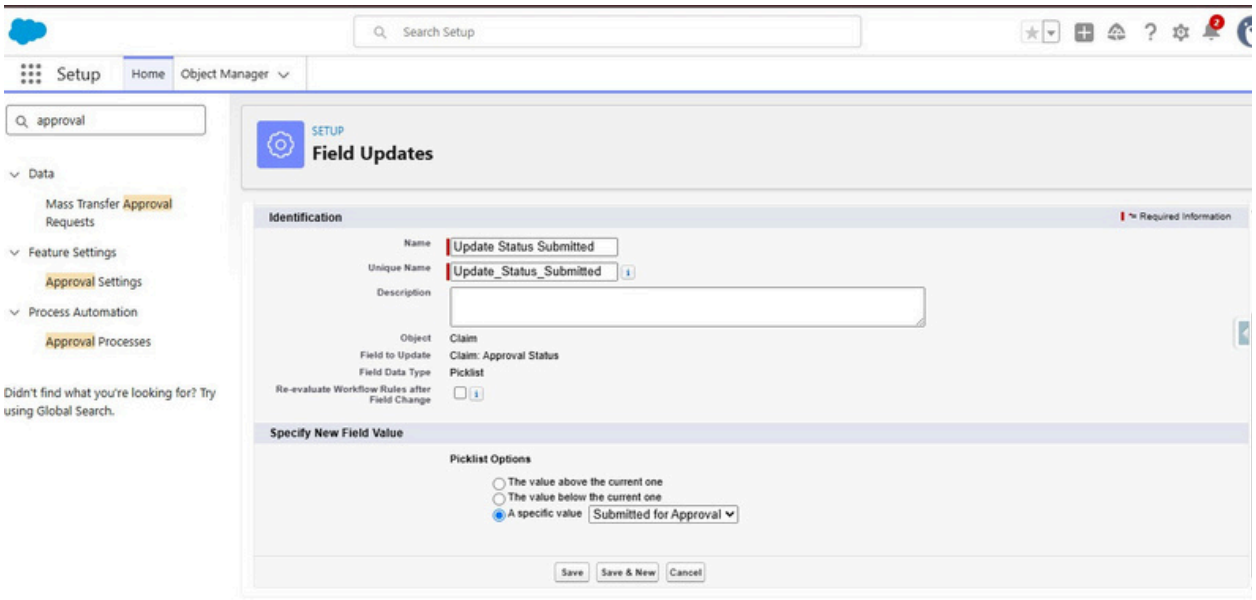



- Under select Approver, Select Automatically assign to approver
- Under that select the Department Manager User
- Under when multiple approvers are selected, select Approve or Reject based on the FIRST Response
- Under Reject behaviour, Select Perform all rejection actions for this step AND all final rejection actions
- Click on Save

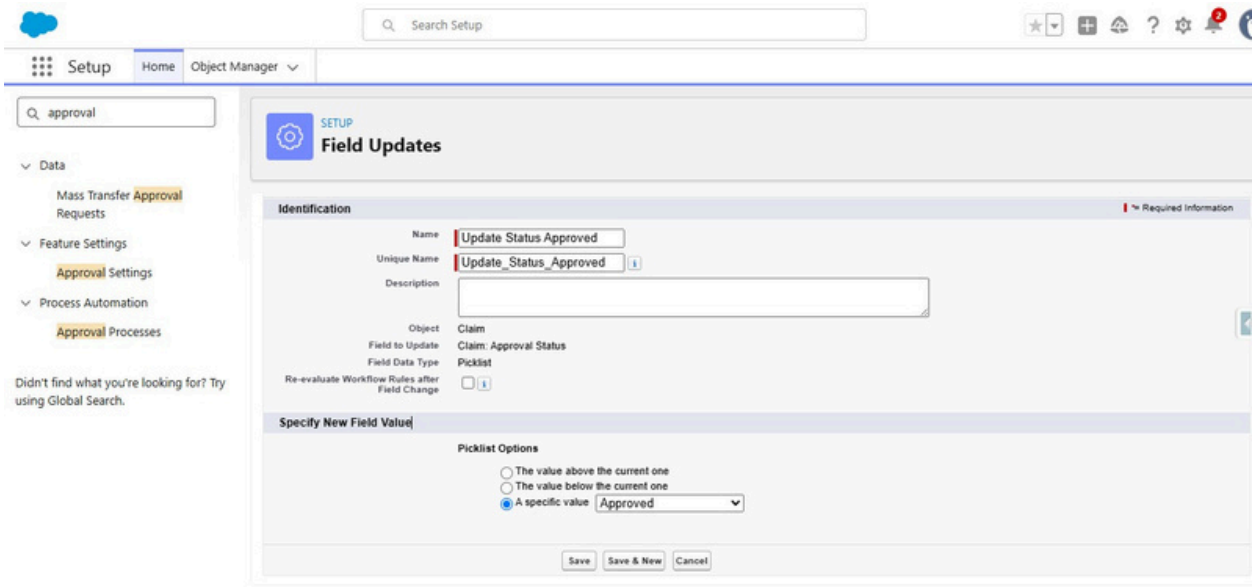



- Under Initial Submission Actions, Click on Add New and select Field Update
- Give the name as Update Status Submitted

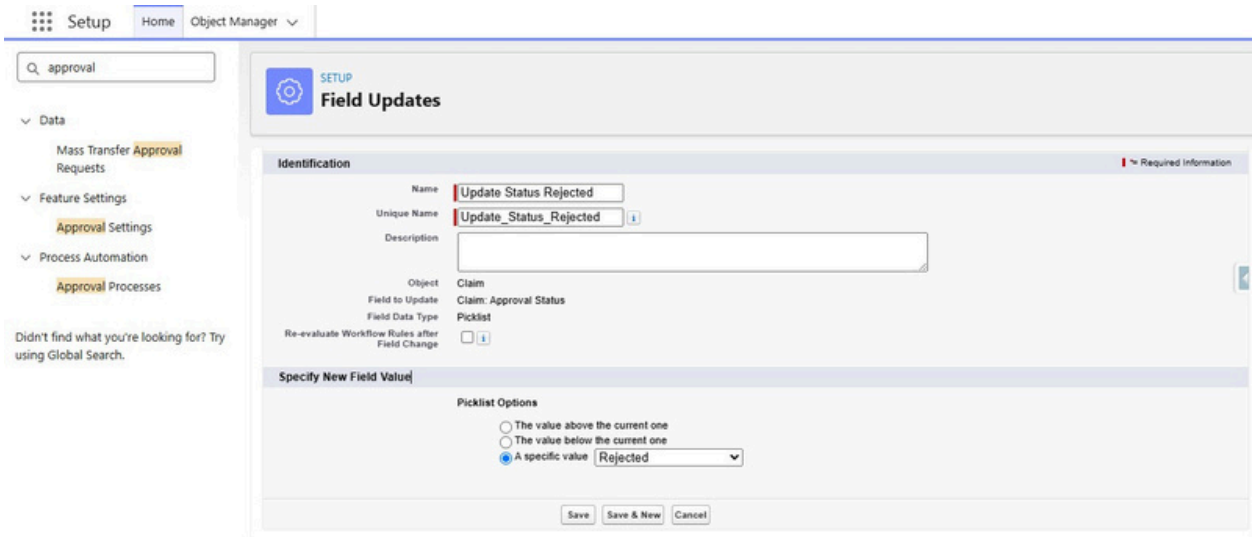
- Unique name will be auto populated
- Select object as Claim
- Select Approval Status for Field to Update
- Select Picklist as field datatype
- Under specify new field value, Select a Specific value and select submitted for Approval
- Click on Save




- Under Final Approval Actions, Click on Add New and select Field Update
- Give the name as Update Status Approved
- Unique name will be auto populated
- Select object as Claim
- Select Approval Status for Field to Update
- Select Picklist as field datatype
- Under specify new field value, Select a Specific value and select Approved
- Click on Save

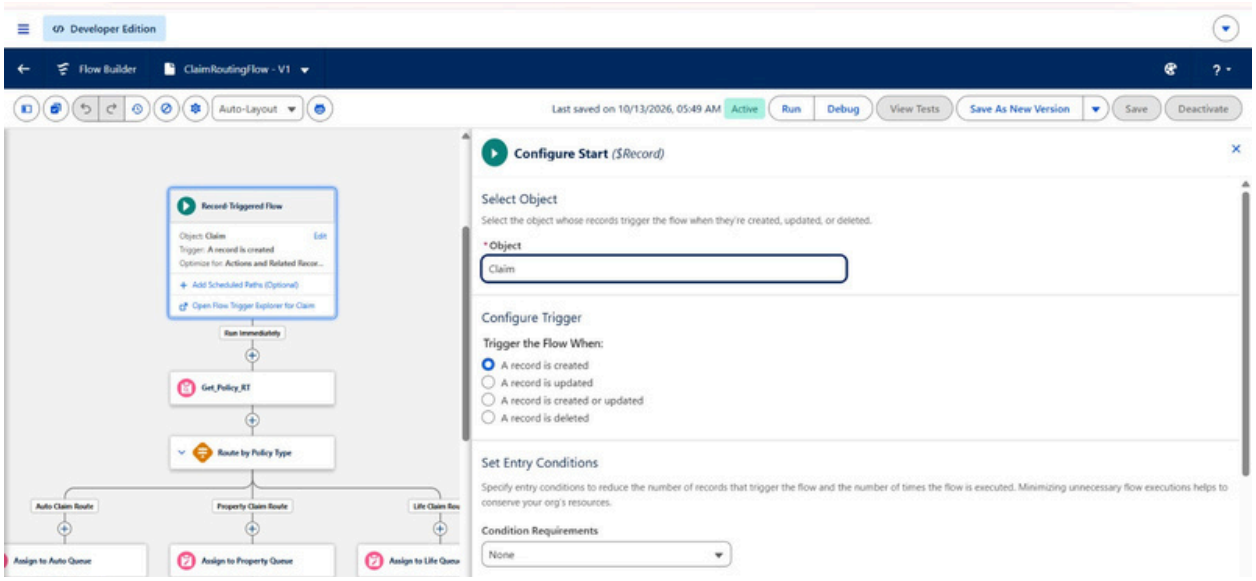


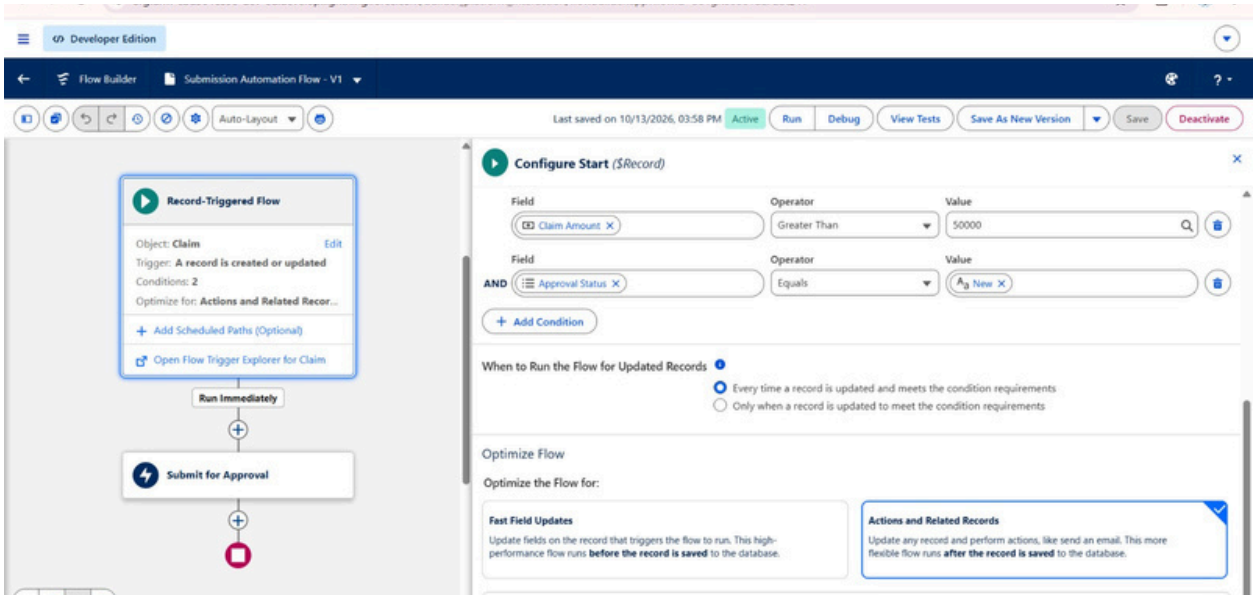

- Under Final Rejection Actions, Click on Add New and select Field Update
- Give the name as Update Status Rejected
- Unique name will be auto populated
- Select object as Claim
- Select Approval Status for Field to Update
- Select Picklist as field datatype
- Under specify new field value, Select a Specific value and select Rejected
- Click on Save



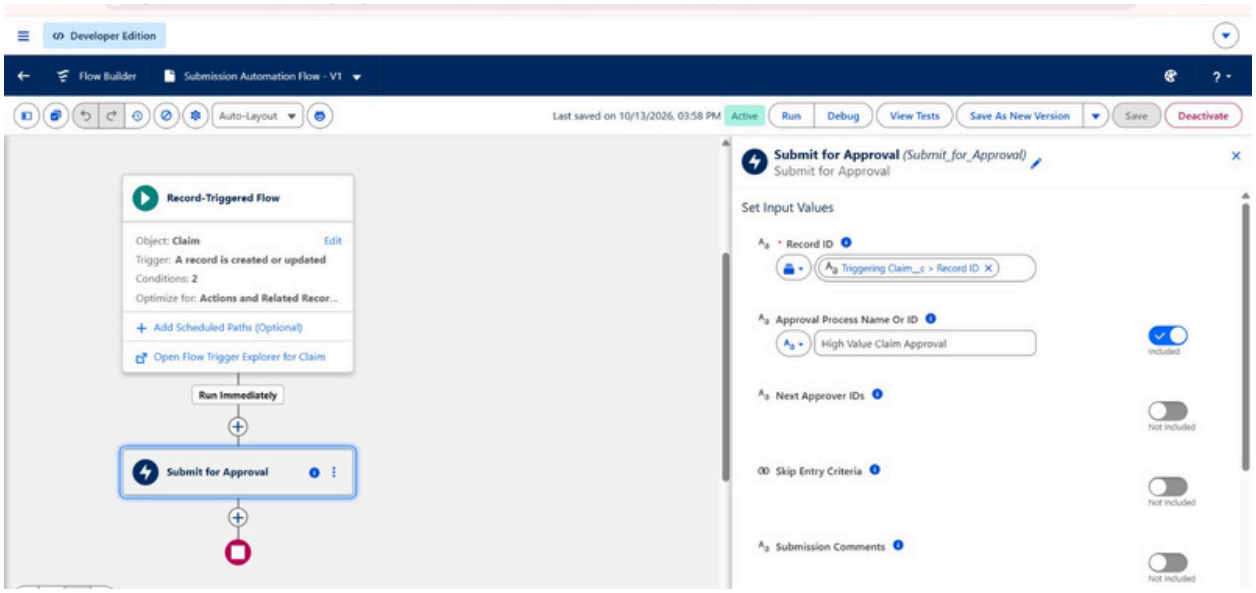
- Activate the Approval Process.

3. **Create a Record-Triggered Flow (After Save) that checks if the Claim Amount is$>\$50,000$and submitsthe recordto anApproval Process.**
    - Go to setup, In the quick find box, search for flows and click on flows.
    - Click on New Flow
    - Select Record Triggered Flow
    - Select object as Claim
    - Configure Trigger When a Record is created or updated
    - Select All conditions are true for conditional requirements
    - The conditions include:
        - Claim Amount Greater than 50000
        - Approval Status Equals New
    - Select Every time a record is updated and meets the conditional requirements for When to run the flow for Updated Records
    - Optimize the flow for Action and Related Records





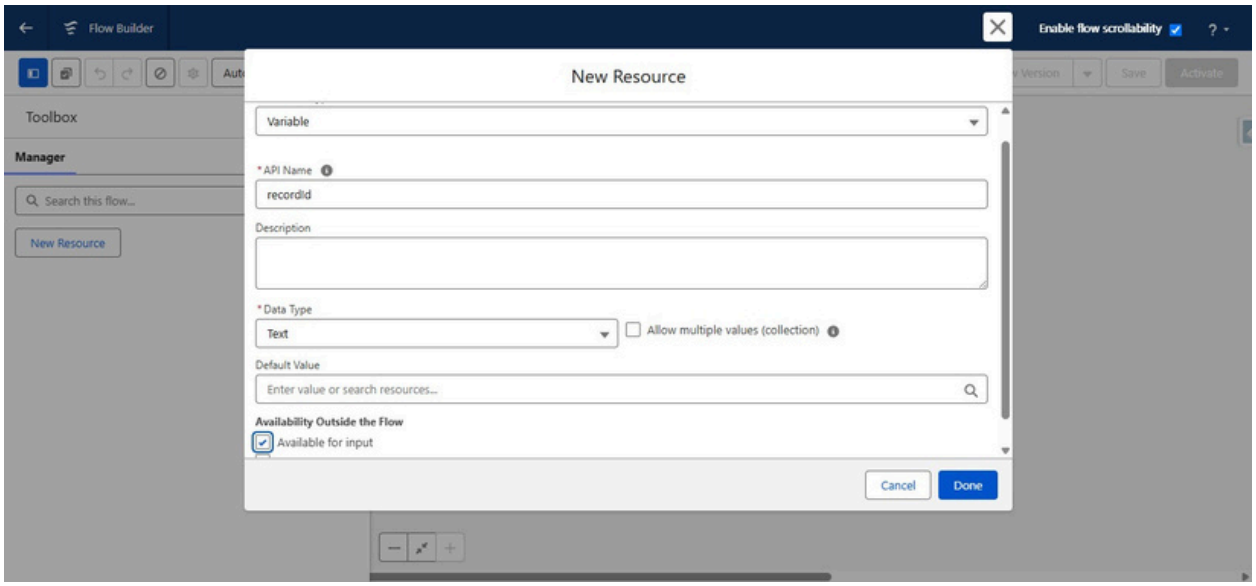

- Add an action element after the Start Element
- Label it as Submit Claim to Approval
- Under Set Input Values, For Record Id, Select Triggering Claim → Record ID
- For select Approval Process Name or Id, Enter the Approval Process Name(High Value Claim Approval)



- Save the Flow with Submission Automation Flow and Activate the flow

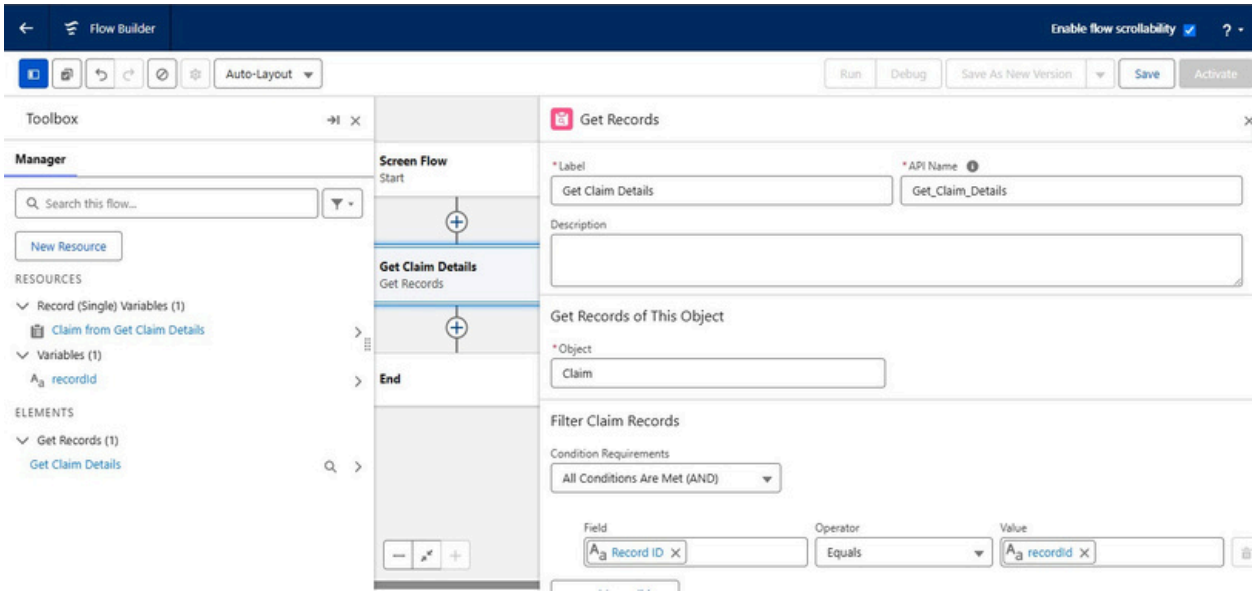
## 4. Implement a Screen Flow for the approvers to quickly review details and approve/rejectdirectly from theClaimrecord.

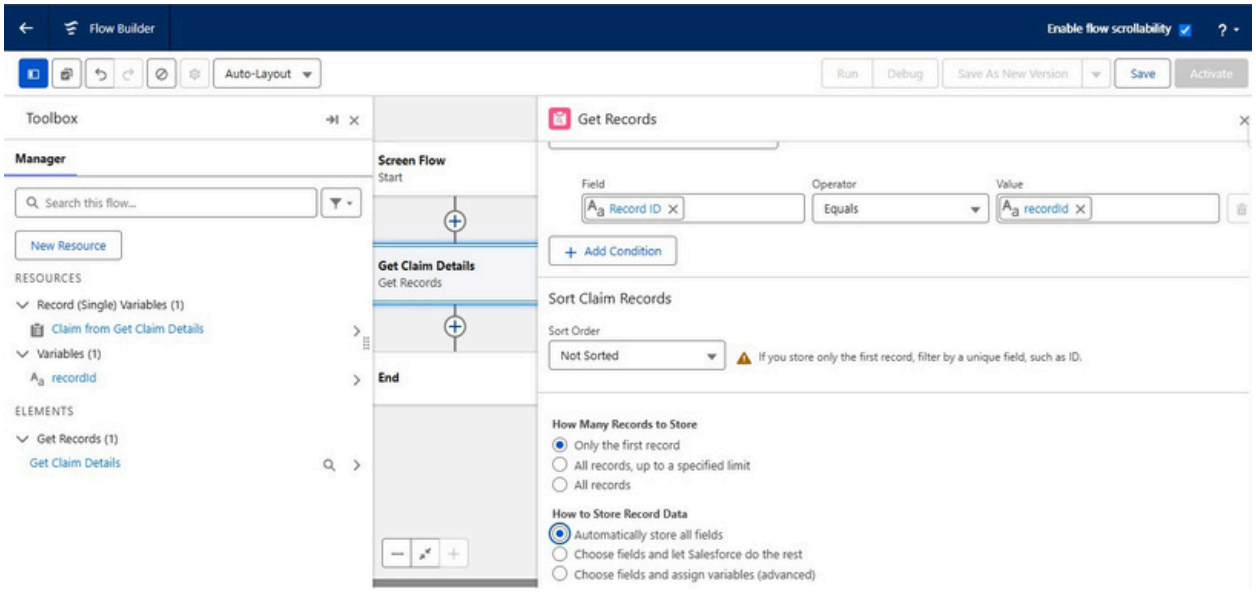
- Go to setup, Search for flows in the quick find box and click on flows.
- Click on New Flow, Choose Screen Flow
- Create a New Resource by clicking on New Resource button in the toolbox manager
- Resource Type would be Variable
- API Name would be recordId
- Data type would be Text
- Under Availability outside the flow, Check Available for Input
- Click on Done




- Add the Get Records Element below the screen
- Label it as Get Claim Details
- Select object as Claim
- Select All conditions are required for conditional Requirements
- The Condition include:

- RecordID equals variable recordId
- Store only the first record and
- Automatically store all fields

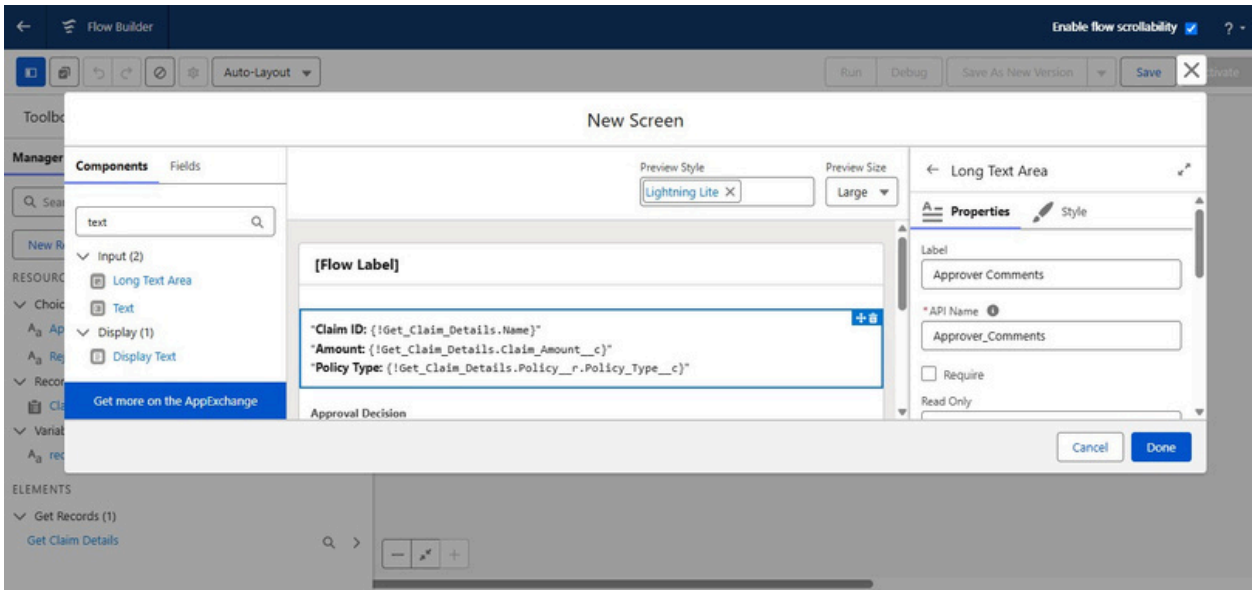


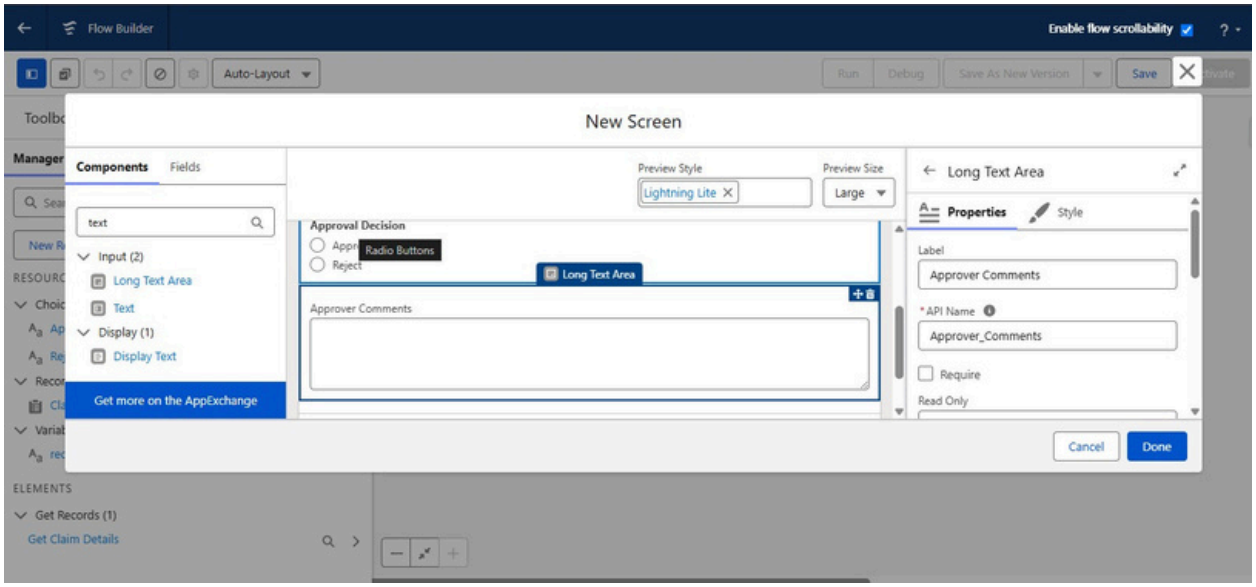




- Add a screen Element under the get Record element
- Label it as Claim Review Screen
- Add Display Text component to the flow canvas, and paste the following:

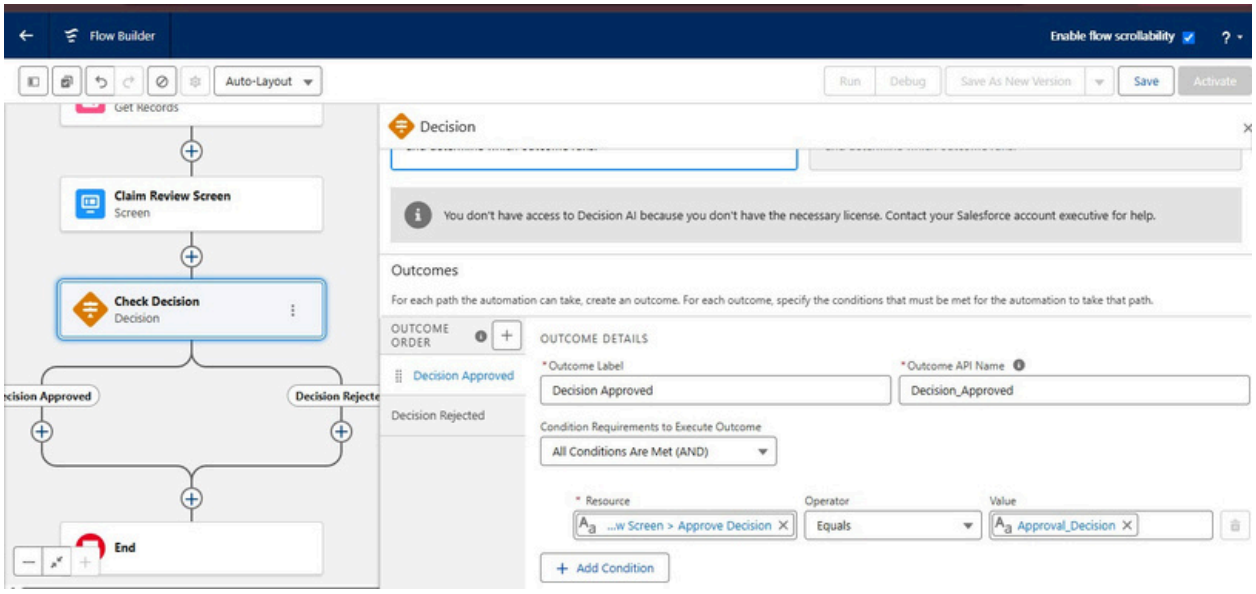
- "**Claim ID:** {!Get_Claim_Details.Name}"
- "**Amount:** {!Get_Claim_Details.Claim_Amount__c}"
- "**Policy Type:** {!Get_Claim_Details.Policy__r.RecordType.Name}"

- Add the Radio buttons component, Label it as Approval Decision
- API Name would be auto populated
- For Choice, Create a resource, Resource type would be choice
- Give the choice label as Approve
- Select Datatype as Text
- Enter the choice value as Approve
- Enter the input label as Approve
- Click on Done
- Add one more Choice, Create a resource, Resource type would be choice
- Give the choice label as Reject
- Select Datatype as Text
- Enter the choice value as Reject
- Enter the input label as Reject
- Click on Done
- Add Long Text area into the flow canvas and Label it as Approver Comments
- API Name would be Auto Populated
- Click on Done

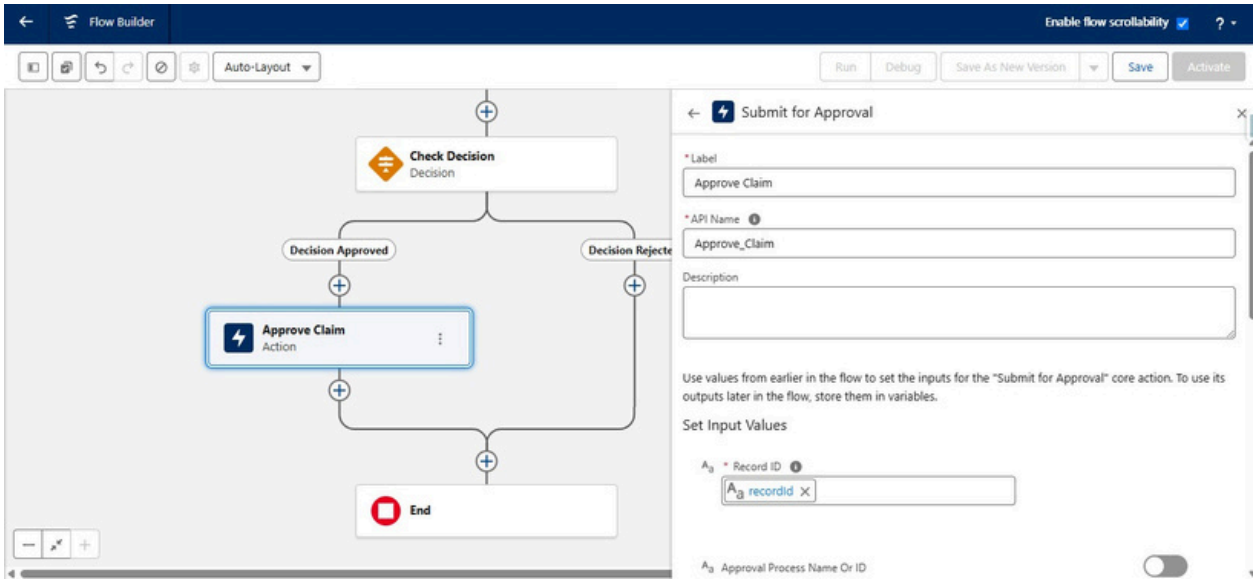


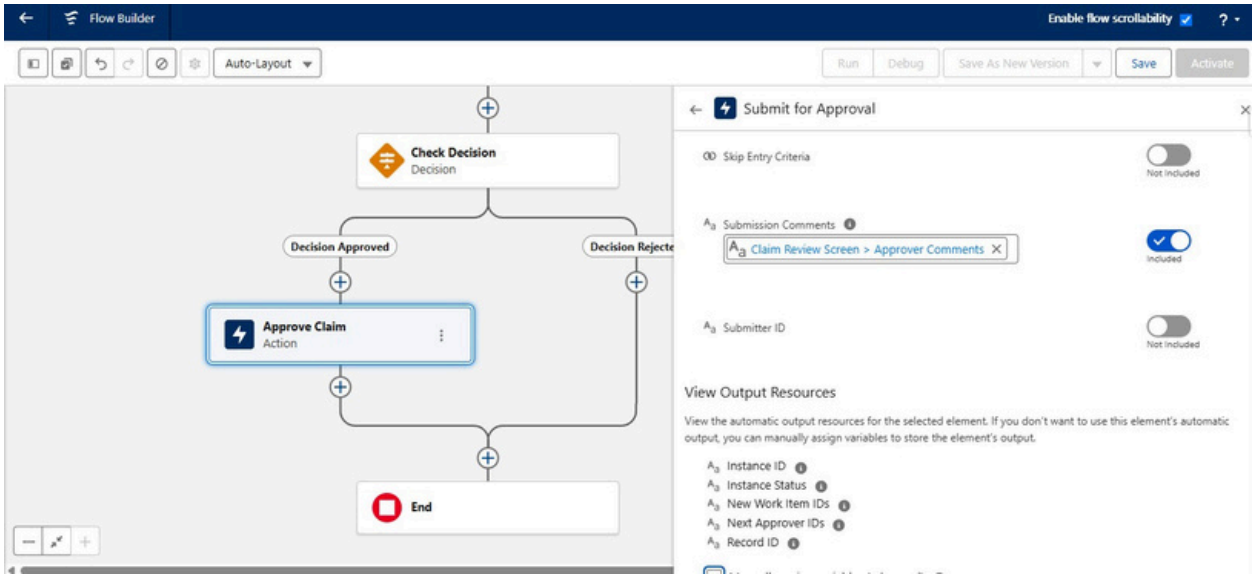



- Add Decision Element under the screen
- Label it as Check Decision
- Select Manually Select Conditions
- Set Outcome1:
    - Label it as Decision Approved
    - Select all conditions are met for conditional requirements
    - The condition includes:
    - Approve Decision From Claim Review Screen → Equals Approval Decision
- Set default Outcome:
    - Label it as Decision Rejected

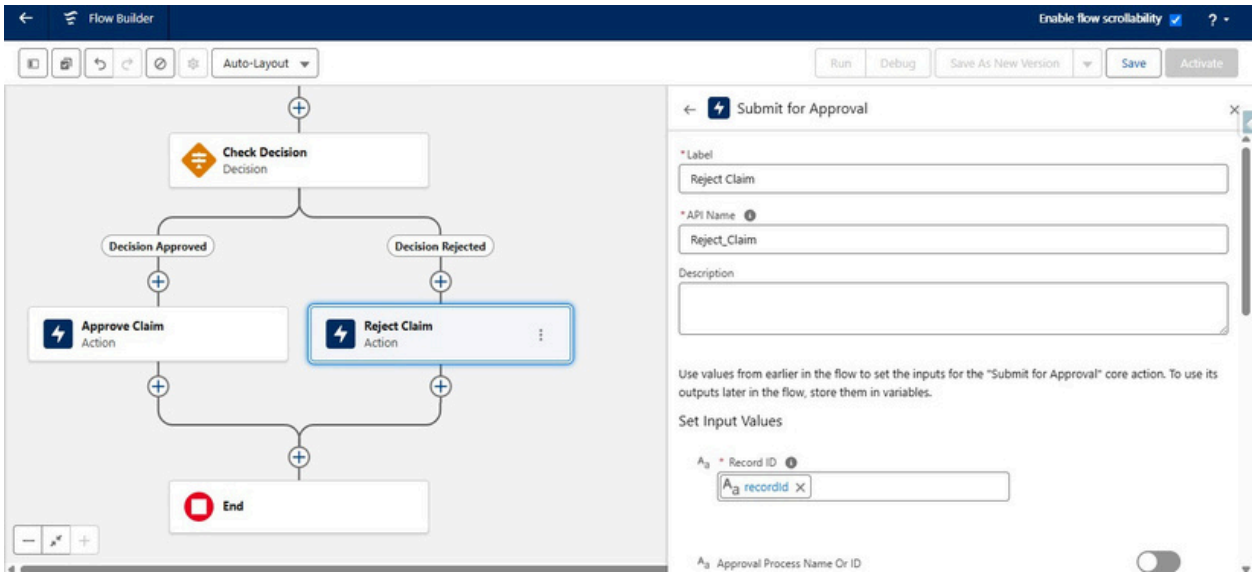


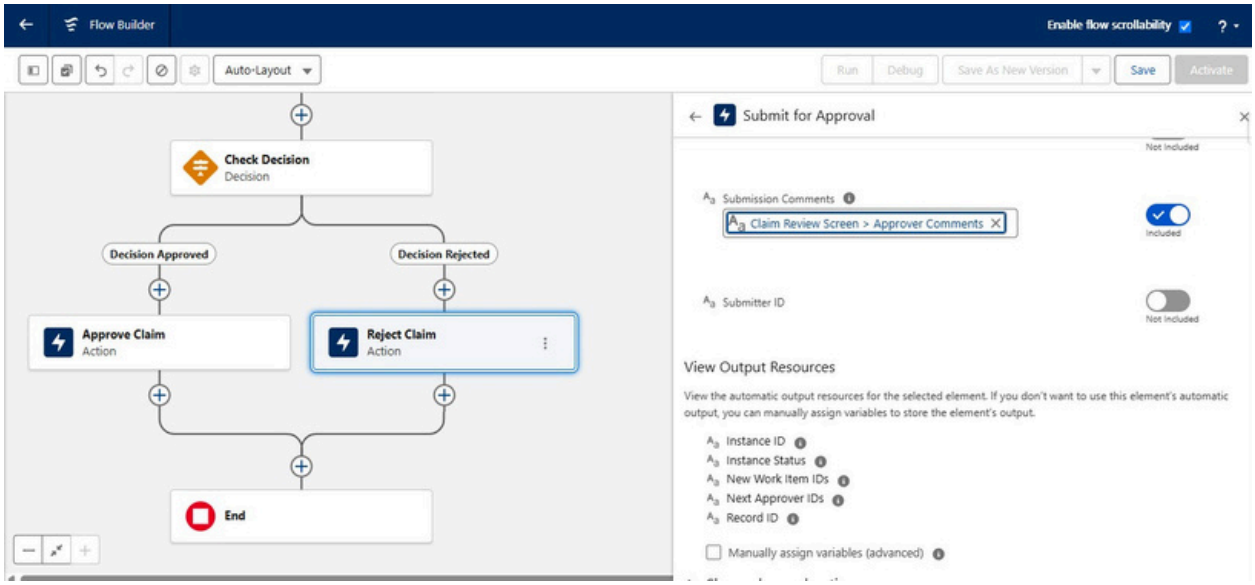

- Add Action Element Under Decision Approved Path
- Search for the submit for approval action and choose it
- Label it as Approve Claim
- Under Set Input Values
- Select variable recordID for Record ID
- Include the submission comments and Enter Approver Comments from Claim Review Screen






- Add Action Element Under Decision Rejected Path
- Search for the submit for approval action and choose it
- Label it as Reject Claim
- Under Set Input Values
- Select variable recordID for Record ID
- Include the submission comments and Enter Approver Comments from Claim Review Screen

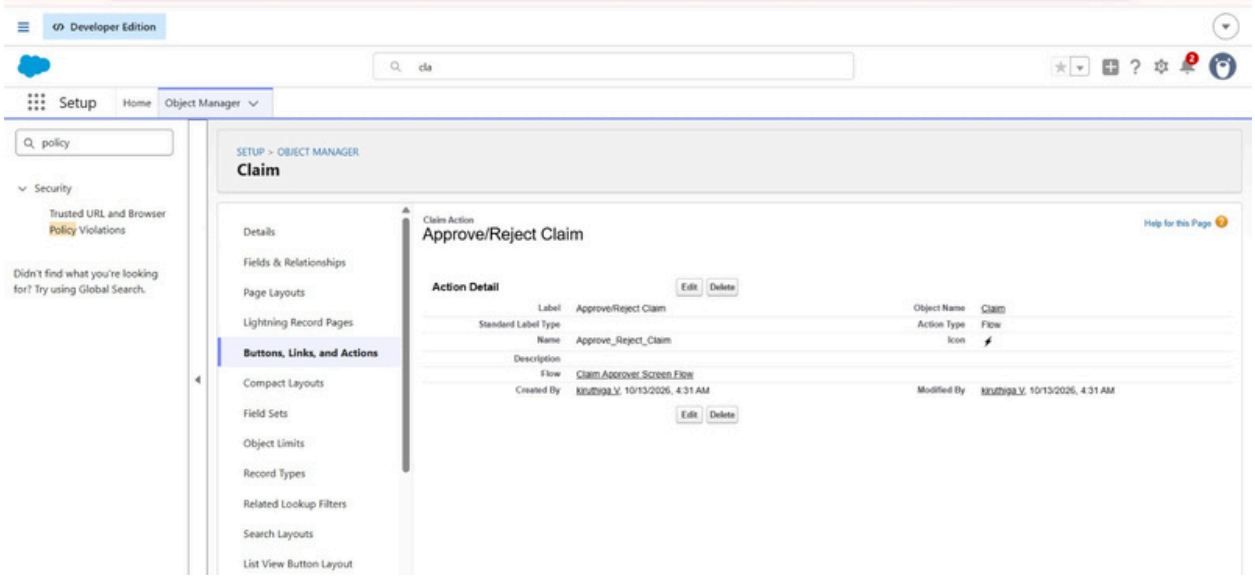




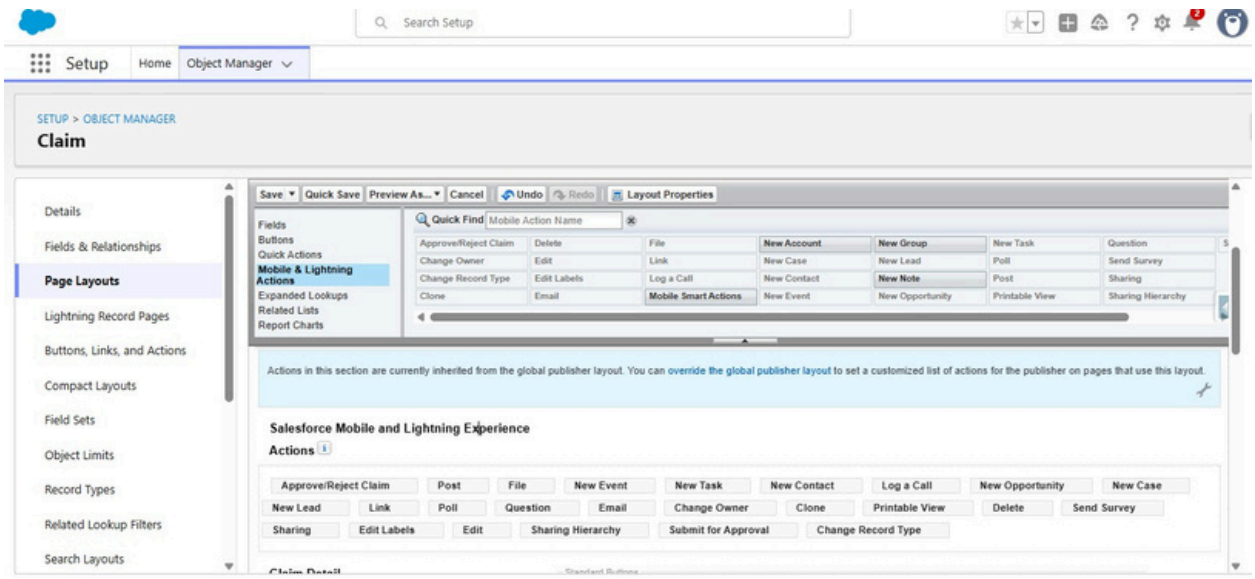

- Save and Activate the flow with Name Claim Approver Screen Flow

### 4.1 Create a Quick Action

- Go to setup, Click on Object Manager, Search for Claim Object and Select it
- In the left side panel, Click on Buttons, Links and Actions
- Click on Create New Action
- Select Action Type as Flow
- Select the flow that you have just created
- Label it as Approve/Reject Claim
- Name would be auto populated
- Click on Save




- Add the quick action to the claim page layout
- In the left panel, Click on page layouts and open the claim page layout
- Go to mobile & lightning actions, Find the Approve/Reject Claim and Drag it to the Salesforce Mobile and Lightning Experience Actions
- Click on Save



5. **Write an extensive Apex Test Class for the ClaimsAdjusterController with 95%+ code coverage. Include test methods for complex claim filtering and data retrieval logic.**
    - Click on the setup icon, Right click on Developer Consoleand click on open link in new tab.
    - On the top left corner, click on file, click on new, click on Apex class and give it a name ClaimsAdjusterControllerTest
    - Paste the following code:

```
@isTest

private class ClaimsAdjusterControllerTest {

    @testSetup

    static void setupTestData() {

        // Create a real active test user

        Profile p = [SELECT Id FROM Profile WHERE Name = 'Standard User' LIMIT 1];

        User u = new User(

            FirstName = 'Test',

            LastName = 'User',

            Alias = 'tuser',

            Username = 'testuser1234343@example.com.test',

            Email = 'testuser123@example.com',
```

```
            TimeZoneSidKey = 'America/Los_Angeles',

            LocaleSidKey = 'en_US',

            EmailEncodingKey = 'UTF-8',

            LanguageLocaleKey = 'en_US',

            ProfileId = p.Id

        );

        insert u;



        // Query Policy__c RecordType

        Id policyRT =
Schema.SObjectType.Policy__c.getRecordTypeInfosByDeveloperName()

            .get('Auto')

            .getRecordTypeId();

        Id claimRT =
Schema.SObjectType.Claim__c.getRecordTypeInfosByDeveloperName()

            .get('Accident')

            .getRecordTypeId();
```

```
        // Create Contact owned by same user
        Contact con = new Contact(
            FirstName = 'John',
            LastName = 'Doe',
            OwnerId = u.Id
        );
        insert con;

        // Create Policy owned by same user
        Policy__c policy = new Policy__c(
            Customer__c = con.Id,
            RecordTypeId = policyRT,
            OwnerId = u.Id,
            VIN__c = '1A903284375893532'
        );
        insert policy;
```

```
        // Create Claim owned by same user
        Claim__c claim = new Claim__c(
            Policy__c = policy.Id,
            RecordTypeId = claimRT,
            Claim_Amount__c = 5000,
            Date_of_Loss__c = Date.today().addDays(-10),
            OwnerId = u.Id
        );
        insert claim;
        System.debug('Claim inserted: '+claim.Id);
    }
    @isTest
    static void testGetAssignedClaims() {
        User u = [SELECT Id FROM User WHERE Alias = 'tuser' LIMIT 1];
        System.runAs(u) {
```

```
        Test.startTest();

        List<ClaimsAdjusterController.ClaimWrapper> results =

            ClaimsAdjusterController.getAssignedClaims();

        System.debug('Result is: '+results);

        Test.stopTest();

        //System.assert(results.size() > 0,

            //'Expected at least one wrapper returned');



        ClaimsAdjusterController.ClaimWrapper wrap = results[0];



        System.assertNotEquals(null, wrap.claimId);

        System.assertNotEquals(null, wrap.claimNumber);

        System.assertEquals('New', wrap.status);

        System.assertEquals(5000, wrap.claimAmount);

        System.assert(wrap.policyHolderName.contains('John'));

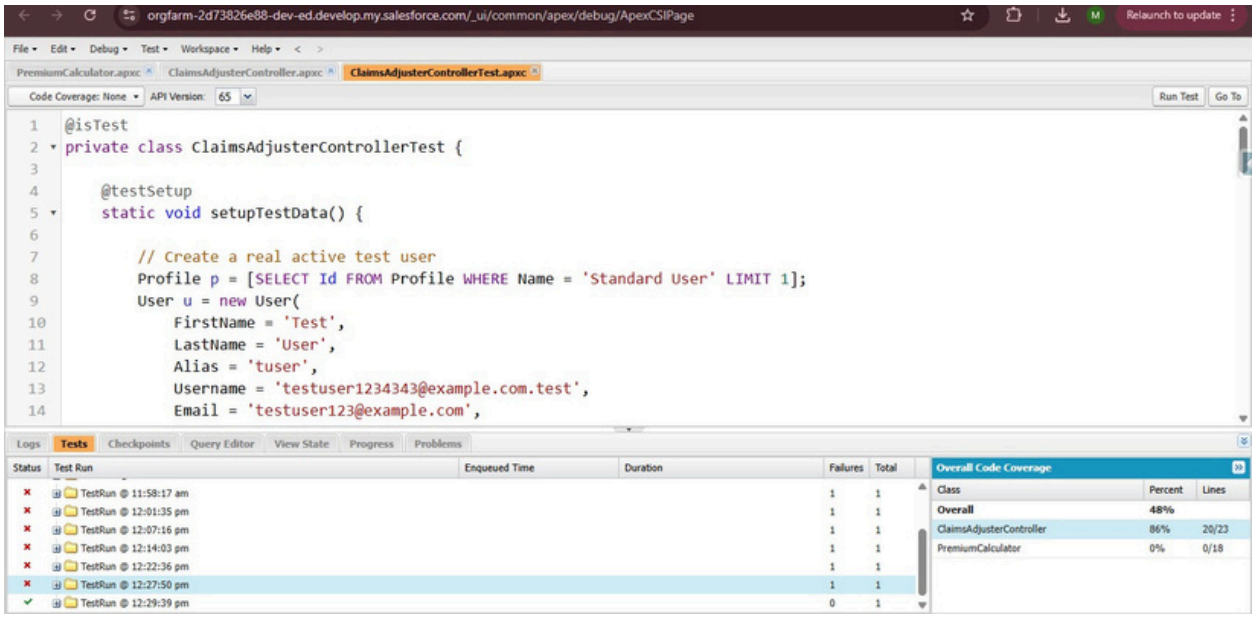
        System.assertNotEquals('Unknown', wrap.policyType);
```

```
    }

  }

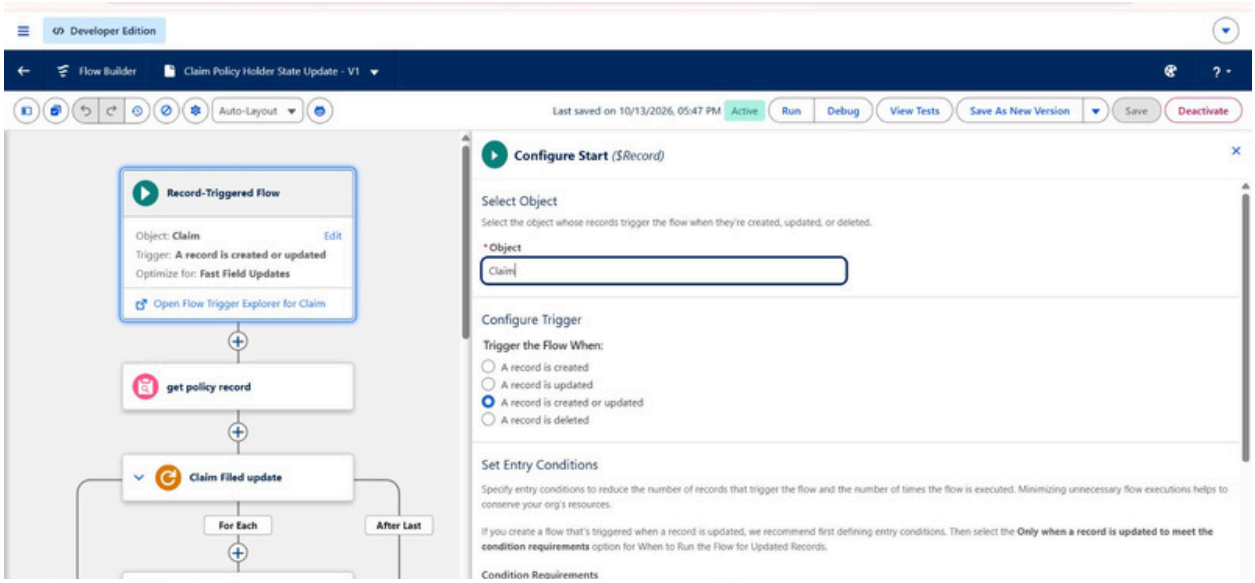
}
```

- Save the file using CTRL+S
- On the top right corner, you will find Run Test, Click on it to run the test class
- To check the test results, Down go for tests and you can see the how much percentage it covers your apex class

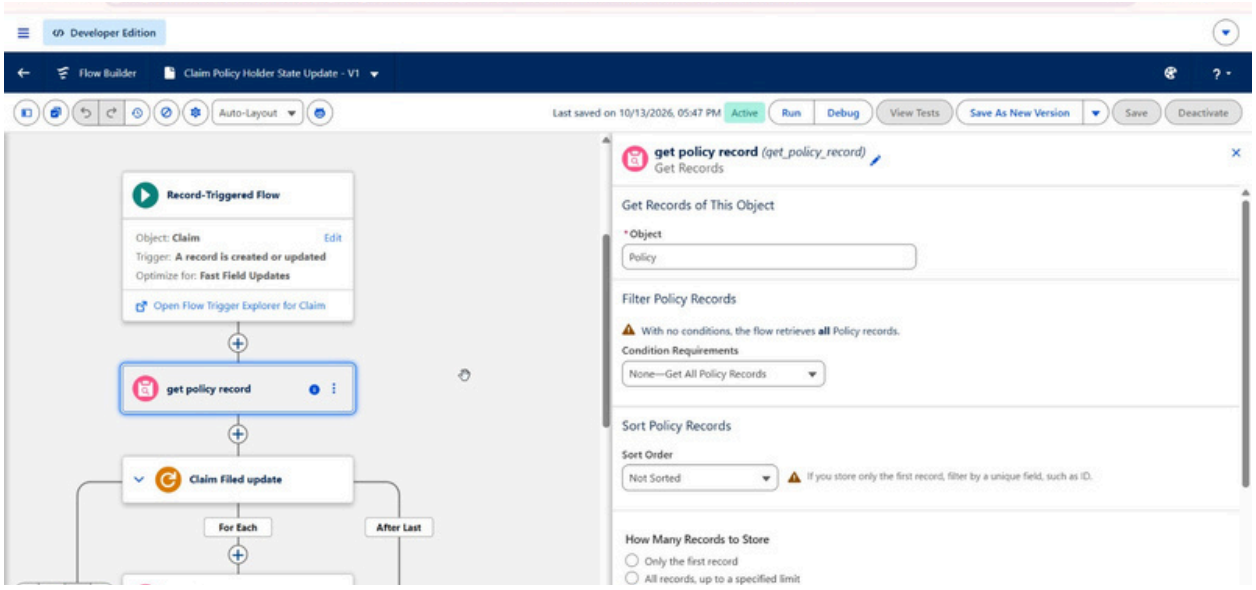


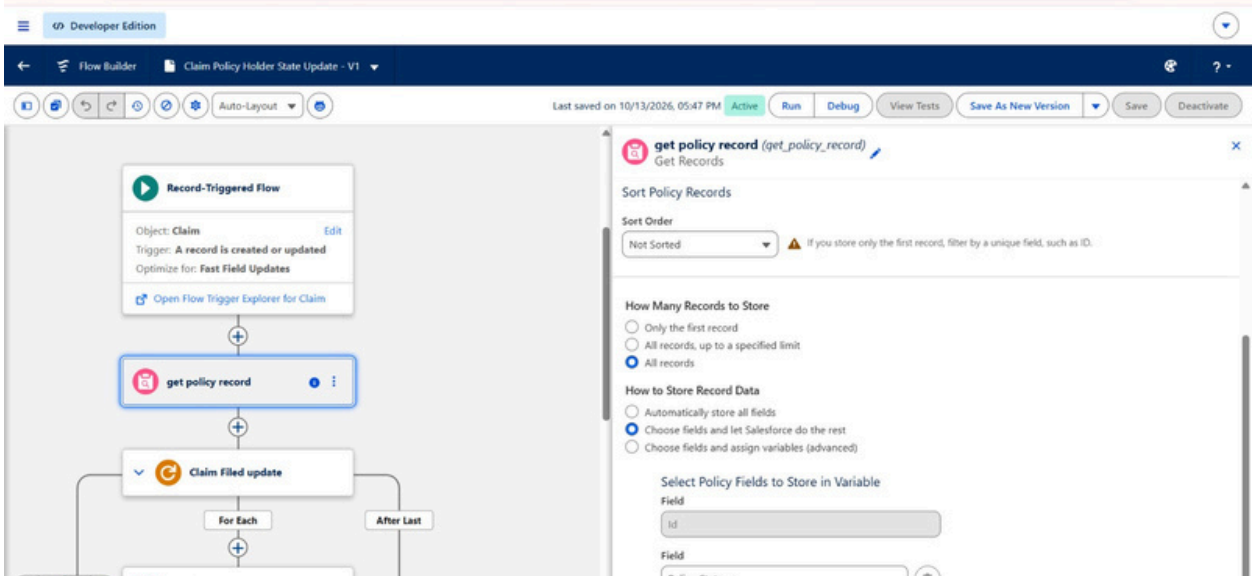

6. **Refine the Sharing Rules to ensure Adjusters only see claims in their assigned territory/state.**
   - Go to setup, Search for flows in the quick find box and click on flows
   - Click on New Flow
   - Select Record Triggered Flow
   - Select the object as claim
   - Configure trigger when a record is created or updated

- Set entry conditions as none
- Optimize the flow for Updates

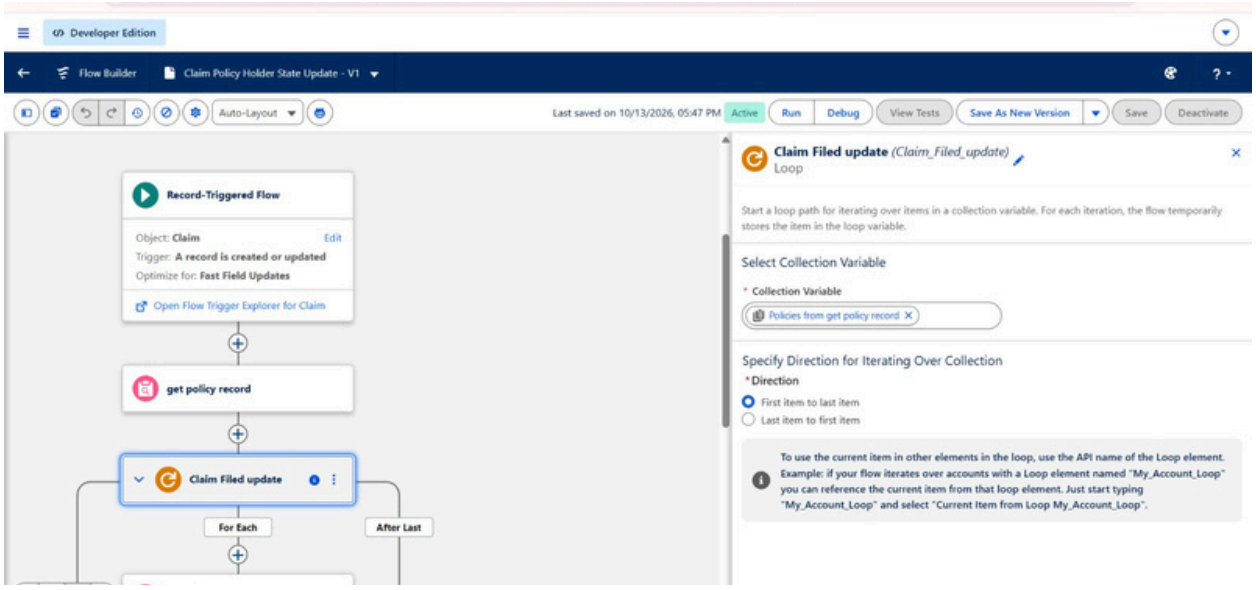



- Add get Records element into the flow canvas
- Label it as get_policy_records
- API name will be auto populated
- Set filter as None
- Store all the records
- Select Choose fields and let salesforce do the rest for How to store Record Data
- Under Select Policy fields to store fields in Variable, Select Policy State Field

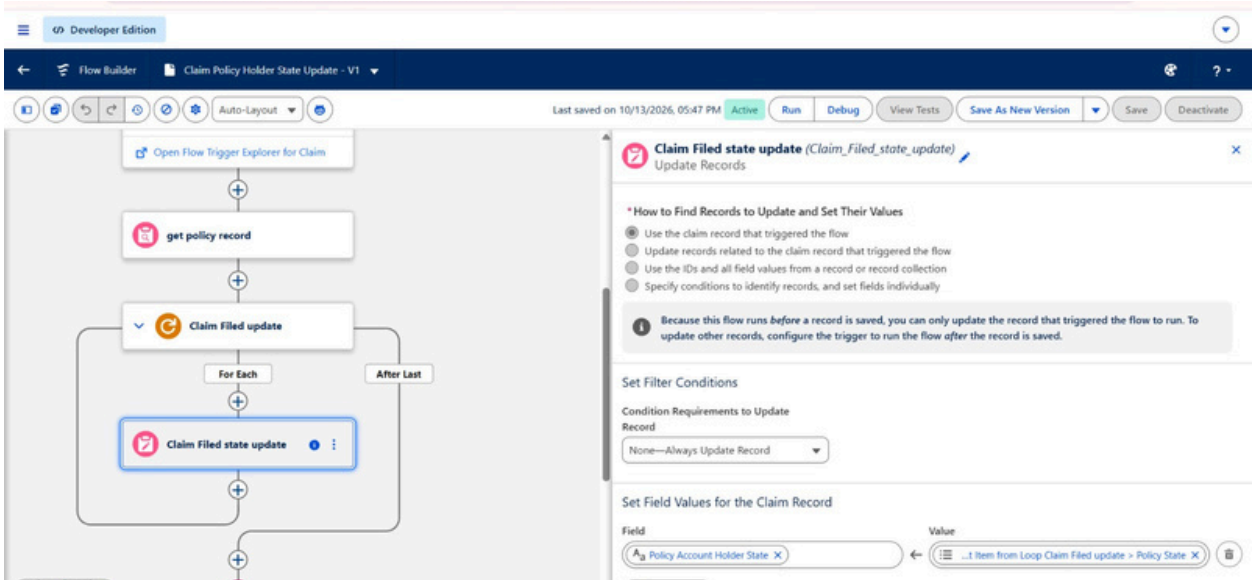



- Add the Loop Element into the canvas after the get Records element
- Label it as Claim_Filed _update
- API name will be auto populated
- Under Select Collection Variables, Select policies from Get Policies from Get Policy Records
- Under Specify direction for Iterating over collection, Select first item to last item



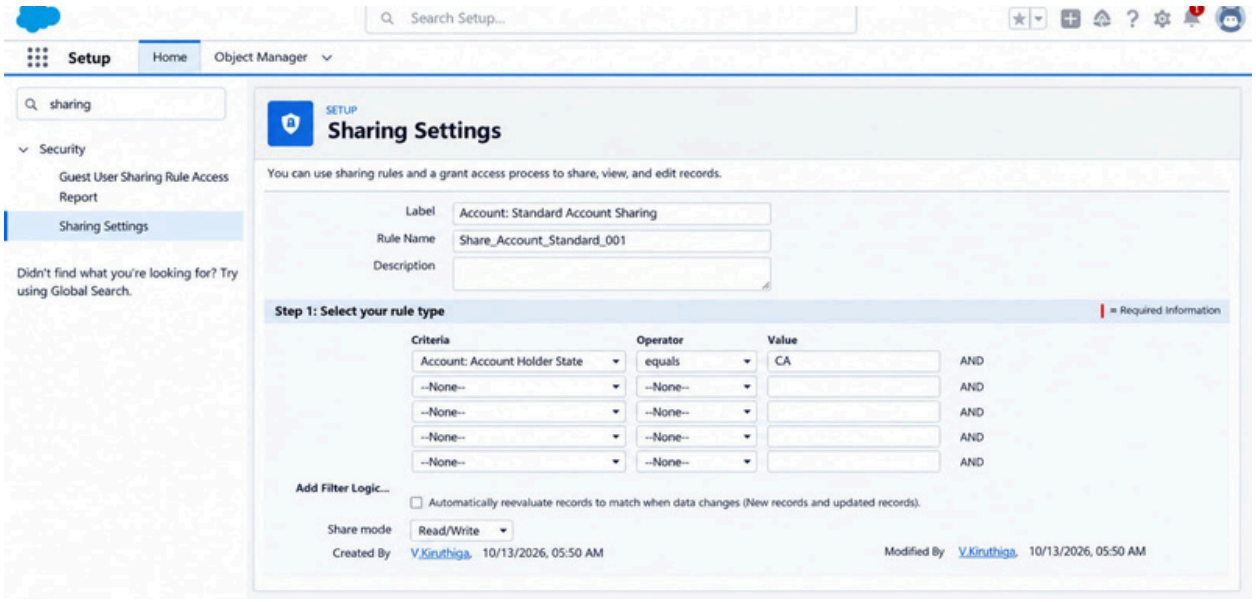

- Under For each, Add the update records element into the canvas
- Label it as Claim_Filed _state_update
- API name will be auto populated
- Set filter conditions to NONE
- Under Set field values for the claim record, Select policy state from current loop and set it to the Policy Account Holder State field



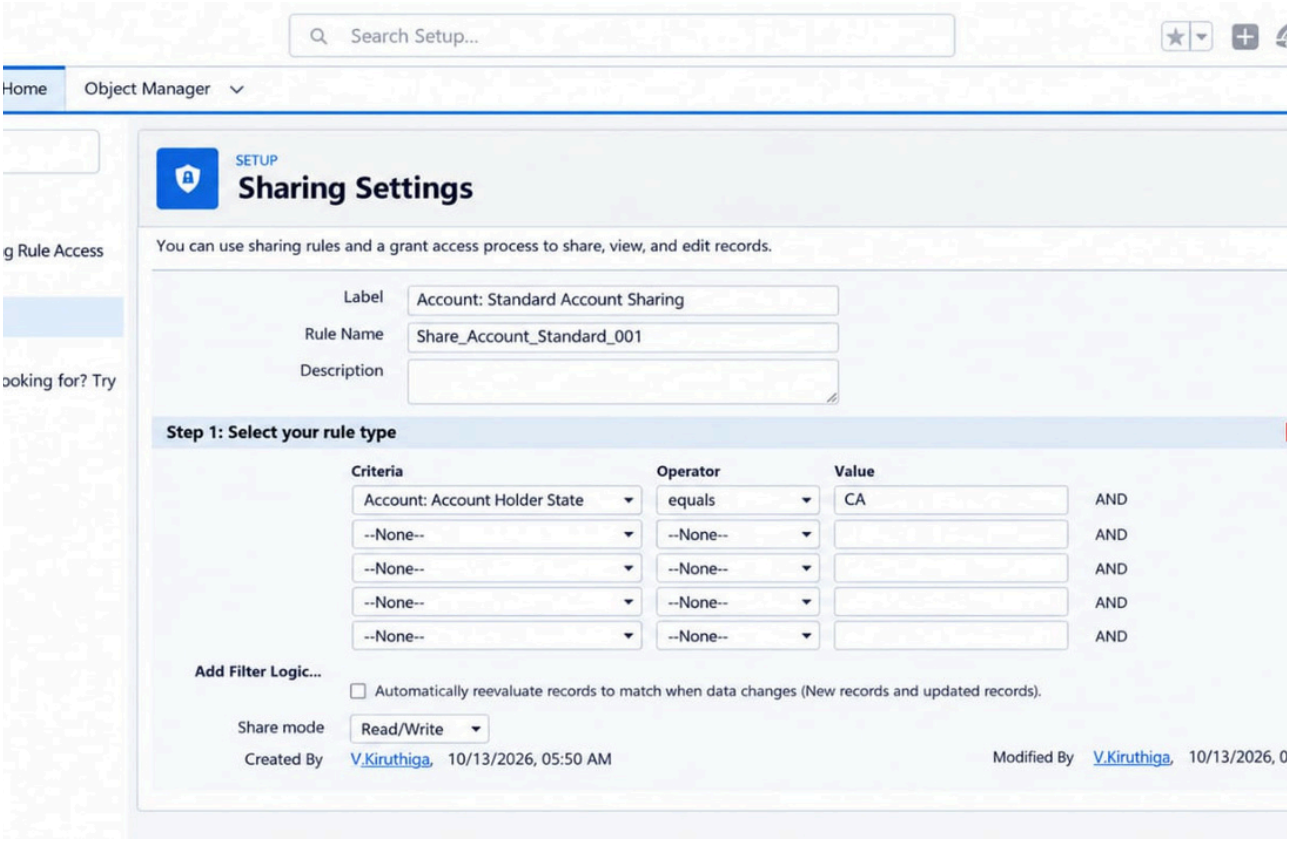
- Save and Activate the flow with name Claim Policy Holder State Update

## 6.1 Define Sharing Rules for Claim Object

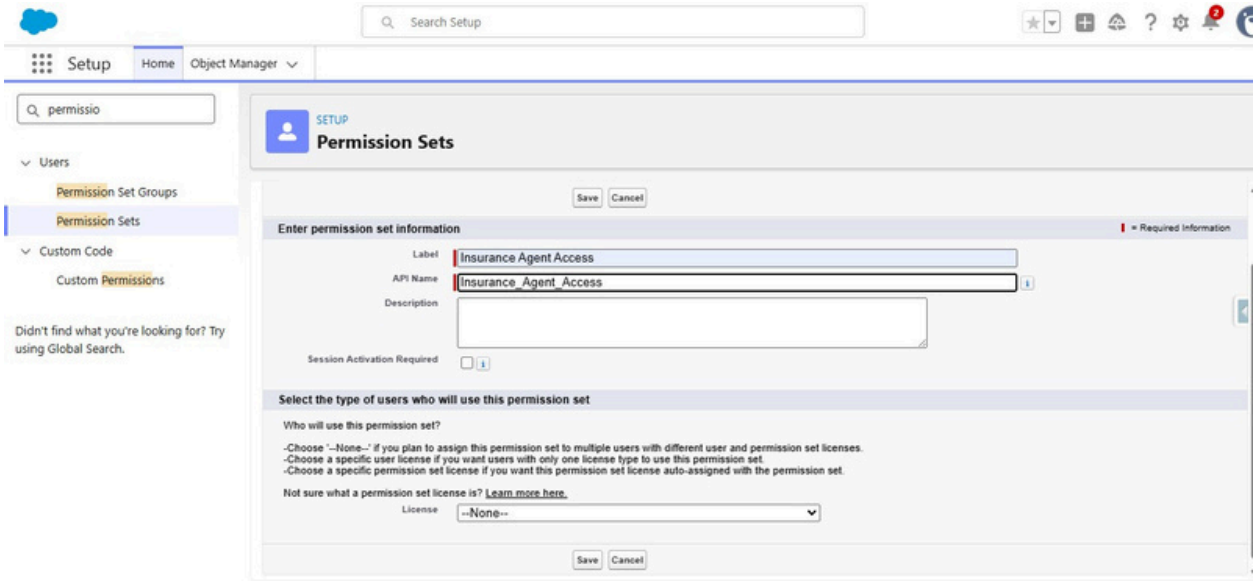
- Go to Set up, In the quick find box, Search for Sharing Settings
- Click on Sharing settings
- Under Manage Sharing Settings for, Select Claim Object
- Scroll down to find Claim Sharing Rules
- Click on new
- Label it as Share Records based on Policy State CA
- Rule Name will be Auto Populated
- Set the criteria to Policy Account Holder State Equals CA
- Give Access Level as Read/Write
- Click on Save




- Again Click on New
- Label it as Share Records based on Policy State
- Rule Name will be Auto Populated
- Set the criteria to Policy Account Holder State Equals TX
- Give Access Level as Read/Write
- Click on Save



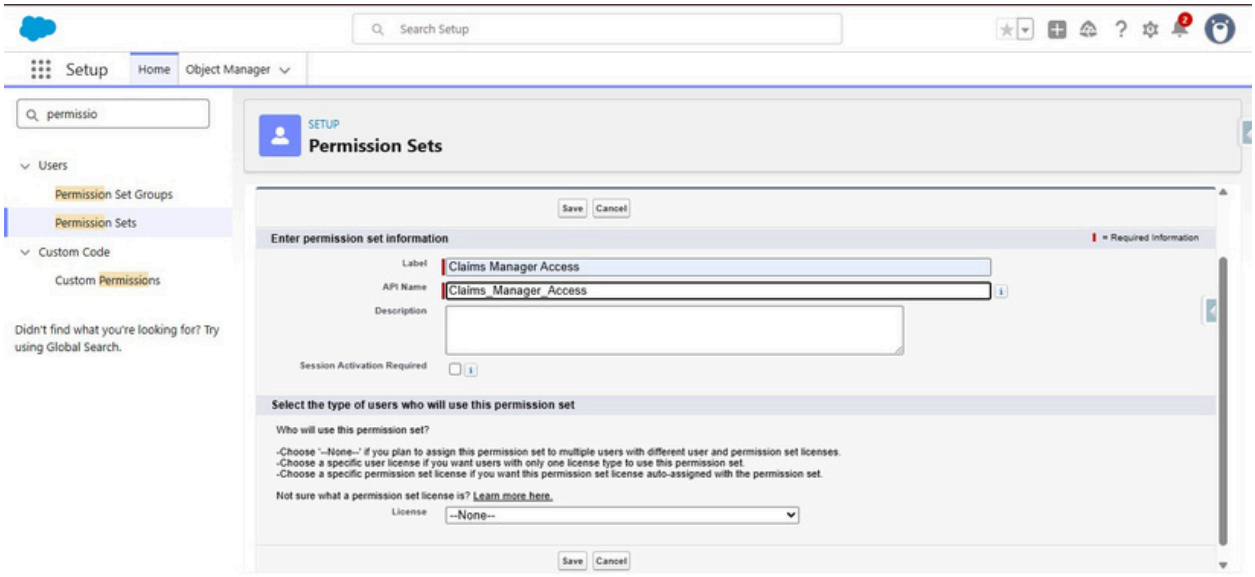

7. **Finalize all Permission Sets to differentiate access between Agents (Quoting only), Adjusters (Claim handling), and Managers (Reporting/Approvals).**
    - Go to setup, In the quick findbox,SearchforPermissionSets, Click on Permission Sets
    - Click on New
    - Give the Label as Insurance Agent Access
    - API Name will be Auto Populated




- Give the access for Objects
- Give Create, Read for Policy object
- Give Read, Edit to contact Object
- Give Read only to Claim object
- Make Flow User Enabled under System permissions
- Make PremiumCalculator Apex class enabled

### 7.1 Create Claims Manager Access Permission set

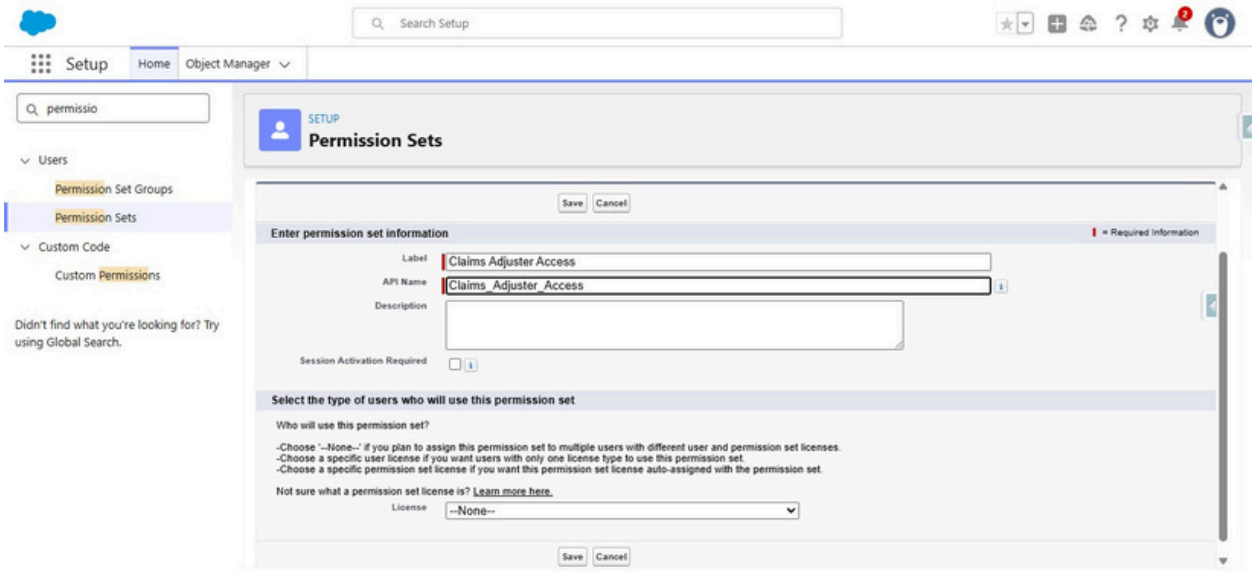
- Click on New
- Give the Label as Claims Manager Access
- API Name will be Auto Populated




- Give the access to objects
- Give Read Only for Policy object
- Give Read, Edit to contact Object
- Give Read, Edit, Delete to Claim object
- Make Run Reports Enabled under System permissions
- Make Manage Approvals enabled under system permissions

### 7.2 Create Claims Adjuster Access Permission Set

- Click on New
- Give the Label as Claims Adjuster Access
- API Name will be Auto Populated




- Give the access to objects
- Give Read Only for Policy object
- Give Read, Edit to Claim object
- Make ClaimsAdjusterController Enabled under Apex Classes

## 7. FUNCTIONAL & PERFORMANCE TESTING

During testing, screenshots were captured from Salesforce to validate vehicle creation, dealer assignment, stock validation triggers, order status updates, batch processing, scheduled reminders, and dashboard analytics. All features were verified to ensure correct functionality and performance.

## 8. ADVANTAGES & DISADVANTAGES

### Advantages

- Centralized Platform – Manages Vehicle, Property, and Life Insurance in one Salesforce system.
- Automated Quoting – Screen Flows simplify and standardize policy quoting.
- Automated Premium Calculation – Apex calculates premiums consistently.

- Automated Claim Routing – Claims are routed to the appropriate queue based on policy type.
- Faster Claim Processing – Approval and escalation processes reduce manual effort.
- Better Visibility – LWC Claims Dashboard gives adjusters a focused view of assigned claims.
- Improved Data Accuracy – Validation rules help maintain accurate policy information.
- Flexible Data Model – Record Types and Field Sets support different insurance products.
- Secure Access – Permission Sets and Sharing Rules support role- and state-based access.
- Scalable Solution – The architecture can support multiple insurance lines and future enhancements.

These advantages are supported by the project's stated objectives and solution approach.

**Disadvantages**

- Complex Data Management – Vehicle, Property, and Life policies require different data fields.
- Approval Complexity – Claim approval must consider claim amount, policy type, and adjuster level.
- User Adoption – Agents need intuitive flows to minimize clicks and improve efficiency.
- Integration Complexity – External database callout simulation requires additional Apex and mock test classes.
- Salesforce Dependency – The solution depends on Salesforce configuration and platform capabilities.
- Implementation Effort – Multiple Flows, Apex classes, LWC components, security rules, and approval processes require careful development and maintenance.

The first four challenges are explicitly identified in the project document; the Salesforce dependency and implementation effort are reasonable project-level limitations inferred from the solution's technical scope.

## 9. Conclusion:

The Multi-Line Insurance Policy and Claims Management System provides a centralized Salesforce-based solution for managing Vehicle, Property, and Life Insurance policies and claims. The project addresses the limitations of manual and fragmented processes by introducing guided quoting, automated premium calculation, claim routing, structured approval processes, and an LWC-based Claims Dashboard.

The solution uses a combination of Salesforce Flows, Apex, Record Types, Field Sets, Validation Rules, Approval Processes, LWC, Permission Sets, and Sharing Rules to improve policy processing, claim handling, data accuracy, security, and operational visibility.

Overall, the project demonstrates how Salesforce can bring multiple insurance processes together on a single scalable platform, helping insurance teams work more efficiently while providing better visibility and service to customers.

## 10. FUTURE SCOPE

The Multi-Line Insurance Policy and Claims Management System can be further enhanced in the following areas:

- External System Integration – Integrate with external vehicle and property databases for real-time quote verification.
- Additional Insurance Products – Extend the solution to support additional insurance lines beyond Vehicle, Property, and Life.

- Advanced Claims Automation – Introduce more automated claim assessment and escalation based on claim characteristics.
- Enhanced Analytics – Provide advanced dashboards and reports for policy, premium, and claims analysis.
- Improved Customer Experience – Introduce customer-facing capabilities for policy and claim status tracking.
- Mobile Accessibility – Enable agents and claims users to access key Salesforce processes from mobile devices.
- AI-Based Assistance – Explore AI capabilities to assist agents and adjusters with policy and claims-related tasks.
- Advanced Integration Architecture – Expand API-based integrations with external insurance and verification systems.

## 11. APPENDIX

- Objects: Policy, Claim, Contact, User
- Record Types: Vehicle, Property, Life; Accident, Property, Life
- Automation: Screen Flows, Record-Triggered Flows, Approval Processes
- Apex: PremiumCalculator, ClaimsAdjusterController
- LWC: Claims Dashboard, Claim Tile
- Validation: 17-character VIN validation
- Security: Permission Sets, State-Based Sharing Rules
- Testing: ClaimsAdjusterControllerTest
- Reporting: Claims Dashboard & Salesforce Reports/Dashboards

# Thank You